

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN

-----□□&□□-----



Báo Cáo Đồ Án Thiết Kế Kiến Trúc Hệ Thống

Nhóm 3:

Ngô Việt Hoàng-0005667-67PM2

Phạm Hồng Kỳ-0157667-67PM2

Nguyễn Đắc Điền-0004667-67PM2

Lê Đức Hải-0047667-67PM2

Nguyễn Văn Đạo-0003967-67PM2

Giảng viên hướng dẫn: Phạm Hữu Tùng

Mục Lục

Chương 1: Giới thiệu đề tài	3
1.1 Lý do chọn đề tài.....	3
1.2 Mục tiêu đề tài.....	3
1.3 Phạm vi và giới hạn	4
1.4 Phương pháp thực hiện	5
Chương 2: Phân tích và thiết kế hệ thống.....	6
2.1 Mô tả bài toán.....	6
2.2 Yêu cầu chức năng	7
2.3 Yêu cầu phi chức năng.....	9
2.4 Biểu đồ use case tổng quát.....	11
2.5 Mô tả use case	16
2.7 Biểu đồ activity	49
2.8 Biểu đồ sequence	59
Chương 3: Cài đặt và triển khai hệ thống.....	70
3.1 Sơ đồ kiến trúc hệ thống.....	70
3.2 Thiết kế lớp	71
3.2.1 Chức năng gửi tin nhắn.....	71
3.2.2 Quản lý cuộc hội thoại	102
3.2.3 Quản lý người dùng	115
3.2.4 Chức năng xác thực	115
3.2.5 Quản lý trang cá nhân	116
3.2.6 Gợi ý tin nhắn bằng AI.....	116
3.3 Thiết kế cơ sở dữ liệu	121
3.4 Thiết kế giao diện	122
Chương 4: Kết quả và đánh giá	132
4.1 Môi trường triển khai	132
4.2 Cài đặt hệ thống	133
4.2.1 Yêu cầu hệ thống.....	133
4.2.2 Các bước cài đặt chương trình	134

Chương 5: Kết quả và đánh giá	134
5.1 Kết quả kiểm thử hệ thống	141
5.2 Đánh giá hệ thống.....	142
Tài liệu tham khảo	144

Chương 1: Giới thiệu đề tài

1.1 Lý do chọn đề tài

Trong môi trường làm việc hiện đại, việc trao đổi thông tin nhanh chóng và chính xác giữa các thành viên trong tổ chức đóng vai trò rất quan trọng. Các doanh nghiệp và nhóm làm việc ngày càng có nhu cầu sử dụng những công cụ giao tiếp trực tuyến nhằm nâng cao hiệu quả phối hợp công việc, giảm thời gian trao đổi và lưu trữ thông tin một cách tập trung.

Xuất phát từ nhu cầu đó, đề tài “Xây dựng hệ thống chat nội bộ trên nền tảng web” được thực hiện nhằm phát triển một ứng dụng hỗ trợ giao tiếp giữa các thành viên trong cùng một tổ chức hoặc nhóm làm việc. Đồng thời, đề tài cũng là cơ hội để nghiên cứu và ứng dụng các công nghệ phát triển web hiện đại như Laravel và React TypeScript vào xây dựng một hệ thống thực tế.

1.2 Mục tiêu đề tài

Mục tiêu lý thuyết

Nắm vững quy trình phát triển phần mềm từ giai đoạn khảo sát, phân tích yêu cầu, thiết kế hệ thống, triển khai đến kiểm thử và đánh giá sản phẩm.

Củng cố kiến thức về phát triển ứng dụng web theo mô hình Client - Server.

Vận dụng các kiến thức về phân tích và thiết kế hệ thống hướng đối tượng thông qua các biểu đồ Use Case, Sequence Diagram, Class Diagram và thiết kế cơ sở dữ liệu.

Tìm hiểu và ứng dụng framework Laravel trong phát triển Backend API cùng React TypeScript (TSX) trong phát triển giao diện người dùng hiện đại.

Mục tiêu thực tiễn

Xây dựng thành công một hệ thống chat nội bộ trên nền tảng web phục vụ nhu cầu trao đổi thông tin giữa các thành viên trong doanh nghiệp hoặc nhóm làm việc, với các chức năng chính dành cho hai đối tượng là Người dùng (User) và Quản trị viên (Admin), cụ thể:

Đối với Người dùng (User)

Đăng nhập và quản lý thông tin cá nhân.

Tìm kiếm người dùng trong hệ thống và tạo cuộc trò chuyện.

Gửi và nhận tin nhắn theo thời gian thực.

Tạo nhóm trò chuyện và tham gia trao đổi thông tin với nhiều thành viên.

Gửi hình ảnh, tài liệu và các tệp tin phục vụ công việc.

Xem lịch sử tin nhắn và nhận thông báo khi có tin nhắn mới.

Đối với Quản trị viên (Admin)

Quản lý tài khoản người dùng trong hệ thống.

Theo dõi và quản lý các nhóm trò chuyện.

Quản lý nội dung, xử lý các trường hợp vi phạm quy định sử dụng hệ thống.

Thống kê số lượng người dùng, số lượng cuộc trò chuyện và các hoạt động trên hệ thống.

Đảm bảo hệ thống hoạt động ổn định, an toàn và đáp ứng nhu cầu trao đổi thông tin nội bộ của tổ chức.

1.3 Phạm vi và giới hạn

Phạm vi nghiên cứu:

Đề tài tập trung vào việc xây dựng một hệ thống chat nội bộ trên nền tảng web phục vụ nhu cầu trao đổi thông tin giữa các thành viên trong doanh nghiệp hoặc nhóm làm việc. Hệ thống được phát triển bằng Laravel cho phần Backend và React TypeScript (TSX) cho phần Frontend. Hệ thống hỗ trợ giao tiếp trực tuyến theo thời gian thực, giúp nâng cao hiệu quả trao đổi thông tin và phối hợp công việc giữa các thành viên.

Các chức năng trong phạm vi hệ thống:

Hệ thống được xây dựng với hai phân hệ chính là Người dùng (User) và Quản trị viên (Admin). Đối với Người dùng, hệ thống hỗ trợ các chức năng đăng ký, đăng nhập, quản lý thông tin cá nhân, tìm kiếm người dùng, tạo và quản lý các cuộc trò chuyện cá nhân hoặc nhóm. Người dùng có thể gửi và nhận tin nhắn theo thời gian thực, gửi hình ảnh và tệp tin đính kèm, xem lịch sử trò chuyện, nhận thông báo khi có tin nhắn mới cũng như thực hiện các thao tác quản lý tin nhắn như chỉnh sửa hoặc thu hồi (nếu được hỗ trợ).

Đối với Quản trị viên, hệ thống cung cấp các chức năng quản lý tài khoản người dùng, quản lý các nhóm trò chuyện, theo dõi và xử lý các nội dung vi phạm quy định sử dụng hệ thống. Ngoài ra, quản trị viên có thể thống kê các hoạt động trên hệ thống như số lượng người dùng, số lượng cuộc trò chuyện và các thông tin cần thiết để giám sát quá trình vận hành của hệ thống.

Giới hạn của đề tài

Do giới hạn về thời gian thực hiện và nguồn lực phát triển, đề tài tập trung chủ yếu vào các chức năng nhắn tin và quản lý người dùng cơ bản. Hệ thống chưa tích hợp các tính năng nâng cao như gọi thoại, gọi video, hội nghị trực tuyến hoặc chia sẻ màn hình. Bên cạnh đó, các công nghệ trí tuệ nhân tạo như chatbot tự động, trợ lý ảo hay gợi ý nội dung thông minh cũng chưa được triển khai trong phạm vi đề tài. Hệ thống được xây dựng hướng đến môi trường doanh nghiệp hoặc nhóm làm việc có quy mô nhỏ và vừa, do đó chưa được tối ưu cho các hệ thống có số lượng người dùng đồng thời rất lớn. Ngoài ra, hệ thống hiện chưa hỗ trợ tích hợp với các nền tảng làm việc và cộng tác của bên thứ ba như Microsoft Teams, Slack hoặc Google Workspace.

1.4 Phương pháp thực hiện

1. Phương pháp khảo sát và thu thập yêu cầu

Nhóm tiến hành khảo sát nhu cầu trao đổi thông tin trong môi trường doanh nghiệp và nhóm làm việc, đồng thời tham khảo các hệ thống nhắn tin phổ biến hiện nay như Microsoft Teams, Slack và Messenger để xác định các chức năng cần thiết. Từ quá trình khảo sát, nhóm xây dựng danh sách các yêu cầu chức năng như quản lý người dùng, nhắn tin cá nhân, trò chuyện nhóm, gửi tệp tin, thông báo thời gian thực và quản lý hệ thống.

2. Phương pháp phân tích và thiết kế hệ thống

Sử dụng ngôn ngữ mô hình hóa UML để phân tích và thiết kế hệ thống.

Thiết kế biểu đồ Use Case nhằm xác định các tác nhân và các chức năng mà hệ thống cung cấp.

Thiết kế biểu đồ Sequence Diagram để mô tả luồng xử lý của các chức năng chính như đăng nhập, gửi tin nhắn, tạo nhóm trò chuyện và gửi tệp tin.

Thiết kế biểu đồ Class Diagram để mô hình hóa các đối tượng và mối quan hệ trong hệ thống.

Thiết kế cơ sở dữ liệu nhằm lưu trữ thông tin người dùng, cuộc trò chuyện, tin nhắn, nhóm trò chuyện, tệp đính kèm và các dữ liệu liên quan khác.

3. Phương pháp cài đặt và kiểm thử

Sử dụng Laravel để xây dựng hệ thống Backend và các API phục vụ trao đổi dữ liệu.

Sử dụng React TypeScript (TSX) để xây dựng giao diện người dùng và các chức năng tương tác trên nền tảng web.

Sử dụng cơ sở dữ liệu để quản lý và lưu trữ dữ liệu của hệ thống.

Triển khai các chức năng nhắn tin thời gian thực, quản lý người dùng và quản lý nhóm trò chuyện.

Thực hiện kiểm thử các chức năng của hệ thống nhằm đảm bảo tính chính xác, ổn định và đáp ứng các yêu cầu đã đề ra.

Đánh giá kết quả thực hiện, khắc phục các lỗi phát sinh và hoàn thiện sản phẩm trước khi đưa vào sử dụng.

Chương 2: Phân tích và thiết kế hệ thống

2.1 Mô tả bài toán

Bối cảnh nghiên cứu Trong kỷ nguyên chuyển đổi số, nhu cầu trao đổi thông tin nội bộ một cách liên tục, an toàn và nhanh chóng là yếu tố sống còn đối với mọi tổ chức và doanh nghiệp. Sự giao tiếp hiệu quả giữa các cá nhân và phòng ban không chỉ nâng cao năng suất lao động mà còn quyết định sự trơn tru trong quy trình vận hành. Đi cùng với sự phát triển đó, các hệ thống nhắn tin tức thời (Instant Messaging) đã trở thành một thành phần hạ tầng không thể thiếu.

Những hạn chế của các giải pháp hiện tại Hiện nay, phần lớn người dùng và các tổ chức vừa và nhỏ đang tận dụng các nền tảng chat công cộng đa dụng như Messenger, Zalo, hay Telegram. Tuy nhiên, việc phụ thuộc vào các nền tảng của bên thứ ba tồn tại nhiều điểm bất cập khi áp dụng vào môi trường doanh nghiệp chuyên nghiệp:

- **Rủi ro bảo mật và rò rỉ dữ liệu:** Toàn bộ lịch sử trao đổi, tệp tin nội bộ và thông tin nhạy cảm đều được lưu trữ trên máy chủ của đơn vị cung cấp dịch vụ, gây ra nguy cơ vi phạm bảo mật hoặc mất quyền kiểm soát dữ liệu.
- **Hạn chế trong việc tùy biến và tích hợp:** Các nền tảng này thường đóng gói dưới dạng dịch vụ (SaaS) với hệ sinh thái khép kín, rất khó để tùy biến sâu hoặc tích hợp luồng dữ liệu trực tiếp vào các hệ thống quản trị nội bộ (ERP, HRM) có sẵn của tổ chức.

- **Chưa tối ưu cho công việc chuyên sâu:** Thiếu các công cụ hỗ trợ đặc thù cho giới kỹ thuật và làm việc nhóm như định dạng mã nguồn (Code Block), Markdown, hoặc quản trị tài khoản tập trung theo mô hình phân cấp nội bộ.

Mục tiêu của hệ thống đề xuất Xuất phát từ các vấn đề thực tiễn trên, đề tài hướng tới việc xây dựng và triển khai một **Hệ thống Chat Messenger nội bộ hoàn chỉnh**, có khả năng tự lưu trữ (Self-hosted) nhằm đảm bảo chủ quyền dữ liệu (Data Sovereignty). Hệ thống được thiết kế nhằm tới các mục tiêu cốt lõi:

1. **Trải nghiệm người dùng liền mạch:** Ứng dụng kiến trúc Single Page Application (SPA) với React JS và Inertia.js v2, kết hợp giao diện hiện đại (Tailwind CSS, DaisyUI) giúp thao tác mượt mà, không gián đoạn do tải lại trang.
2. **Tương tác thời gian thực (Real-time):** Ứng dụng công nghệ WebSockets qua Laravel Reverb và Laravel Echo để xử lý luồng dữ liệu đồng bộ tức thời với độ trễ cực thấp.
3. **Toàn vẹn và linh hoạt về tính năng:** Đáp ứng đầy đủ các hình thức giao tiếp từ 1-1 đến hội thoại nhóm; hỗ trợ mạnh mẽ việc trao đổi đa phương tiện (tệp tin dung lượng lớn, ghi âm trực tiếp, hiển thị mã nguồn trực quan).
4. **Quản trị tập trung và bảo mật cao:** Cung cấp công cụ quản trị (Admin Dashboard) mạnh mẽ, tự động hóa các quy trình cấp phát tài khoản (thông qua luồng xử lý bất đồng bộ Laravel Queue/Job), phân quyền và kiểm soát truy cập chặt chẽ.

2.2 Yêu cầu chức năng

Dựa trên việc phân tích luồng nghiệp vụ và nhu cầu thực tế, các yêu cầu chức năng của hệ thống được phân rã và nhóm hóa theo từng đối tượng tác nhân (Actor) và phân hệ nghiệp vụ cụ thể như sau:

2.2.1 Yêu cầu chức năng đối với Người dùng thường (Regular User)

Đây là tác nhân chính trực tiếp sử dụng hệ thống để thực hiện các thao tác giao tiếp hàng ngày.

- **Phân hệ Xác thực & Quản lý tài khoản:**

Đăng nhập: Cho phép người dùng đăng nhập truy cập vào hệ thống.

Xác thực và Bảo mật: Hỗ trợ tính năng khôi phục mật khẩu qua email và xác minh địa chỉ email để đảm bảo tính chính danh.

Quản lý hồ sơ: Hỗ trợ cập nhật thông tin cá nhân cơ bản và đăng xuất khỏi thiết bị.

- **Phân hệ Nhắn tin trực tiếp (Direct Message - 1:1):**

Khởi tạo luồng giao tiếp: Cho phép tìm kiếm người dùng trong hệ thống và tự động khởi tạo kênh chat riêng tư ngay khi chọn.

- **Phân hệ Nhắn tin nhóm (Group Chat):**

Quản lý vòng đời nhóm: Hỗ trợ tạo nhóm chat mới, cho phép chọn nhiều thành viên tham gia từ danh bạ hệ thống.

Quản trị nội dung nhóm: Cung cấp quyền cập nhật thông tin nhóm (thay đổi tên nhóm, mô tả).

Quản trị nhân sự nhóm: Hỗ trợ thêm thành viên mới vào nhóm đang hoạt động hoặc mời/bớt thành viên hiện tại.

- **Phân hệ Tương tác và Định dạng tin nhắn:**

Tương tác hai chiều: Cho phép phản hồi (Reply) một tin nhắn cụ thể để duy trì mạch hội thoại.

Kiểm soát dữ liệu cá nhân: Cung cấp chức năng thu hồi/xóa tin nhắn đã gửi.

Định dạng văn bản: Hỗ trợ soạn thảo bằng văn bản thuần túy hoặc Markdown; đặc biệt có tính năng làm nổi bật các đoạn mã lập trình (Code Block).

- **Phân hệ Truyền tải Đa phương tiện (Multimedia):**

Cảm xúc và Đồ họa: Tích hợp bộ biểu tượng cảm xúc (Icon/Emoji).

Audio Recorder: Cho phép ghi âm giọng nói trực tiếp qua Micro trên trình duyệt và gửi đi dưới dạng tin nhắn thoại.

Quản lý tệp tin: Hỗ trợ tải lên tài liệu đính kèm. Đối với các tệp tin có dung lượng lớn, hệ thống phải áp dụng cơ chế cắt nhỏ (Chunk Upload) nhằm tối ưu băng thông và giảm thiểu lỗi mạng.

Media Preview: Cung cấp cửa sổ xem trước (Preview Modal) đối với các tệp hình ảnh và video trực tiếp trên giao diện chat mà không cần tải về thiết bị.

2.2.2 Yêu cầu chức năng đối với Quản trị viên (Admin)

Tác nhân quản trị có toàn quyền kiểm soát hệ thống, đóng vai trò giám sát, vận hành và quản lý nhân sự.

- **Phân hệ Quản trị Bảng điều khiển (Admin Dashboard):**

Giám sát danh sách người dùng: Cung cấp giao diện tổng quan để xem toàn bộ tài khoản trong hệ thống, tích hợp bộ lọc tìm kiếm nâng cao theo trạng thái (Active, Blocked, Admin, User).

Cấp phát tài khoản: Cho phép Admin tạo mới tài khoản thủ công từ trang quản trị.

Quản lý Phân quyền: Cung cấp công cụ thăng quyền một người dùng thông thường lên Quản trị viên (Promote) hoặc hạ quyền trở lại (Demote).

Kiểm soát truy cập: Cho phép khóa tài khoản (Block) để lập tức chặn đứng các truy cập trái phép hoặc mở khóa (Unblock) để khôi phục trạng thái hoạt động.

Loại bỏ dữ liệu: Hỗ trợ xóa vĩnh viễn dữ liệu tài khoản người dùng khỏi hệ thống.

2.3 Yêu cầu phi chức năng

1. Yêu cầu về hiệu năng (Performance)

- **Độ trễ tin nhắn:** Tin nhắn văn bản phải được chuyển đến người nhận với độ trễ dưới 1 giây trong điều kiện mạng ổn định.
- **Thời gian phản hồi giao diện:**
 - Thao tác xóa tin nhắn và cập nhật trên giao diện phải diễn ra tức thì
 - Thời gian phản hồi khi mở một cuộc hội thoại, tải danh sách thành viên hoặc tìm kiếm hội thoại phải thực hiện trong khoảng thời gian không quá 2 giây.
 - Kết quả tìm kiếm tin nhắn trong các cuộc hội thoại ngắn phải trả về dưới 1 giây.
 - Thao tác xóa toàn bộ cuộc hội thoại phải được thực hiện trong thời gian dưới 3 giây.
- **Tải dữ liệu:** Thời gian tải danh sách người dùng không được vượt quá 2 giây cho 1000 bản ghi.

2. Yêu cầu về bảo mật (Security)

- **Mã hóa dữ liệu:** Mật khẩu của người dùng bắt buộc phải được mã hóa một chiều (hashing) trước khi lưu trữ vào cơ sở dữ liệu. Toàn bộ lưu lượng dữ liệu truyền tải phải được mã hóa qua HTTPS và giao thức WebSocket phải được bảo mật bằng wss:// thông qua chứng chỉ SSL/TLS.
- **Chủ quyền dữ liệu (Data Sovereignty):** Hệ thống có khả năng tự lưu trữ (Self-hosted) ngay trên hạ tầng của doanh nghiệp để ngăn chặn rủi ro rò rỉ dữ liệu nội bộ.
- **Phân quyền truy cập:** Dữ liệu nhạy cảm phải được che mờ và chỉ những người có đúng quyền hạn (như Admin hoặc thành viên trong hội thoại) mới có thể xem tin nhắn, xóa thành viên hoặc xem danh sách người dùng.

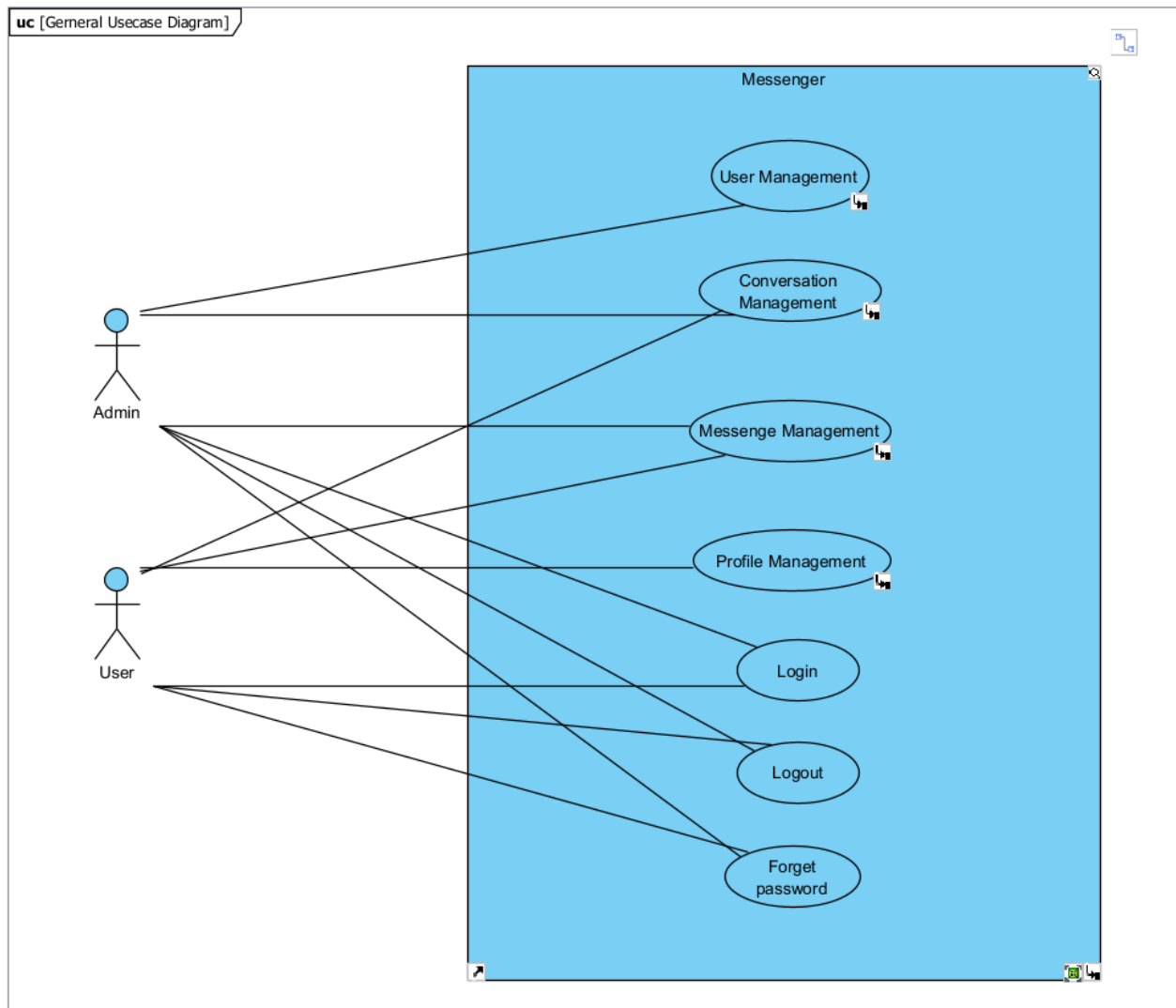
3. Yêu cầu về trải nghiệm người dùng (Usability/UX)

- **Hoạt động mượt mà:** Hệ thống được thiết kế theo kiến trúc Single Page Application (SPA). Chuyển hướng giữa các trang (như từ trang chủ sang trang Profile) sử dụng Persistent Layout của Inertia.js, đảm bảo giao diện không bị tải lại toàn trang (re-mount) gây gián đoạn.
- **Cập nhật tức thời:** Mọi thay đổi về thông tin (như chỉnh sửa hồ sơ cá nhân) phải được phản ánh ngay lập tức trên giao diện ngay sau khi nhấn lưu.

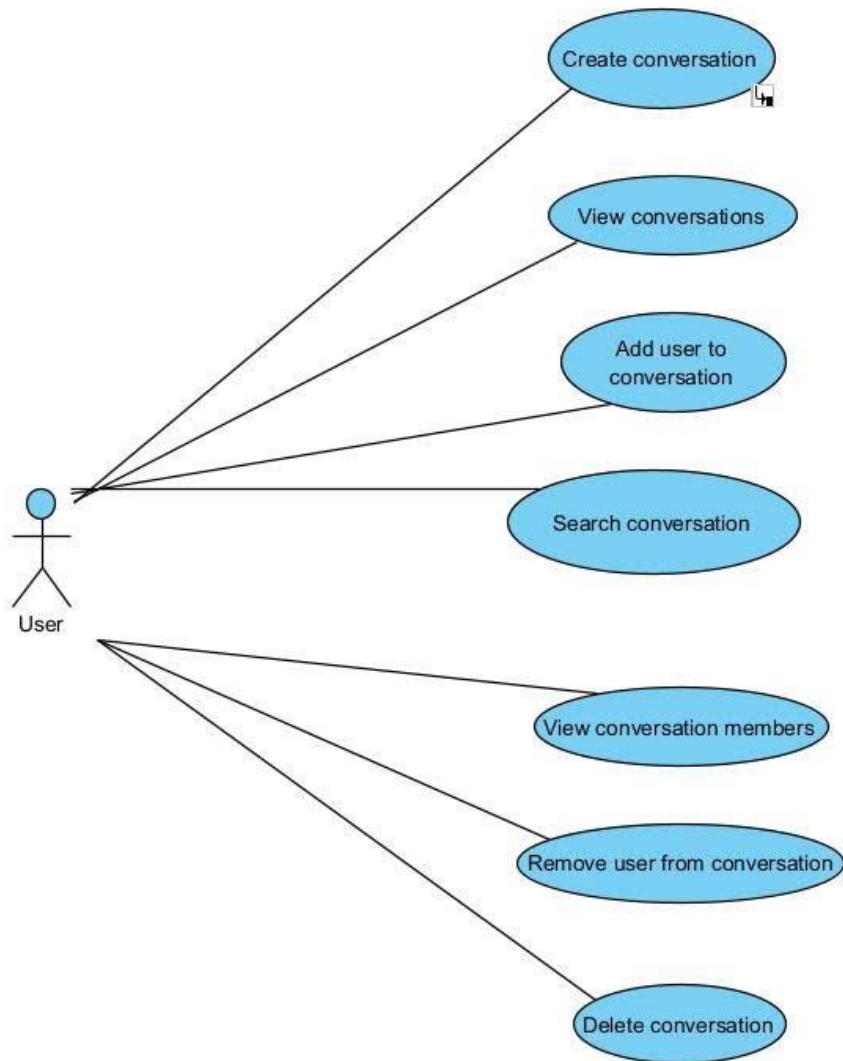
4. Yêu cầu về tính sẵn sàng và kiến trúc (Reliability & Architecture)

- **Xử lý thời gian thực:** Kết nối luồng dữ liệu song công (Duplex) liên tục qua WebSockets (Laravel Reverb) để đẩy (Push) dữ liệu tức thời từ máy chủ đến Client.
- **Xử lý tác vụ nền:** Các thao tác nặng hoặc mất thời gian (ví dụ: cấp phát tài khoản tự động, dọn dẹp thư mục rác khi xóa nhóm, gửi email) phải được xử lý bất đồng bộ thông qua hệ thống hàng đợi (Queue & Job) để không làm ảnh hưởng đến luồng hoạt động chính của hệ thống

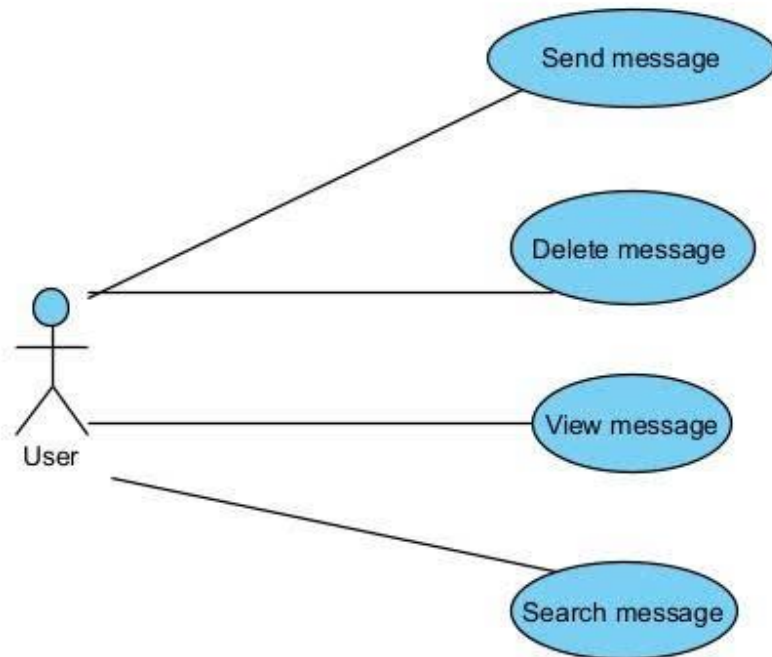
2.4 Biểu đồ use case tổng quát



Hình ảnh 1 Use Case Tổng quát

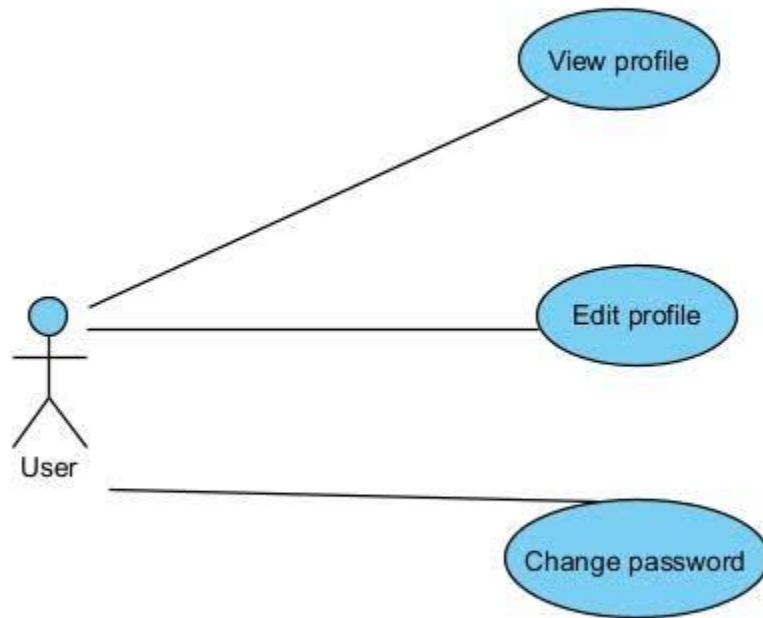


Hình ảnh 2 Use case Conversation Management

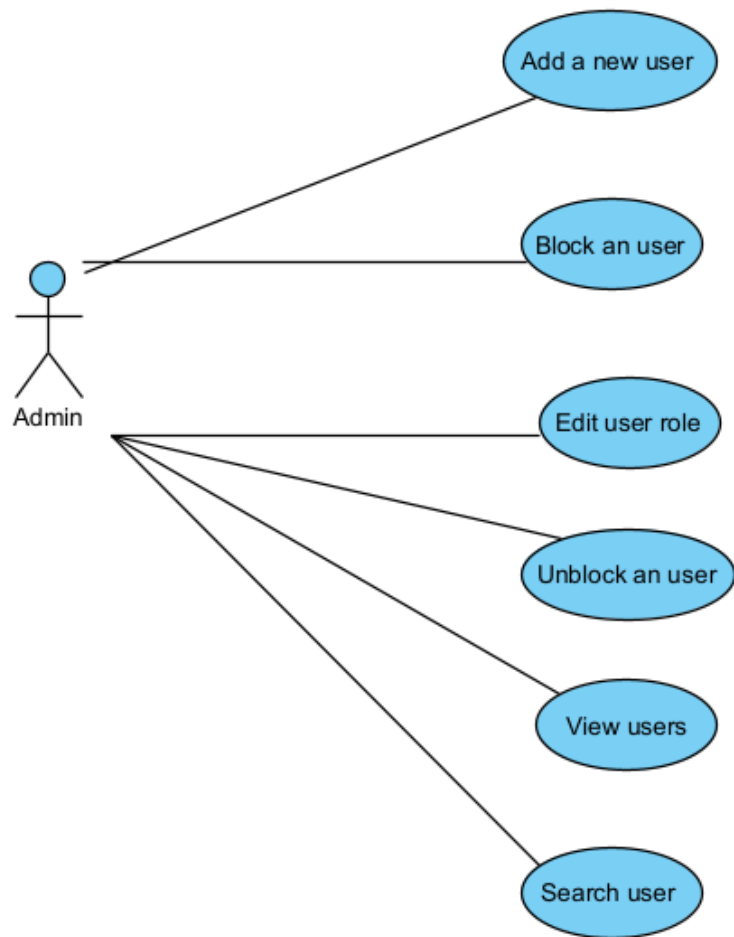


Hình ảnh 3 Use Case Message Management

uc [Profile Management Use Case Diagram]



Hình ảnh 4 Use Case Profile Management



Hình ảnh 5 Use Case User Management

2.5 Mô tả use case

Add a new user

Name	Add a new user	Code	UC01
Description	Admin thêm tài khoản mới vào hệ thống để cấp quyền truy cập cho nhân viên/người dùng khác.		
Actor	Admin	Trigger	Admin nhấn nút "Thêm mới" tại màn hình Quản lý người dùng.
Priority	Must have		
Pre_Condition	Admin đã đăng nhập vào hệ thống và có quyền quản trị.		
Post_Condition	Tài khoản mới được lưu vào cơ sở dữ liệu và có thể đăng nhập.		
Error situation	1. Mất kết nối Internet.		
System state in error situation	1. Hệ thống báo lỗi kết nối, không thể gửi yêu cầu xác thực.		
Activities Flow			
Main Flow			
Admin		System	
1. Admin mở giao diện "Thêm người dùng". 2. Nhập các thông tin: Họ tên, Email, Mật khẩu, Vai trò. 3. Nhấn nút "Lưu".		4. Kiểm tra tính hợp lệ và duy nhất của thông tin.	

		5. Lưu thông tin vào DB và thông báo "Thêm thành công".
Exception Flow	Thêm không thành công do thiếu thông tin	
Actor		System
		4a. Hệ thống kiểm tra thấy trường thông tin bị trống. 5a. Hệ thống hiển thị thông báo: "Please fill in all fields!"
Exception Flow	Thêm không thành công do thông tin sai	
Actor		System
		4b. Hệ thống kiểm tra thấy tài khoản hoặc mật khẩu không khớp. 5b. Hệ thống hiển thị thông báo: "Invalid login credentials!"
Business Rules	Email phải là email chưa được đăng ký	
Non_Functional Requirement	N/A	

Block a user

Name	Block a user	Code	UC02
Description	Admin khóa tài khoản của một người dùng để ngăn họ truy cập vào hệ thống.		
Actor	Admin	Trigger	Admin nhấn biểu tượng "Khóa" (Block) tại dòng tương ứng với người dùng.
Priority	Must have		
Pre_Condition	Admin đã đăng nhập vào hệ thống và có quyền quản trị.		
Post_Condition	Người dùng bị chặn tài khoản không thể nhắn tin		
Error situation	1. Mất kết nối Internet.		
System state in error situation	1. Hệ thống báo lỗi kết nối, không thể gửi yêu cầu xác thực.		
Activities Flow			
Main Flow			
Admin		System	
1. Chọn người dùng cần khóa. 2. Nhấn nút "Khóa". 3. Xác nhận "Đồng ý" tại thông báo xác nhận.		4. Cập nhật trạng thái người dùng trong DB. 5. Hiển thị thông báo "Đã khóa tài khoản thành công".	
Exception Flow	N/A		
Business Rules	N/A		
Non_Functional Requirement	N/A		

Edit role user

Name	Edit role user	Code	UC03
Description	Admin thay đổi vai trò hoặc quyền hạn của một người dùng hiện có.		
Actor	Admin	Trigger	Admin chọn "Chỉnh sửa" tại thông tin người dùng.
Priority	Must have		
Pre_Condition	Admin đã đăng nhập vào hệ thống và có quyền quản trị.		
Post_Condition	Người dùng được thay đổi quyền		
Error situation	1. Mất kết nối Internet.		
System state in error situation	1. Hệ thống báo lỗi kết nối, không thể gửi yêu cầu xác thực.		
Activities Flow			
Main Flow			
Admin		System	
1. Chọn người dùng cần sửa. 2. Thay đổi quyền người dùng 3. Nhấn "Cập nhật".		4. Ghi nhận vai trò mới vào hệ thống. 5. Thông báo "Cập nhật vai trò thành công".	
Exception Flow	N/A		
Business Rules	N/A		

Unblock User

Name	Unblock User	Code	UC04
Description	Admin mở khóa cho một tài khoản đang bị chặn để họ có thể tiếp tục sử dụng hệ thống.		
Actor	User, Admin	Trigger	Admin nhấn nút "Mở khóa" tại danh sách người dùng bị khóa.
Priority	Must have		
Pre_Condition	Admin đã đăng nhập vào hệ thống và có quyền quản trị.		
Post_Condition	Người dùng được mở khóa		
Error situation	1. Mất kết nối Internet.		
System state in error situation	1. Hệ thống báo lỗi kết nối, không thể gửi yêu cầu xác thực.		
Activities Flow			
Main Flow			
Admin		System	
1. Lọc danh sách người dùng bị khóa. 2. Nhấn nút "Mở khóa".		3. Thay đổi trạng thái tài khoản. 4. Thông báo "Mở khóa thành công".	
Exception Flow	N/A		
Business Rules	N/A		
Non_Functional Requirement	N/A		

View users

Name	View users	Code	UC05
Description	Quản trị viên xem danh sách người dùng trong hệ thống để quản lý.		
Actor	Admin	Trigger	Admin nhấn vào menu "Quản lý người dùng" từ giao diện hệ thống.
Priority	Must have		
Pre_Condition	Admin đã đăng nhập với quyền quản trị.		
Post_Condition	Hệ thống hiển thị danh sách tất cả người dùng.		
Error situation	1. Mất kết nối Internet.		
System state in error situation	1. Hệ thống báo lỗi kết nối, không thể tải dữ liệu.		
Activities Flow			
Main Flow	Xem danh sách thành công		
Admin		System	
1. Admin chọn menu Quản lý người dùng. 2. Hệ thống nhận yêu cầu.		3. Kiểm tra quyền truy cập. 4. Truy xuất danh sách người dùng từ DB. 5. Hiển thị danh sách lên màn hình.	
Exception Flow	Không có dữ liệu người dùng		
Actor		System	
		4a. Hệ thống kiểm tra thấy danh sách trống. 5a. Hiển thị thông báo: "No users found in the system!"	

Exception Flow	Lỗi truy xuất kết nối dữ liệu
Actor	System
	4b. Lỗi kết nối cơ sở dữ liệu. 5b. Hiển thị thông báo: "Database connection failed!"
Business Rules	1. Chỉ Admin mới có quyền xem danh sách này. Dữ liệu nhạy cảm phải được che mờ.
Non_Functional Requirement	Thời gian tải danh sách không quá 2 giây cho 1000 bản ghi.

Create conversation

Name	Create conversation	Code	UC06
Description	Người dùng tạo một cuộc hội thoại mới để bắt đầu nhắn tin.		
Actor	User	Trigger	Người dùng nhấn nút "Tạo hội thoại" từ màn hình chính.
Priority	Must have		
Pre_Condition	Người dùng đã đăng nhập vào hệ thống.		
Post_Condition	Hội thoại mới được tạo thành công.		
Error situation	1. Mất kết nối Internet		
System state in error situation	1. Hệ thống báo lỗi mạng, không thể khởi tạo.		

Activities Flow		
Main Flow	Tạo hội thoại thành công	
<i>User</i>	<i>System</i>	
1. Nhấn nút tạo mới. 2. Chọn người dùng muốn nhắn tin. 3. Nhấn "Bắt đầu".	4. Kiểm tra tính hợp lệ của người nhận. 5. Khởi tạo bản ghi hội thoại mới. 6. Điều hướng đến màn hình chat.	
Exception Flow	Chưa chọn người nhận	
<i>Actor</i>	<i>System</i>	
	4a. Hệ thống phát hiện chưa có người dùng nào được chọn. 5a. Hiển thị thông báo: "Please select at least one recipient!"	
Exception Flow	Người nhận không hợp lệ	
<i>Actor</i>	<i>System</i>	
	4b. Tài khoản người nhận đã bị khóa hoặc không tồn tại. 5b. Hiển thị thông báo: "Invalid recipient selected!"	
Business Rules	1. Hệ thống tự động kiểm tra xem đã có hội thoại 1-1 trước đó chưa để tránh tạo trùng.	
Non_Functional Requirement	N/A	

View conversations

Name	View conversations	Code	UC07
Description	Người dùng xem danh sách các cuộc hội thoại mà họ đang tham gia.		
Actor	User	Trigger	Người dùng truy cập vào mục "Tin nhắn" hoặc "Hội thoại".
Priority	Must have		
Pre_Condition	Người dùng đã đăng nhập vào hệ thống.		
Post_Condition	Hiển thị danh sách hội thoại sắp xếp theo tin nhắn mới nhất.		
Error situation	1. Mất kết nối Internet		
System state in error situation	1. Hệ thống báo lỗi kết nối máy chủ.		
Activities Flow			
Main Flow	Xem danh sách thành công		
User		System	
1. Mở giao diện hội thoại.		2. Xác định ID người dùng hiện tại. 3. Tải danh sách các hội thoại liên quan. 4. Hiển thị danh sách và nội dung tin nhắn cuối cùng.	
Exception Flow	Người dùng chưa có hội thoại nào		
Actor		System	

		3a. Hệ thống không tìm thấy lịch sử hội thoại. 4a. Hiện thị thông báo: "You have no conversations yet!"
Exception Flow	Lỗi đồng bộ tin nhắn	
Actor		System
		3b. Hệ thống không thể đồng bộ tin nhắn mới nhất. 4b. Hiện thị biểu tượng cảnh báo lỗi đồng bộ.
Business Rules	1. Tin nhắn mới nhất phải được in đậm nếu chưa đọc.	
Non_Functional Requirement	N/A	

Add user to conversation

Name	Add user to conversation	Code	UC08
Description	Thêm thành viên mới vào cuộc hội thoại nhóm hiện tại.		
Actor	User	Trigger	Người dùng nhấn nút "Thêm thành viên" trong cài đặt nhóm.
Priority	Should have		
Pre_Condition	Người dùng đang ở trong hội thoại nhóm và có quyền thêm người.		
Post_Condition	Thành viên mới được thêm vào và thấy được lịch sử nhóm (tùy cấu hình).		
Error situation	1. Mất kết nối Internet		

System state in error situation	1. Hệ thống báo lỗi, thành viên chưa được thêm.	
Activities Flow		
Main Flow	Thêm thành viên thành công	
User		System
1. Nhấn nút Thêm. 2. Chọn thành viên cần thêm. 3. Nhấn Xác nhận.		4. Kiểm tra xem người này đã có trong nhóm chưa. 5. Cập nhật danh sách thành viên. 6. Gửi thông báo hệ thống vào nhóm.
Exception Flow	Thành viên đã có sẵn trong nhóm	
Actor		System
		4a. Hệ thống phát hiện ID người dùng đã tồn tại trong nhóm. 5a. Hiển thị thông báo: "User is already a member of this group!"
Exception Flow	Không có quyền thêm thành viên	
Actor		System
		4b. Người dùng hiện tại không phải Admin nhóm. 5b. Hiển thị thông báo: "You don't have permission to add members!"
Business Rules	Số lượng thành viên tối đa trong một nhóm là 500 người.	

Non_Functional Requirement	N/A
-----------------------------------	-----

Search conversation

Name	Search Conversation	Code	UC09
Description	Là người dùng, tôi muốn tìm kiếm các cuộc hội thoại theo từ khóa để tra cứu lại thông tin.		
Actor	User	Trigger	Người dùng nhập từ khóa vào thanh tìm kiếm cuộc hội thoại
Priority	Must have		
Pre_Condition	Người dùng đã đăng nhập và đang ở màn hình danh sách cuộc hội thoại		
Post_Condition	Danh sách cuộc hội thoại phù hợp với từ khóa được hiển thị		
Error situation	1. Mất kết nối Internet		
System state in error situation	1. Hệ thống thông báo không thể thực hiện tìm kiếm lúc này		
Activities Flow			
Main Flow	Tìm kiếm thành công		
User		System	

1. Nhập từ khóa vào ô tìm kiếm 2. Nhấn Enter hoặc nút tìm kiếm		3. Nhận từ khóa và xử lý tìm kiếm 4. Trả về danh sách kết quả phù hợp
Exception Flow	Không nhập từ khóa	
Actor		System
		3a. Phát hiện từ khóa trống 4a. Hiển thị: "Please enter search keyword!"
Exception Flow	Không có kết quả	
Actor		System
		3b. Không tìm thấy dữ liệu phù hợp 4b. Hiển thị: "No conversations found!"
Business Rules	1. Tìm kiếm theo tên cuộc hội thoại hoặc tên thành viên 2. Không phân biệt chữ hoa/thường	
Non_Functional Requirement	Thời gian phản hồi ≤ 2 giây	

View conversation members

Name	View Conversation	Code	UC10
Description	Người dùng muốn xem danh sách thành viên trong cuộc hội thoại.		
Actor	User	Trigger	Người dùng nhấn vào danh sách thành viên trong cuộc hội thoại
Priority	Must have		
Pre_Condition	Người dùng đã đăng nhập và đang ở trong một cuộc hội thoại		
Post_Condition	Danh sách thành viên được hiển thị		
Error situation	1. Mất kết nối Internet		
System state in error situation	1. Hệ thống không thể tải danh sách thành viên		
Activities Flow			
Main Flow	Xem danh sách thành viên		
User		System	
1. Mở cuộc hội thoại		3. Lấy danh sách thành viên từ DB	
2. Nhấn vào danh sách thành viên		4. Hiển thị danh sách	
Exception Flow	Không tải được dữ liệu		
Actor		System	
		3a. Không lấy được dữ liệu	

		4a. Hiển thị: "Cannot load members list!"
Exception Flow	Không có kết quả	
Business Rules	1. Chỉ hiển thị thành viên thuộc cuộc hội thoại đó	
Non_Functional Requirement	Thực hiện trong ≤ 2 giây	

Remove user from conversation

Name	Remove User	Code	UC11
Description	Người dùng có quyền muốn xóa một thành viên khỏi cuộc hội thoại.		
Actor	User	Trigger	Người dùng chọn chức năng "Remove user"
Priority	Must have		
Pre_Condition	Người dùng là admin/owner của cuộc hội thoại		
Post_Condition	Thành viên bị xóa khỏi cuộc hội thoại		
Error situation	1. Mất kết nối Internet		
System state in error situation	1. Hệ thống thông báo không thể thực hiện tìm kiếm lúc này		
Activities Flow			
Main Flow	Xóa thành viên thành công		
<i>User</i>		<i>System</i>	

1. Mở danh sách thành viên 2. Chọn thành viên cần xóa 3. Nhấn "Remove"		4. Kiểm tra quyền của user 5. Xóa thành viên khỏi cuộc hội thoại 6. Hiển thị: "Remove successful"
Exception Flow	Không có quyền	
Actor		System
		4a. User không có quyền 5a. Hiển thị: "Permission denied!"
Exception Flow	Không có kết quả	
Actor		System
		3b. Không tìm thấy dữ liệu phù hợp 4b. Hiển thị: "No conversations found!"
Business Rules	1. Chỉ admin/owner mới được xóa thành viên	
Non_Functional Requirement	Thực hiện trong ≤ 2 giây	

Delete Conversation

Name	Delete Conversation	Code	UC12
Description	Người dùng muốn xóa toàn bộ cuộc hội thoại.		
Actor	User	Trigger	Người dùng chọn "Delete conversation"
Priority	Must have		
Pre_Condition	Người dùng là admin/owner của cuộc hội thoại		
Post_Condition	Cuộc hội thoại bị xóa khỏi hệ thống		
Error situation	1. Mất kết nối Internet		
System state in error situation	1. Hệ thống không thể xóa cuộc hội thoại		
Activities Flow			
Main Flow	Xóa cuộc hội thoại thành công		
User		System	
1. Mở cuộc hội thoại		4. Kiểm tra quyền user	
2. Chọn "Delete conversation"		5. Xóa dữ liệu cuộc hội thoại	
3. Xác nhận xóa		6. Điều hướng về danh sách cuộc hội thoại	
Exception Flow	Không có quyền		
Actor		System	
		4a. User không có quyền	
		5a. Hiển thị: "Permission denied!"	

Business Rules	1. Chỉ admin/owner mới được xóa
Non_Functional Requirement	Thực hiện trong ≤ 3 giây

Send message

Name	Send message	Code	UC13
Description	Người dùng soạn thảo và gửi tin nhắn văn bản hoặc tệp đính kèm cho người dùng khác trong hệ thống.		
Actor	User, Admin	Trigger	Người dùng nhấn nút "Gửi" hoặc phím "Enter" sau khi nhập nội dung vào ô soạn thảo.
Priority	Must have		
Pre_Condition	Người dùng đã đăng nhập và đang ở trong một cuộc hội thoại cụ thể.		
Post_Condition	Tin nhắn được gửi đi, hiển thị trong cửa sổ chat và lưu vào lịch sử hệ thống.		
Error situation	1. Mất kết nối Internet.		
System state in error situation	1. Hệ thống báo lỗi kết nối, tin nhắn hiển thị trạng thái "Gửi lỗi" (Failed to send).		
Activities Flow			
Main Flow	Gửi tin nhắn thành công		

<i>Actor</i>		<i>System</i>
1. Người dùng nhập nội dung vào ô soạn thảo tin nhắn. 2. Nhấn nút "Gửi" hoặc phím Enter.		3. Hệ thống kiểm tra tính hợp lệ của nội dung (không để trống). 4. Hệ thống lưu tin nhắn vào cơ sở dữ liệu và đánh dấu thời gian gửi. 5. Hệ thống hiển thị tin nhắn mới lên màn hình hội thoại của cả người gửi và người nhận.
Exception Flow		Gửi tin nhắn trống
<i>Actor</i>		<i>System</i>
		3a. Hệ thống kiểm tra thấy ô nhập liệu bị bỏ trống. 4a. Hệ thống không thực hiện gửi và hiển thị thông báo: "Cannot send an empty message!"
Exception Flow		Tệp đính kèm vượt quá dung lượng
<i>Actor</i>		<i>System</i>
1b. Người dùng đính kèm tệp có dung lượng quá lớn.		2b. Hệ thống kiểm tra thấy vượt quá giới hạn cho phép. 3b. Hệ thống hiển thị thông báo lỗi: "File size exceeds the limit!"
Business Rules		1. Tin nhắn văn bản không được vượt quá 2000 ký tự. 2. Cho phép gửi kèm các định dạng tệp phổ biến (JPG, PNG, PDF, DOCX,...).

	3. Người dùng có thể thu hồi tin nhắn trong vòng 5 phút sau khi gửi.
Non_Functional Requirement	Tin nhắn phải được chuyển đến người nhận với độ trễ dưới 1 giây trong điều kiện mạng ổn định.

Delete message

Name	Delete message	Code	UC14
Description	Người dùng xóa một hoặc nhiều tin nhắn trong cuộc hội thoại để dọn dẹp nội dung.		
Actor	User, Admin	Trigger	Người dùng nhấn giữ vào tin nhắn hoặc nhấn vào biểu tượng thùng rác/tùy chọn "Delete".
Priority	High		
Pre_Condition	Người dùng đã đăng nhập và đang ở trong giao diện chi tiết cuộc hội thoại.		
Post_Condition	Tin nhắn được loại bỏ khỏi giao diện người dùng và cập nhật trong cơ sở dữ liệu.		
Error situation	1. Mất kết nối Internet.		
System state in error situation	1. Hệ thống báo lỗi kết nối, tin nhắn vẫn tồn tại sau khi tải lại trang.		
Activities Flow			
Main Flow	Xóa tin nhắn thành công		
Actor		System	

1. Người dùng chọn tin nhắn cần xóa. 2. Nhấn nút "Delete". 4. Người dùng nhấn "Confirm/Yes".		3. Hệ thống hiển thị hộp thoại xác nhận: "Are you sure you want to delete this message?". 5. Hệ thống thực hiện xóa tin nhắn trong cơ sở dữ liệu. 6. Hệ thống cập nhật lại giao diện và thông báo: "Message deleted successfully!".
Exception Flow	Người dùng hủy lệnh xóa	
Actor		System
4a. Người dùng nhấn "Cancel" tại hộp thoại xác nhận.		5a. Hệ thống đóng hộp thoại và giữ nguyên tin nhắn.
Exception Flow	Lỗi quyền truy cập (Dành cho Admin)	
Actor		System
		3b. Hệ thống kiểm tra quyền hạn của người dùng đối với tin nhắn này. 4b. Nếu không có quyền xóa (ví dụ tin nhắn hệ thống), hiển thị thông báo: "You do not have permission to delete this message!".
Business Rules	1. Hệ thống phải hỏi xác nhận trước khi thực hiện xóa vĩnh viễn. 2. Cho phép tùy chọn: "Delete for me" (Xóa phía tôi) hoặc "Delete for everyone" (Xóa đối với mọi người) nếu là người gửi.	

	3. Sau khi xóa, nội dung tin nhắn có thể được thay thế bằng dòng chữ "This message was deleted".
Non_Functional Requirement	Thao tác xóa và cập nhật giao diện phải diễn ra tức thì (dưới 1 giây).

View messages

Name	View messages	Code	UC15
Description	Người dùng hoặc Quản trị viên xem danh sách các cuộc hội thoại và nội dung chi tiết của tin nhắn.		
Actor	User, Admin	Trigger	Người dùng nhấn vào biểu tượng tin nhắn hoặc mục "Messages" từ giao diện chính.
Priority	Must have		
Pre_Condition	Người dùng đã đăng nhập thành công vào hệ thống.		
Post_Condition	Hệ thống hiển thị danh sách tin nhắn hoặc nội dung chi tiết của hội thoại đã chọn.		
Error situation	1. Mất kết nối Internet.		
System state in error situation	1. Hệ thống báo lỗi kết nối, không thể tải dữ liệu tin nhắn mới		
Activities Flow			
Main Flow	Xem tin nhắn thành công		
Actor		System	

1. Người dùng chọn mục "Messages" từ menu điều hướng. 3. Người dùng nhấn chọn một cuộc hội thoại cụ thể.		2. Hệ thống truy xuất và hiển thị danh sách các cuộc hội thoại gần đây. 4. Hệ thống tải và hiển thị nội dung chi tiết của các tin nhắn trong hội thoại đó. 5. Hệ thống đánh dấu các tin nhắn mới là "Đã xem" (Read).
Exception Flow	Không có tin nhắn	
Actor		System
3b. Người dùng chọn một hội thoại nhưng dữ liệu không thể tải xuống.		2a. Hệ thống kiểm tra và thấy người dùng chưa có cuộc hội thoại nào. 3a. Hệ thống hiển thị thông báo: "You have no messages yet!" 4b. Hệ thống hiển thị thông báo lỗi: "Unable to load messages. Please try again later!"
Business Rules	1. Tin nhắn mới nhất phải được hiển thị lên đầu danh sách hội thoại. 2. Chỉ người tham gia cuộc hội thoại mới có quyền xem nội dung tin nhắn đó. 3. Hiển thị thời gian gửi tin nhắn theo định dạng thời gian thực (ví dụ: 5 phút trước, Hôm qua).	
Non_Functional Requirement	Thời gian phản hồi khi mở một cuộc hội thoại không quá 2 giây.	

Search message

Name	Search message	Code	UC16
Description	Là người dùng, tôi muốn tìm kiếm các tin nhắn cũ theo từ khóa để tra cứu lại thông tin.		
Actor	User, Admin	Trigger	Người dùng nhập từ khóa vào thanh tìm kiếm trong cuộc hội thoại.
Priority	Must have		
Pre_Condition	Người dùng đã đăng nhập và đang ở trong một cuộc hội thoại cụ thể.		
Post_Condition	Danh sách các tin nhắn chứa từ khóa được hiển thị.		
Error situation	1. Mất kết nối Internet.		
System state in error situation	1. Hệ thống thông báo không thể thực hiện tìm kiếm lúc này.		
Activities Flow			
Main Flow	Tìm kiếm tin nhắn thành công		
Admin		System	
1. Người dùng chọn biểu tượng "Tìm kiếm" trong cuộc hội thoại. 2. Người dùng nhập từ khóa cần tìm. 3. Người dùng nhấn Enter hoặc nút Tìm kiếm.		4. Hệ thống quét dữ liệu tin nhắn trong cuộc hội thoại. 5. Hệ thống hiển thị danh sách các tin nhắn có chứa từ khóa .	
Exception Flow	Không tìm thấy kết quả		
Actor		System	

		4a. Hệ thống không tìm thấy tin nhắn nào khớp với từ khóa. 5a. Hệ thống hiển thị thông báo: "No messages found" hoặc "Không tìm thấy tin nhắn nào".
Exception Flow	Từ khóa quá ngắn	
Actor		System
		4b. Người dùng nhập ít hơn số ký tự quy định (VD: 1 ký tự). 5b. Hệ thống yêu cầu: "Vui lòng nhập tối thiểu 2 ký tự".
Business Rules	1. Chỉ tìm kiếm được các tin nhắn trong cuộc hội thoại hiện tại. 2. Từ khóa tìm kiếm không phân biệt chữ hoa, chữ thường.	
Non_Functional Requirement	Kết quả tìm kiếm phải trả về nhanh chóng (< 1 giây) đối với các cuộc hội thoại ngắn.	

View profile

Name	View Profile	Code	UC17
Description	Người dùng hoặc Quản trị viên truy cập vào trang cá nhân để xem các thông tin hồ sơ của mình (Tên, Email, Ảnh đại diện)		

Actor	User, Admin	Trigger	Người dùng nhấn vào Menu Dropdown (tên/avatar của mình) trên thanh Header và chọn mục "Profile"
Priority	High		
Pre_Condition	Người dùng đã đăng nhập thành công vào hệ thống.		
Post_Condition	Hệ thống hiển thị trang Profile chứa các thông tin cá nhân hiện tại của người dùng.		
Error situation	1. Mất kết nối Internet khi đang chuyển trang. 2. Phiên đăng nhập (Session) đã hết hạn.		
System state in error situation	1. Trình duyệt báo lỗi mạng. 2. Hệ thống tự động đẩy người dùng về trang Đăng nhập (Login).		
Activities Flow			
Main Flow	Xem thông tin hồ sơ thành công		
Actor		System	
1. Tại bất kỳ trang nào (có AuthenticatedLayout), người dùng click vào Dropdown menu ở góc trên cùng bên phải. 2. Chọn mục "Profile".		3. Hệ thống tiếp nhận yêu cầu, kiểm tra xác thực thông qua Middleware. 4. Lấy dữ liệu của người dùng hiện tại (Current User) chia sẻ qua Inertia props (page.props.auth.user) 5. Điều hướng và render component profile/edit.jsx, hiển thị thông tin gồm: Ảnh đại diện (Avatar), Tên (Name), và Email	

Exception Flow	Xem hồ sơ thất bại do phiên đăng nhập hết hạn		
Actor	System		
	3a. Hệ thống kiểm tra thấy phiên đăng nhập đã hết hạn (Middleware từ chối yêu cầu). 4a. Hệ thống chuyển hướng (redirect) người dùng về trang Đăng nhập.		
Business Rules	1. Chỉ được phép xem thông tin hồ sơ của chính tài khoản đang đăng nhập. 2. Dữ liệu hiển thị mặc định không bao gồm Mật khẩu (Password).		
Non_Functional Requirement	Giao diện trang Profile sử dụng tính năng Persistent Layout của Inertia.js để khi chuyển từ trang chủ (Home) sang trang Profile, giao diện layout bọc ngoài không bị tải lại (re-mount), mang lại trải nghiệm mượt mà (SPA)		

Edit profile

Name	Edit profile	Code	UC18
Description	Là người dùng, tôi muốn chỉnh sửa thông tin cá nhân (Họ tên, email, ảnh đại diện,...) để cập nhật hồ sơ của mình.		
Actor	User, Admin	Trigger	Người dùng nhấn nút "Chỉnh sửa" hoặc "Lưu" trong giao diện Hồ sơ cá nhân.

Priority	Must have
Pre_Condition	Người dùng đã đăng nhập thành công vào hệ thống.
Post_Condition	Thông tin cá nhân mới được cập nhật và hiển thị trên hệ thống.
Error situation	1. Mất kết nối Internet. 2. Dung lượng file ảnh đại diện quá lớn.
System state in error situation	1. Hệ thống báo lỗi kết nối, dữ liệu không được lưu. 2. Hệ thống báo lỗi file không hợp lệ.
Activities Flow	
Main Flow	Cập nhật thông tin thành công
<i>Admin</i>	<i>System</i>
1. Người dùng truy cập vào mục "Hồ sơ cá nhân". 2. Chọn chức năng "Chỉnh sửa". 3. Thay đổi các thông tin cần thiết (Tên, giới tính, tiểu sử, ảnh...). 4. Nhấn nút "Lưu thay đổi".	5. Hệ thống kiểm tra tính hợp lệ của dữ liệu. 6. Hệ thống cập nhật cơ sở dữ liệu và hiển thị thông báo "Cập nhật thành công".
Exception Flow	Bỏ trống thông tin bắt buộc
<i>Actor</i>	<i>System</i>
	5a. Hệ thống phát hiện các trường bắt buộc (như Tên) bị để trống.

		6a. Hệ thống hiển thị thông báo: "Vui lòng không để trống thông tin bắt buộc!"
Exception Flow	Định dạng dữ liệu không hợp lệ	
Actor		System
		5b. Người dùng nhập sai định dạng (VD: Sai định dạng email). 6b. Hệ thống hiển thị thông báo lỗi tại trường tương ứng.
Business Rules	1. Một số trường như "Tên đăng nhập" hoặc "Email" có thể bị khóa không cho chỉnh sửa . 2. Ảnh đại diện phải thuộc định dạng: .jpg, .png.	
Non_Functional Requirement	Thay đổi phải được phản ánh ngay lập tức trên giao diện sau khi nhấn Lưu.	

Change password

Name	Change password	Code	UC19
Description	Là người dùng, tôi muốn thay đổi mật khẩu hiện tại để tăng tính bảo mật cho tài khoản.		
Actor	User, Admin	Trigger	Người dùng chọn chức năng "Đổi mật khẩu" trong phần Quản lý hồ sơ cá nhân.
Priority	Must have		
Pre_Condition	Đã đăng nhập vào trong hệ thống.		

Post_Condition	Mật khẩu mới được cập nhật vào hệ thống; mật khẩu cũ không còn hiệu lực.
Error situation	1. Mất kết nối Internet.
System state in error situation	1. Hệ thống không gửi được yêu cầu thay đổi và báo lỗi kết nối.
Activities Flow	
Main Flow	Thay đổi mật khẩu thành công
<i>Admin</i>	<i>System</i>
1. Người dùng truy cập vào mục "Thay đổi mật khẩu". 2. Nhập mật khẩu hiện tại. 3. Nhập mật khẩu mới và Nhập lại mật khẩu mới. 4. Nhấn nút "Cập nhật".	5. Hệ thống kiểm tra mật khẩu cũ, độ khớp của mật khẩu mới và lưu vào cơ sở dữ liệu. 6. Hệ thống thông báo thay đổi thành công và (tùy chọn) yêu cầu đăng nhập lại.
Exception Flow	Sai mật khẩu cũ
<i>Actor</i>	<i>System</i>
	5a. Hệ thống kiểm tra mật khẩu hiện tại không chính xác. 6a. Thông báo: "Mật khẩu hiện tại không đúng!"
Exception Flow	Mật khẩu mới không khớp
<i>Actor</i>	<i>System</i>

	5b. Hệ thống kiểm tra mật khẩu mới và phần nhập lại không trùng nhau. 6b. Thông báo: "Mật khẩu nhập lại không khớp!"
Business Rules	1. Mật khẩu mới không được trùng với mật khẩu hiện tại. 2. Tất cả các trường thông tin không được để trống.
Non_Functional Requirement	Mật khẩu phải được mã hóa một chiều (hashing) trước khi lưu vào DB.

Use case Login

Name	Login	Code	UC20
Description	Người dùng/Quản trị viên đăng nhập vào hệ thống để sử dụng các tính năng.		
Actor	User, Admin	Trigger	Người dùng nhấn nút đăng nhập từ màn hình chào hoặc màn hình đăng nhập.
Priority	Must have		
Pre_Condition	Người dùng đã có tài khoản được đăng ký/cấp quyền trong hệ thống.		
Post_Condition	Hệ thống chuyển về giao diện trang chủ.		
Error situation	1. Mất kết nối Internet.		

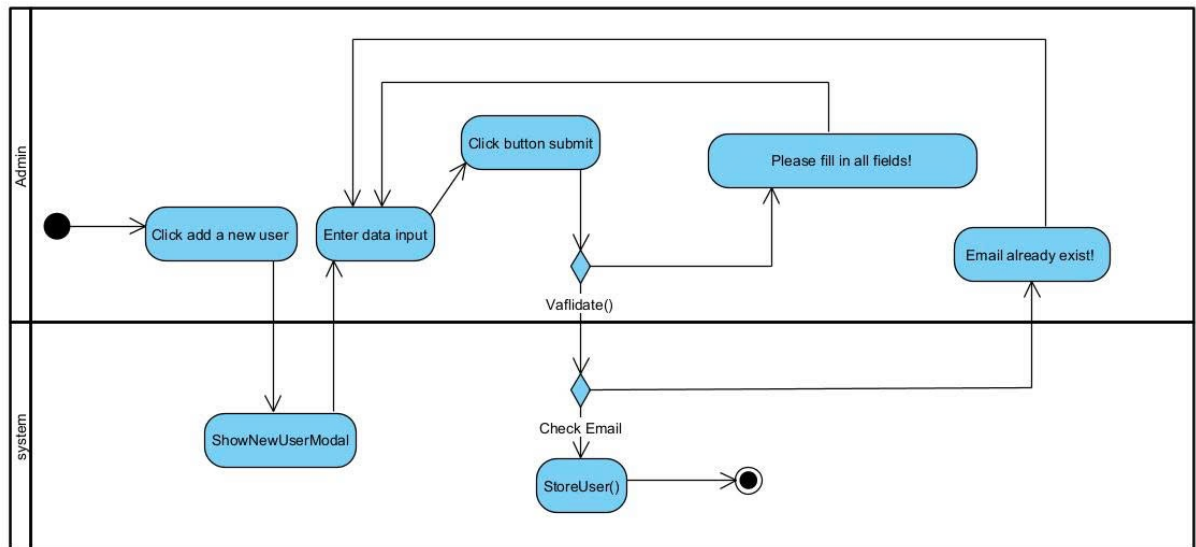
System state in error situation	1. Hệ thống báo lỗi kết nối, không thể gửi yêu cầu xác thực.
Activities Flow	
Main Flow	Đăng nhập thành công
<i>Admin</i>	<i>System</i>
1. Người dùng mở giao diện đăng nhập. 2. Nhập thông tin Tài khoản và Mật khẩu. 3. Nhấn nút "Đăng nhập".	4. Kiểm tra tính hợp lệ của thông tin. 5. Xác thực thành công và điều hướng về trang chủ.
Exception Flow	Đăng nhập không thành công do thiếu thông tin
<i>Actor</i>	<i>System</i>
	4a. Hệ thống kiểm tra thấy trường thông tin bị trống. 5a. Hệ thống hiển thị thông báo: <i>"Please fill in all fields!"</i>
Exception Flow	Đăng nhập không thành công do sai thông tin
<i>Actor</i>	<i>System</i>
	4b. Hệ thống kiểm tra thấy tài khoản hoặc mật khẩu không khớp. 5b. Hệ thống hiển thị thông báo: <i>"Invalid login credentials!"</i>
Business Rules	Mật khẩu phải được ẩn khi nhập. Cho phép đăng nhập bằng Email hoặc Username.
Non_Functional Requirement	N/A

Use case Logout

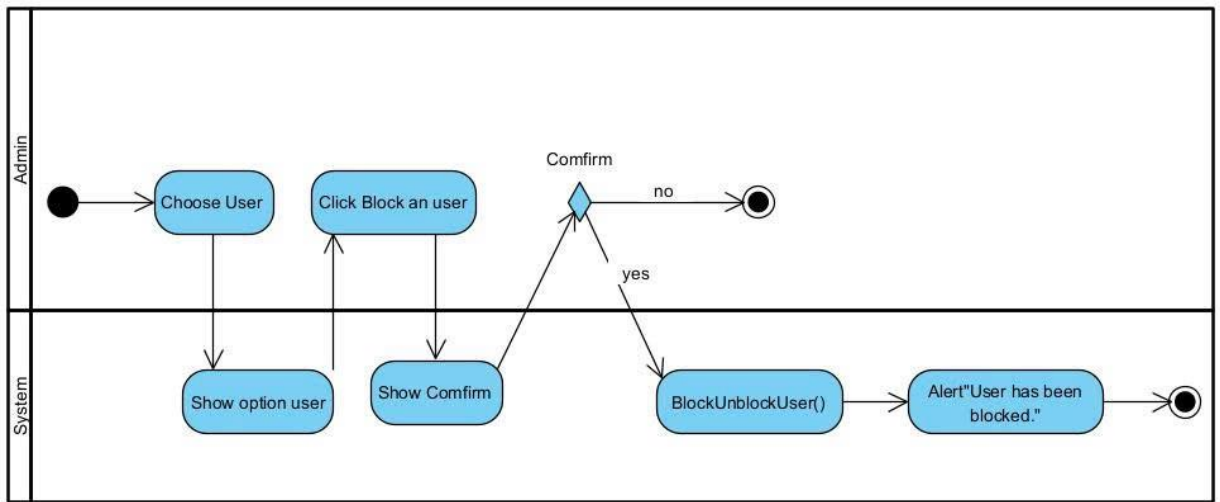
Name	Logout	Code	UC21
Description	Người dùng hoặc Quản trị viên muốn thoát khỏi phiên làm việc hiện tại.		
Actor	User, Admin	Trigger	Người dùng nhấn vào nút "Đăng xuất" (Logout).
Priority	Must have		
Pre_Condition	Người dùng đang ở trạng thái đã đăng nhập vào hệ thống.		
Post_Condition	Hệ thống hủy phiên đăng nhập (Session/Token) và quay về màn hình đăng nhập.		
Error situation	1. Mất kết nối Internet khi đang gửi yêu cầu hủy session tới server.		
System state in error situation	1. Hệ thống báo lỗi không thể đăng xuất hoặc tự động xóa session cục bộ và yêu cầu đăng nhập lại sau.		
Activities Flow			
Main Flow	Đăng xuất thành công		
Actor		System	
1. Người dùng truy cập vào trang cá nhân hoặc menu cài đặt. 2. Người dùng nhấn nút "Đăng xuất". 4. Người dùng nhấn "Xác nhận" trên pop-up.		3. Hệ thống hiển thị thông báo/pop-up xác nhận đăng xuất. 5. Hệ thống xóa phiên làm việc và điều hướng về giao diện Đăng nhập.	
Exception Flow	Người dùng hủy lệnh đăng xuất		
Actor	System		

4a. Tại bước 3, người dùng nhấn "Hủy" (Cancel).	5a. Hệ thống đóng pop-up và giữ nguyên trạng thái tại màn hình hiện tại.
Business Rules	Sau khi đăng xuất, các nút "Back" trên trình duyệt/thiết bị không được phép quay lại trang trước đó nếu không đăng nhập lại.
Non_Functional Requirement	N/A

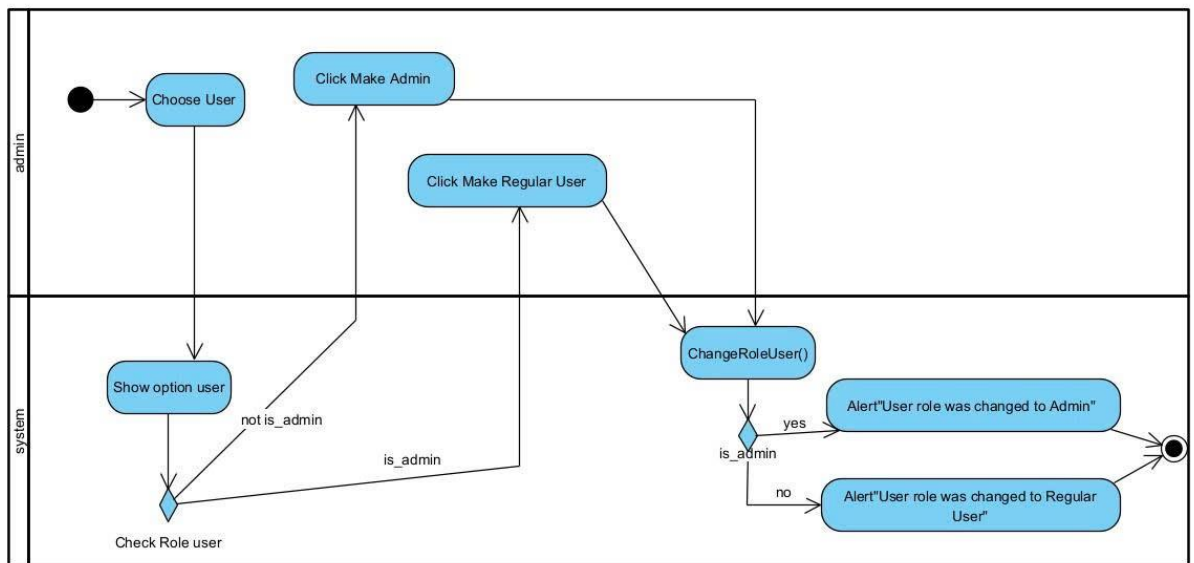
2.7 Biểu đồ activity



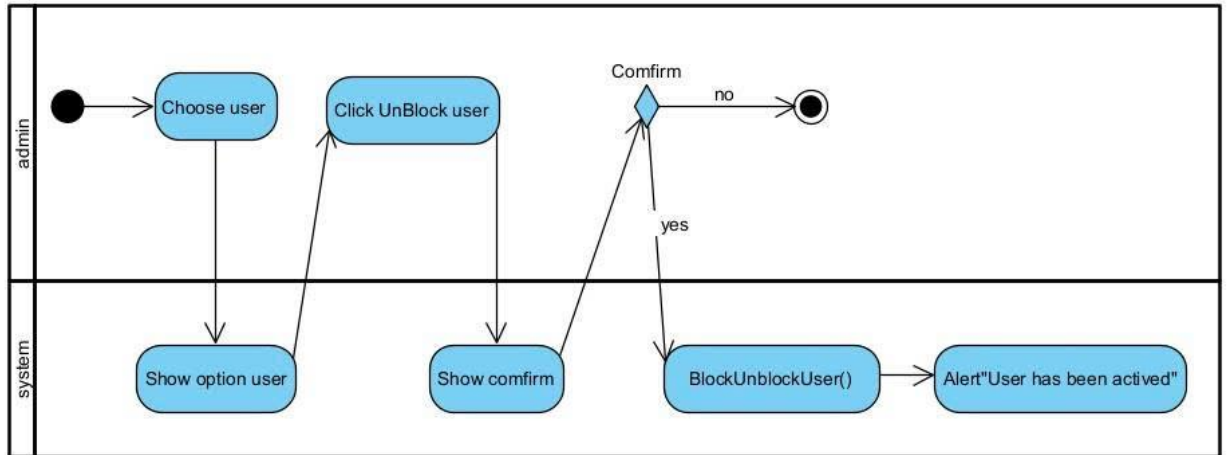
Hình ảnh 6 Add a new user activity



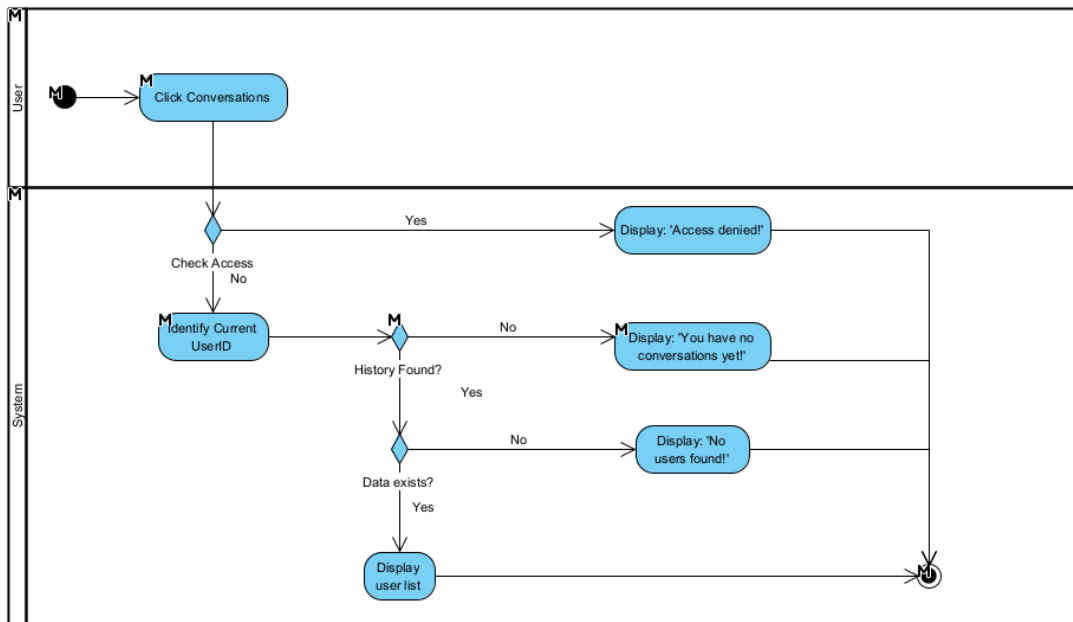
Hình ảnh 7 Block a user activity



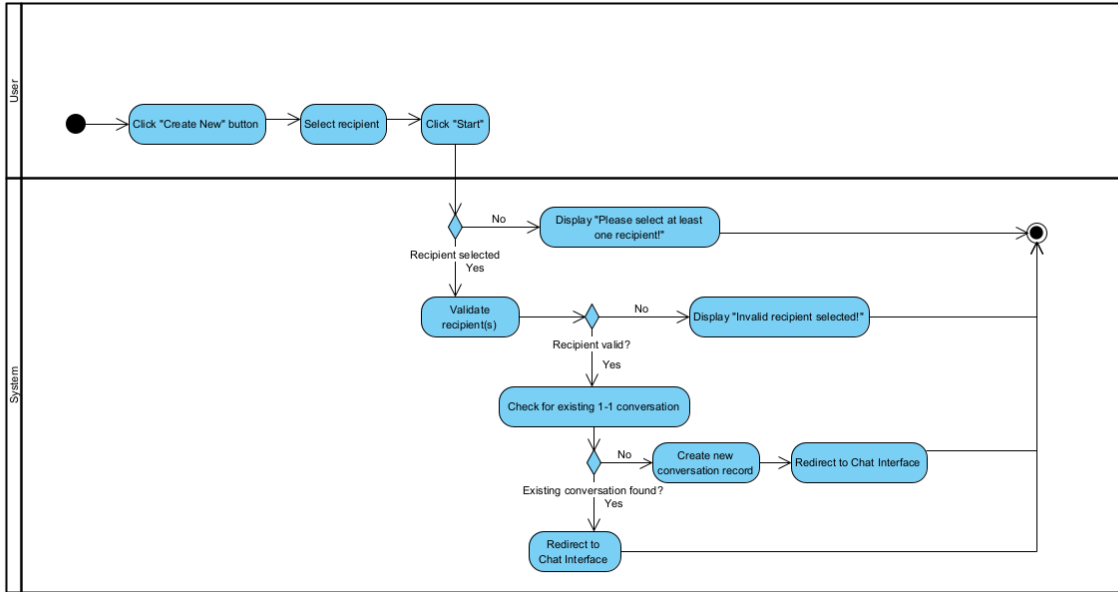
Hình ảnh 8 Change role user activity



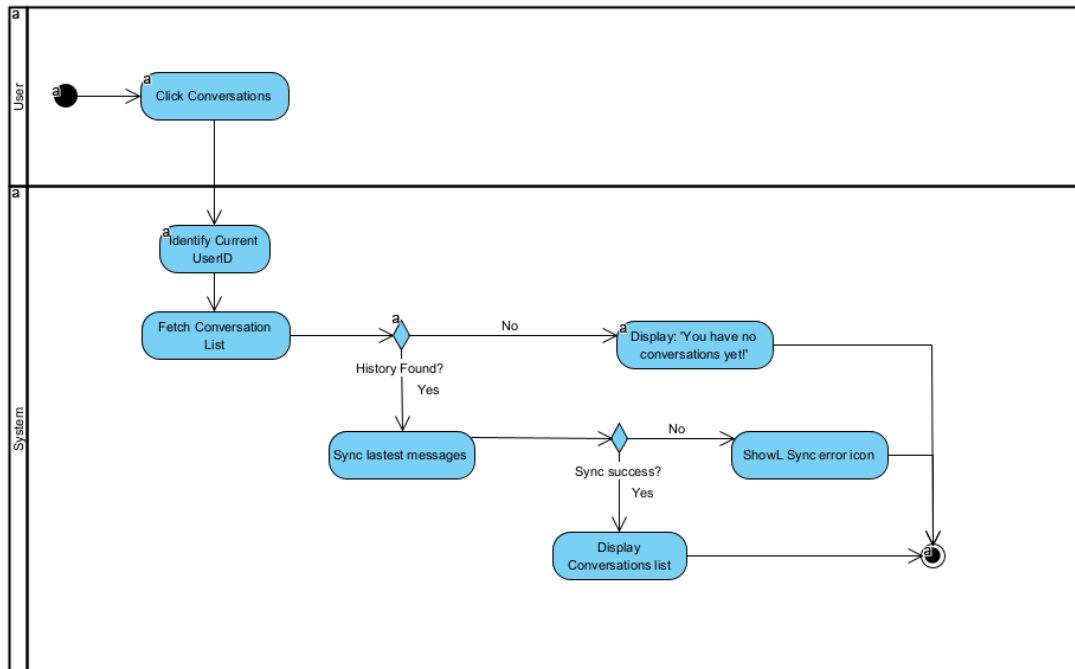
Hình ảnh 9 Unblock user activity



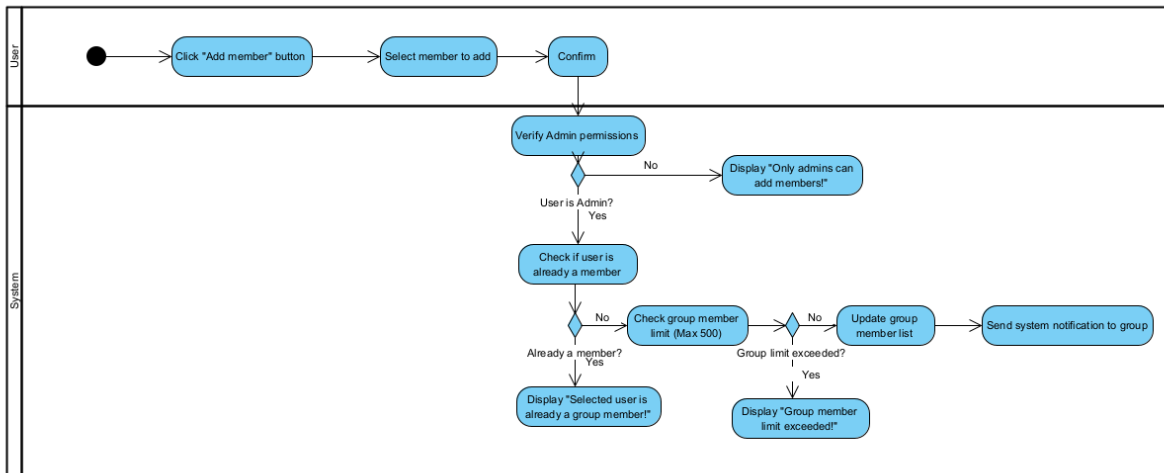
Hình ảnh 10 View users activity



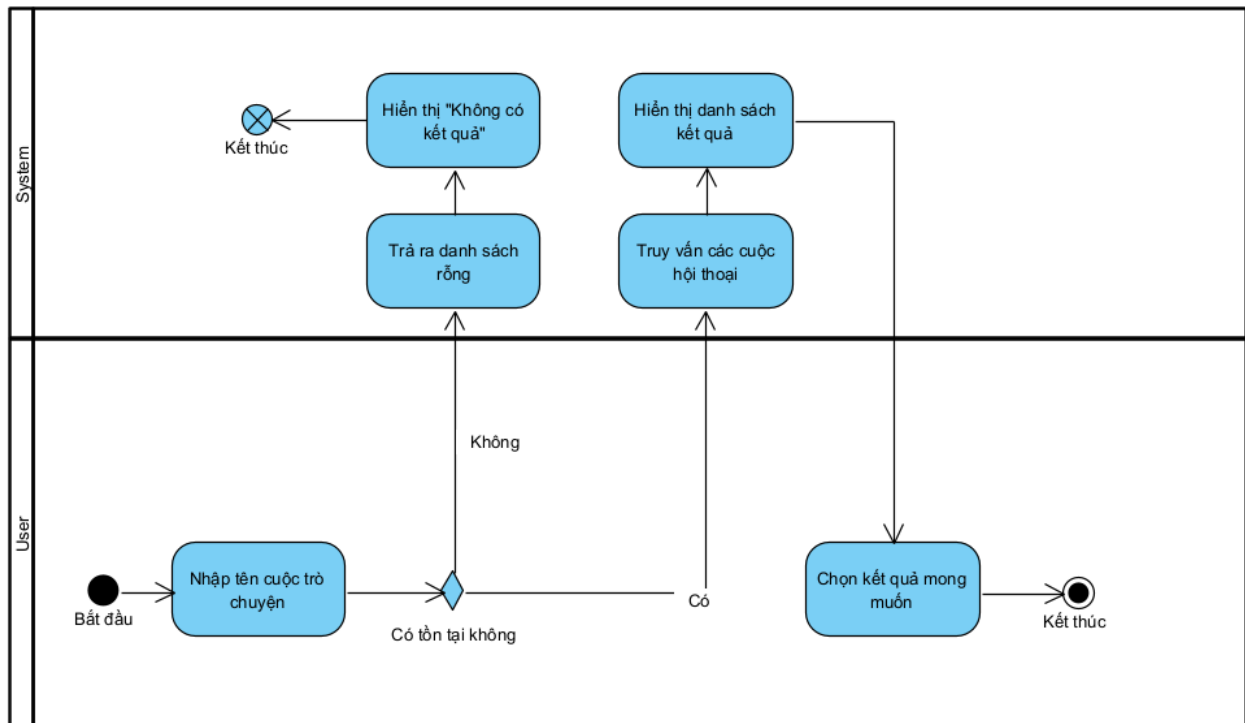
Hình ảnh 11 Create conversation activity



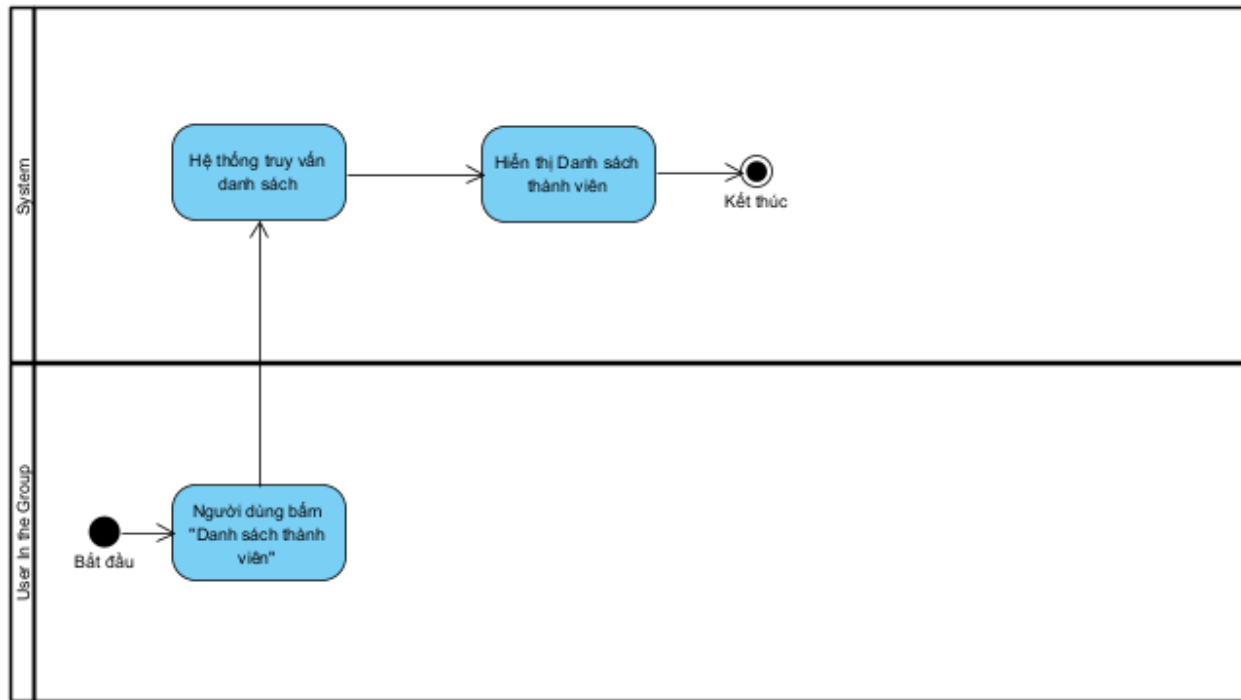
Hình ảnh 12 View conversation activity



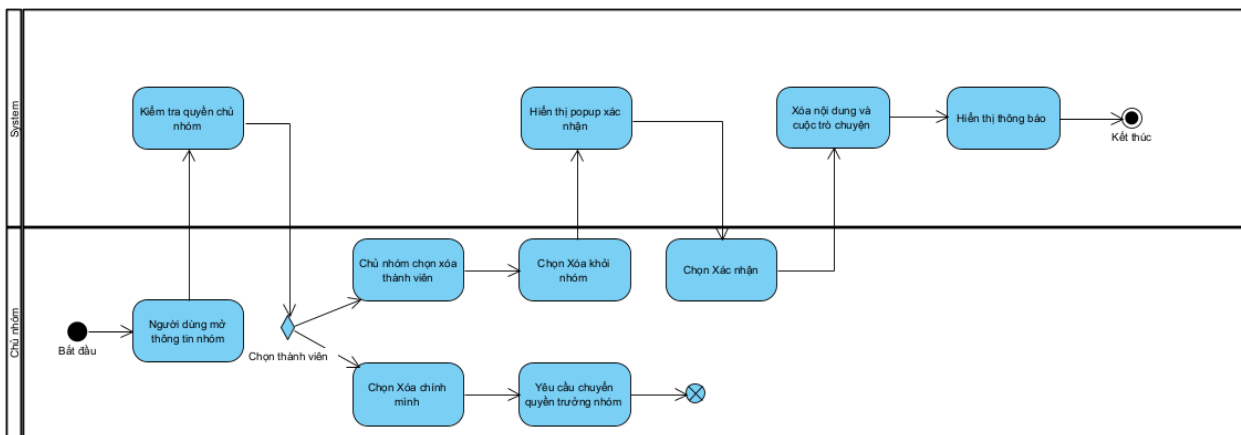
Hình ảnh 13 Add user to conversation activity



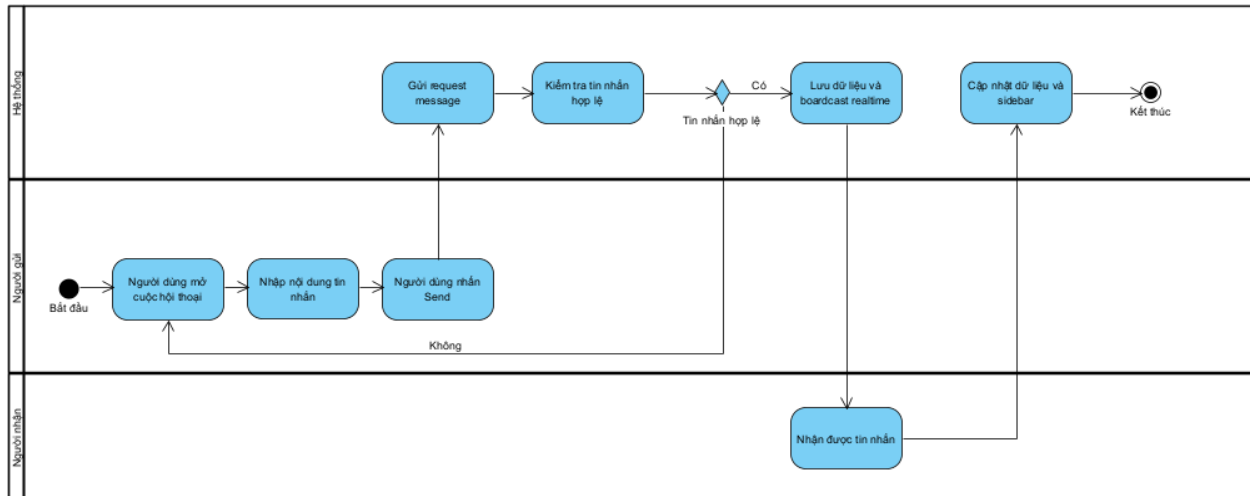
Hình ảnh 14 Search conversation activity



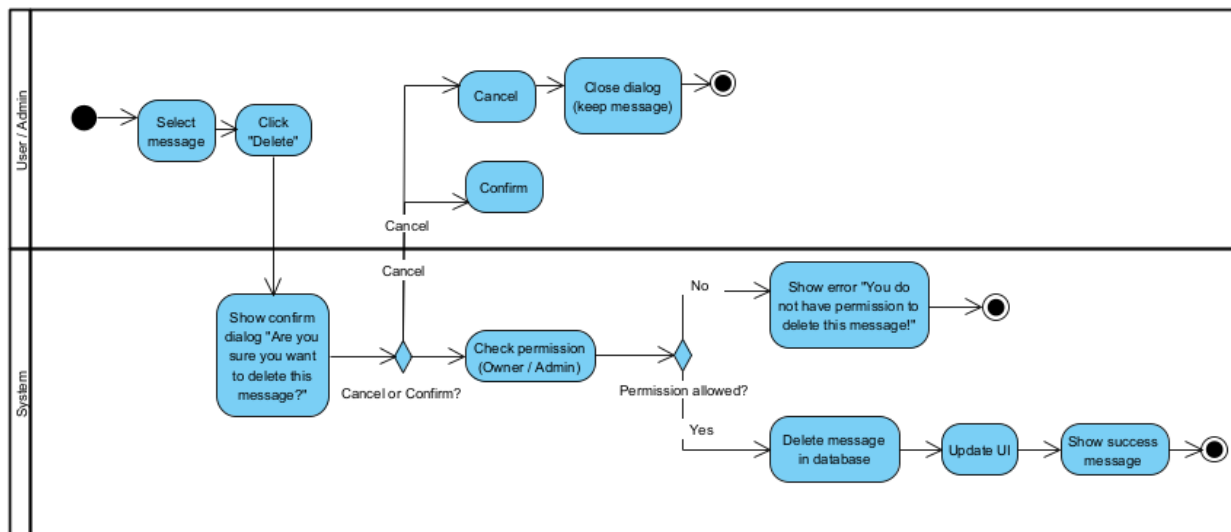
Hình ảnh 15 View conversation members



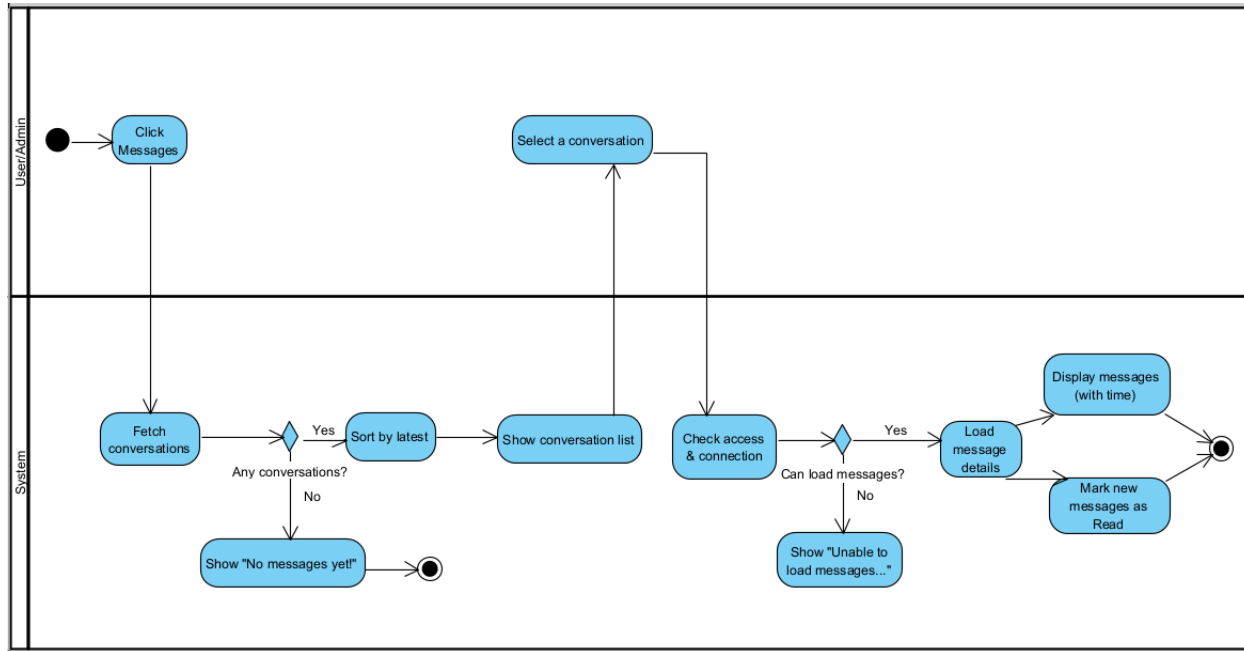
Hình ảnh 16 Remove User from conversation activity



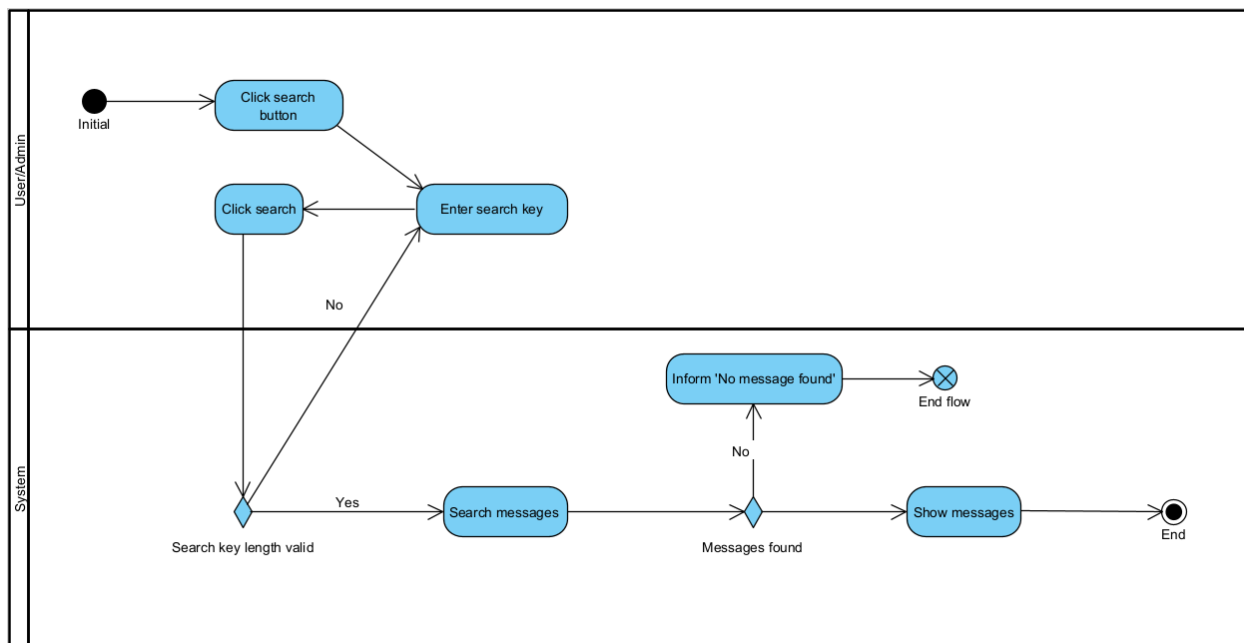
Hình ảnh 17 Send message activity



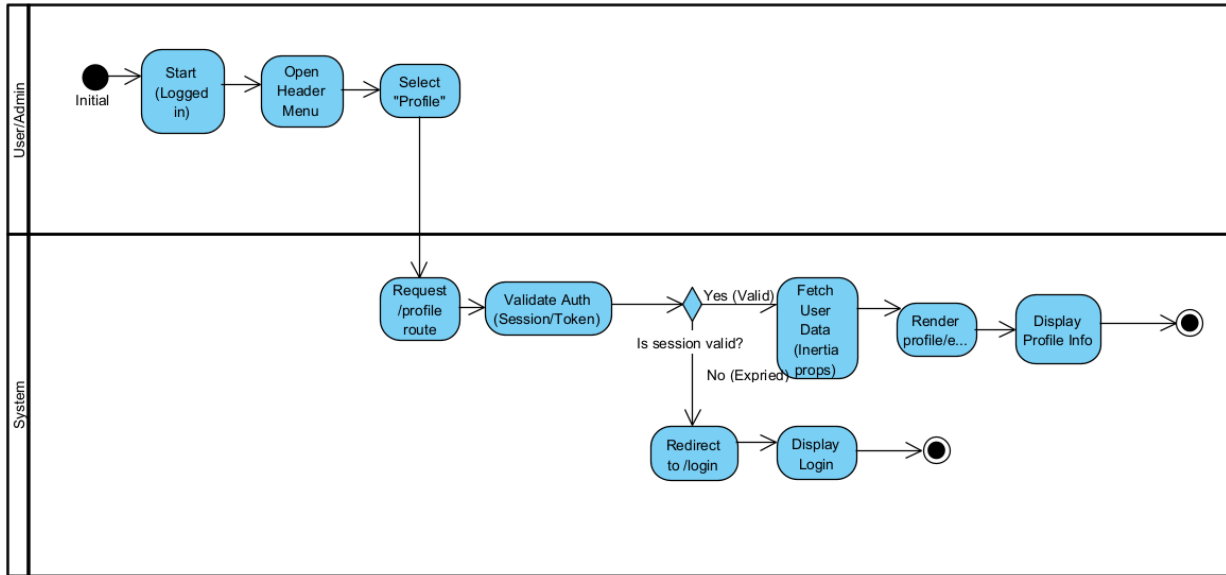
Hình ảnh 18 Delete message activity



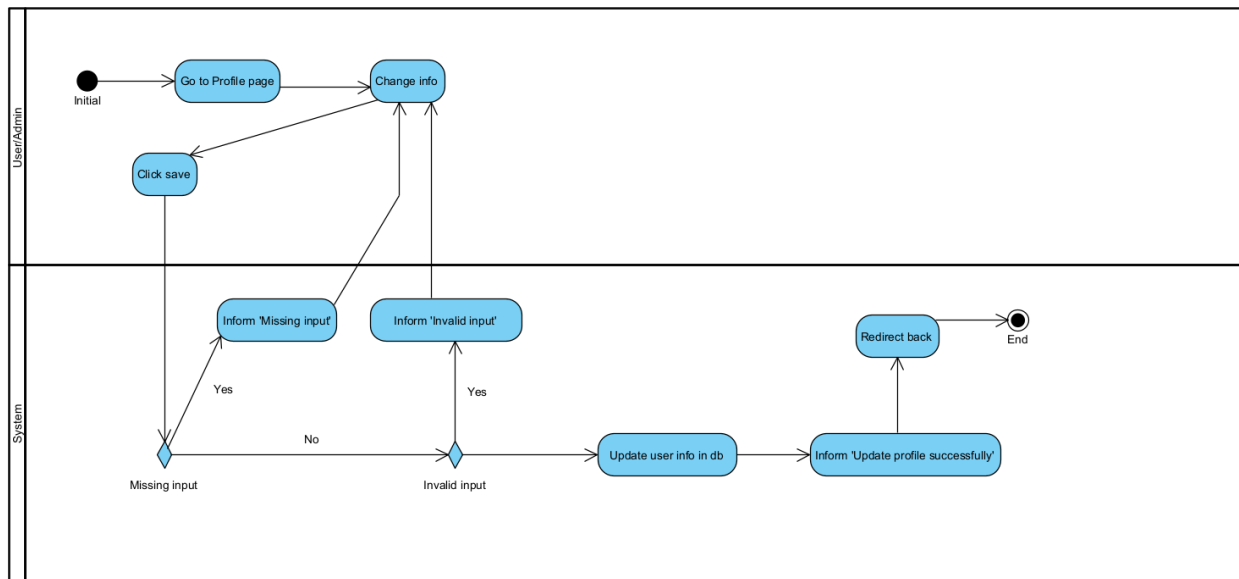
Hình ảnh 19 View messages activity



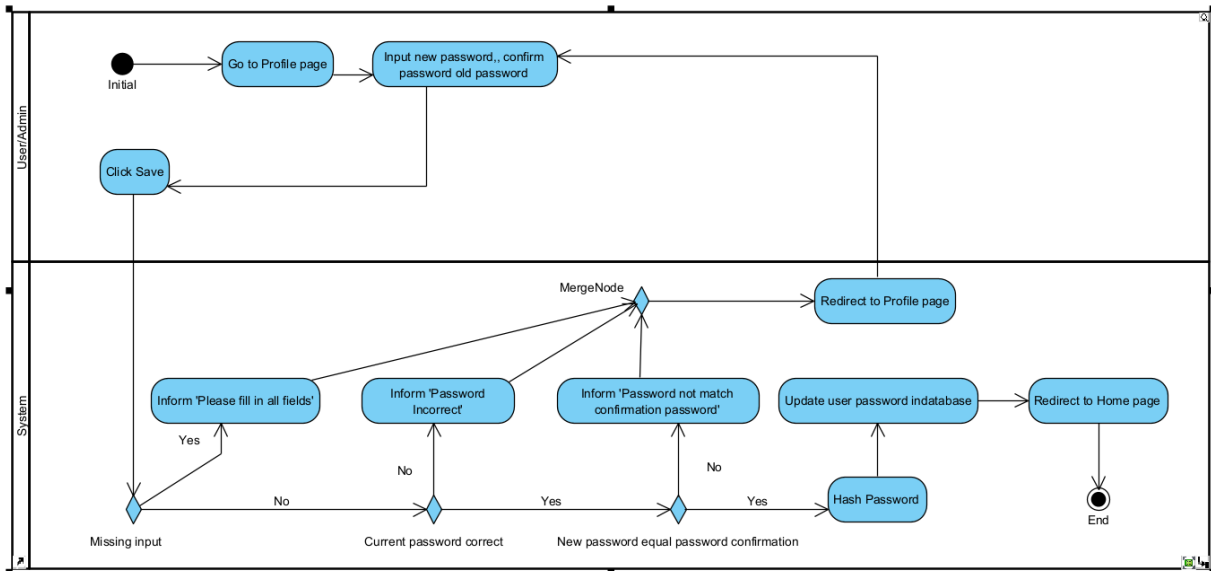
Hình ảnh 20 Search message activity



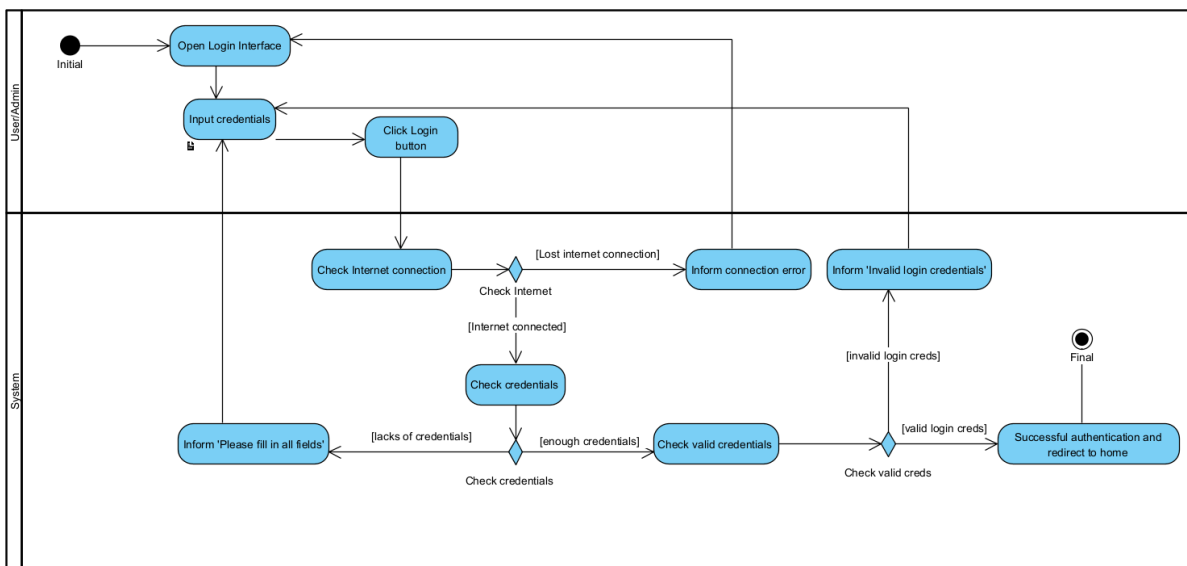
Hình ảnh 21 View profile activity



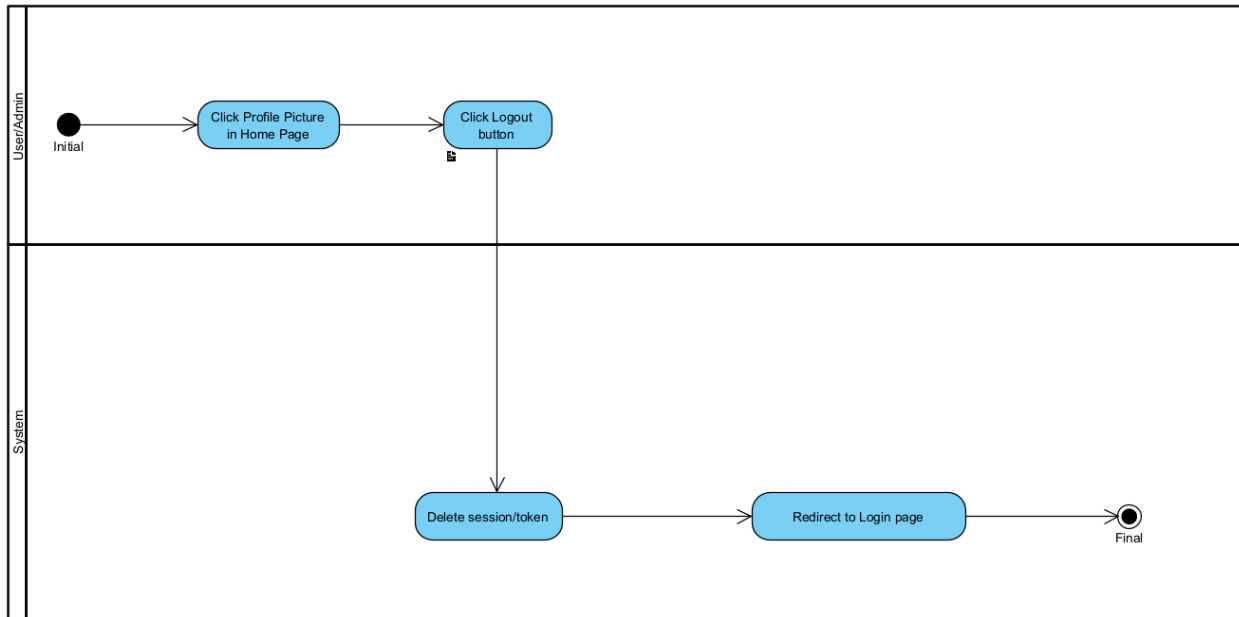
Hình ảnh 22 Edit profile activity



Hình ảnh 23 Change password activity

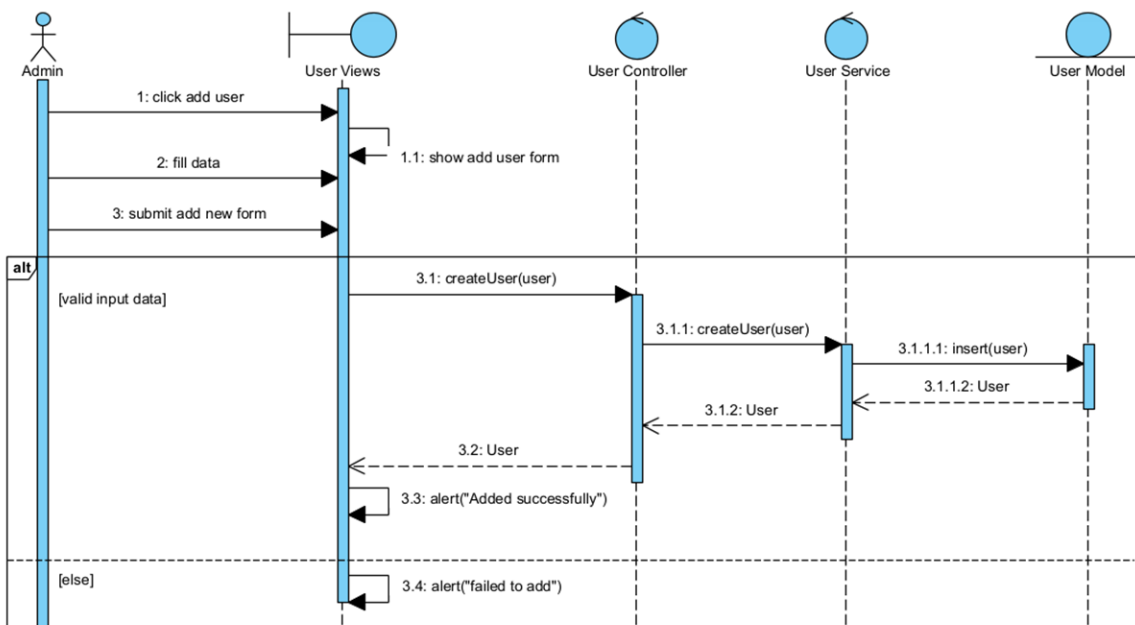


Hình ảnh 24 login activity

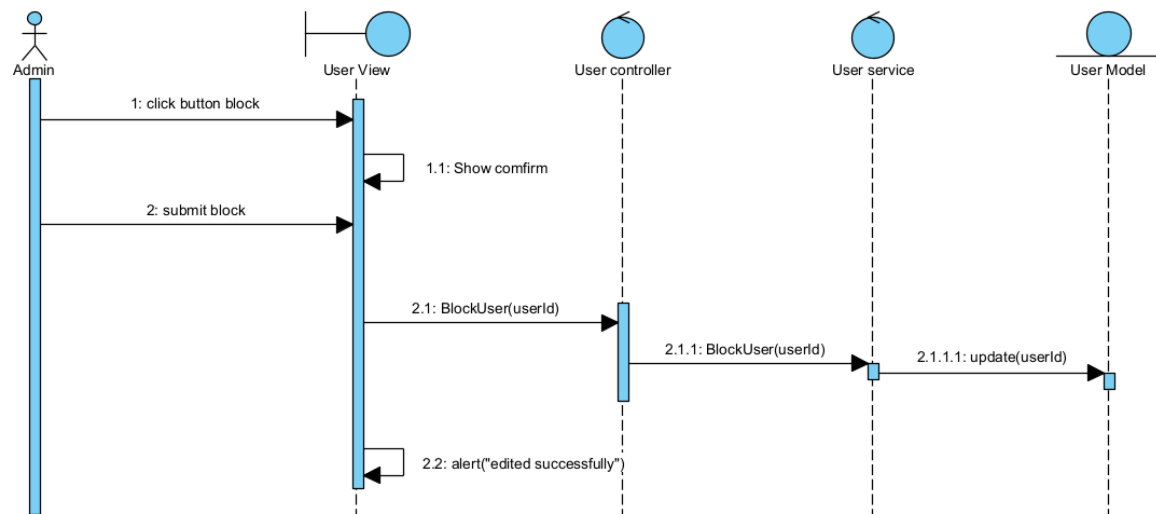


Hình ảnh 25 logout activity

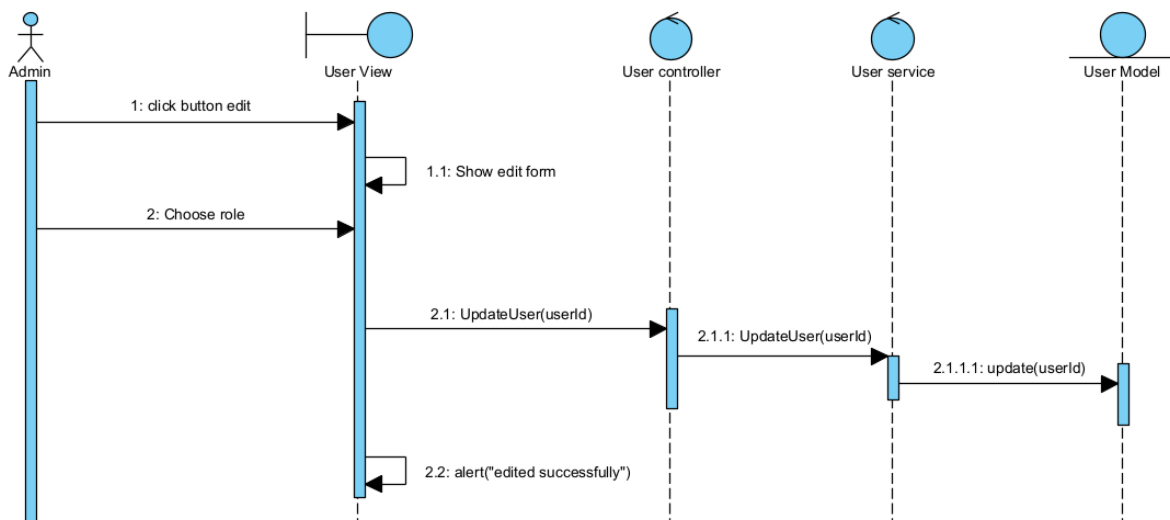
2.8 Biểu đồ sequence



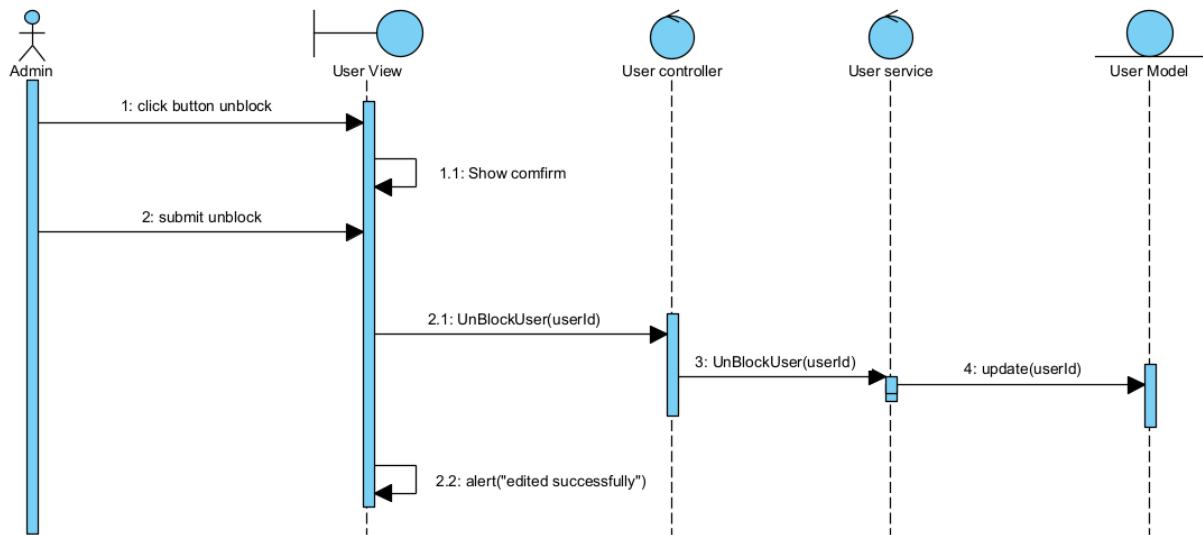
Hình ảnh 26 sq Add a new user



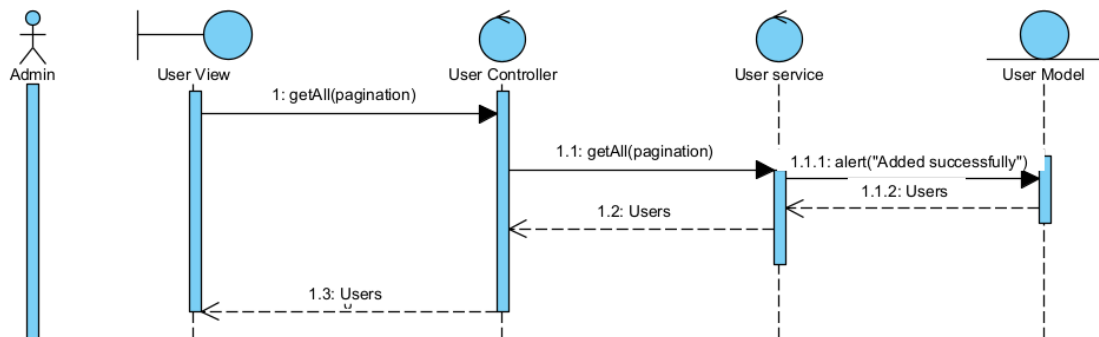
Hình ảnh 27 sq Block a user



Hình ảnh 28 sq Edit role user

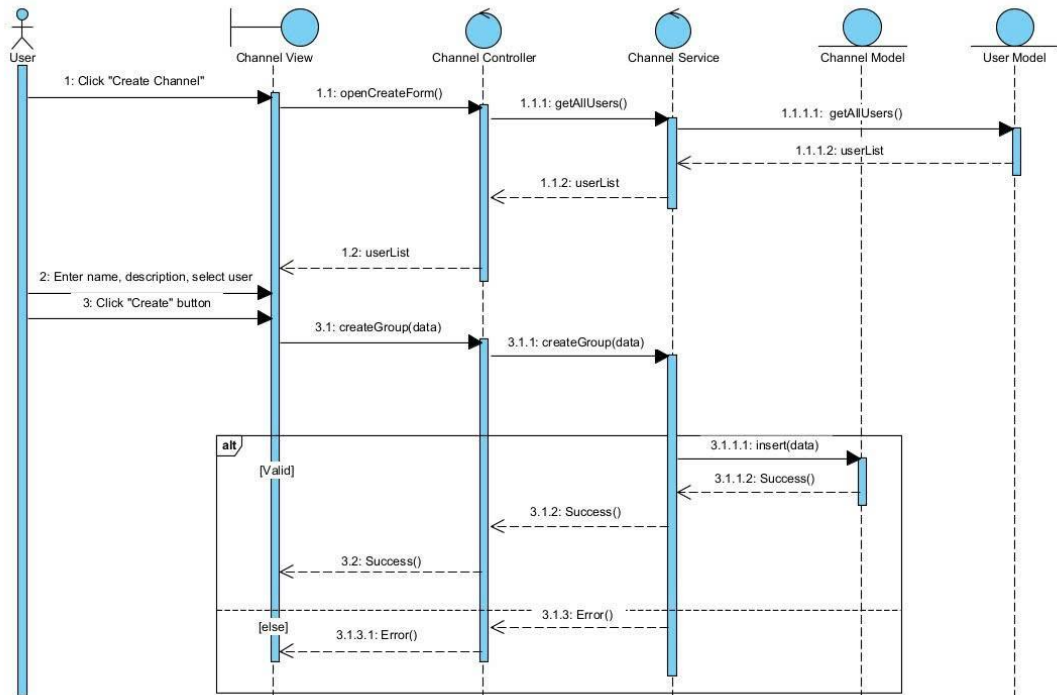


Hình ảnh 29 sq Unblock a user

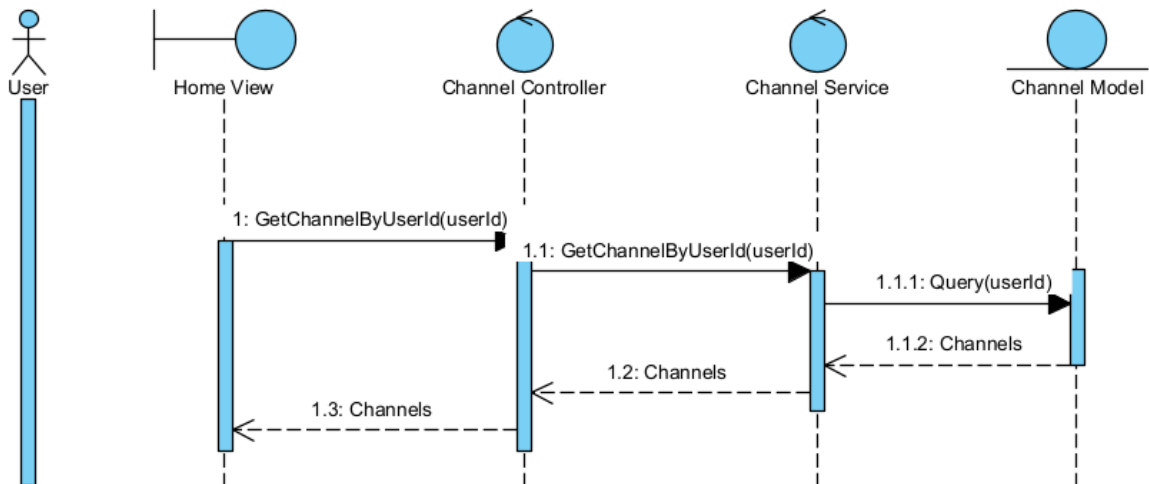


Hình ảnh 30 sq View Users

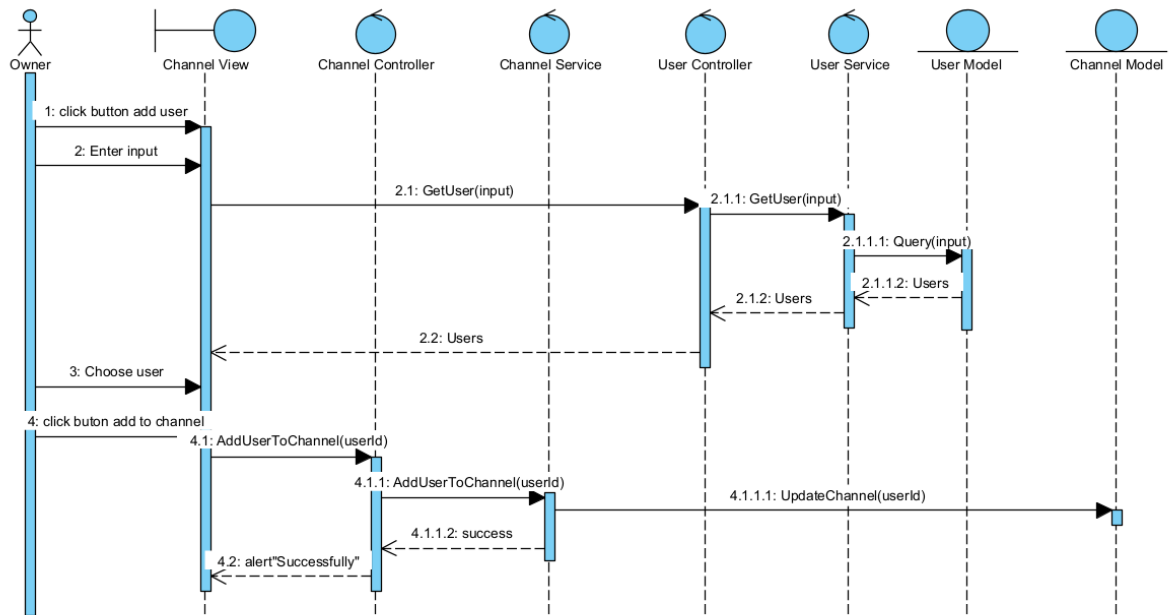
sd [Create Channel]



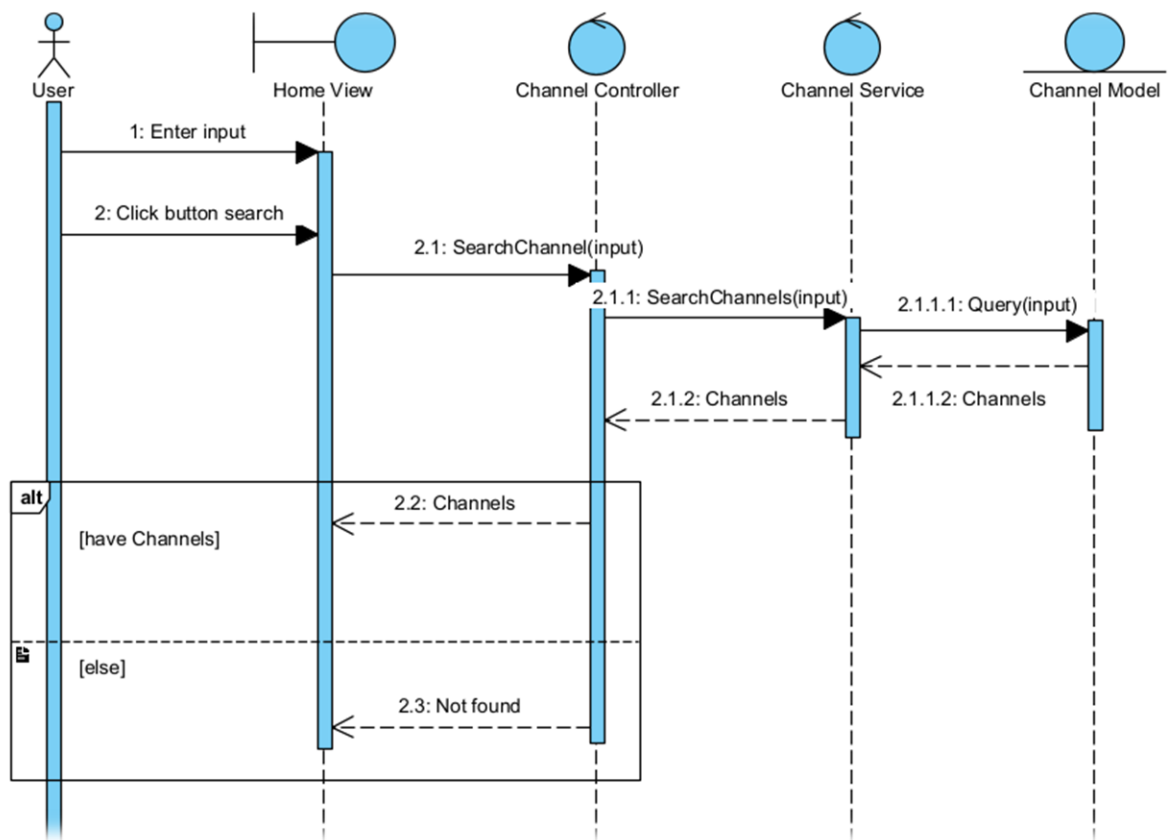
Hình ảnh 31 sq Create Conversation



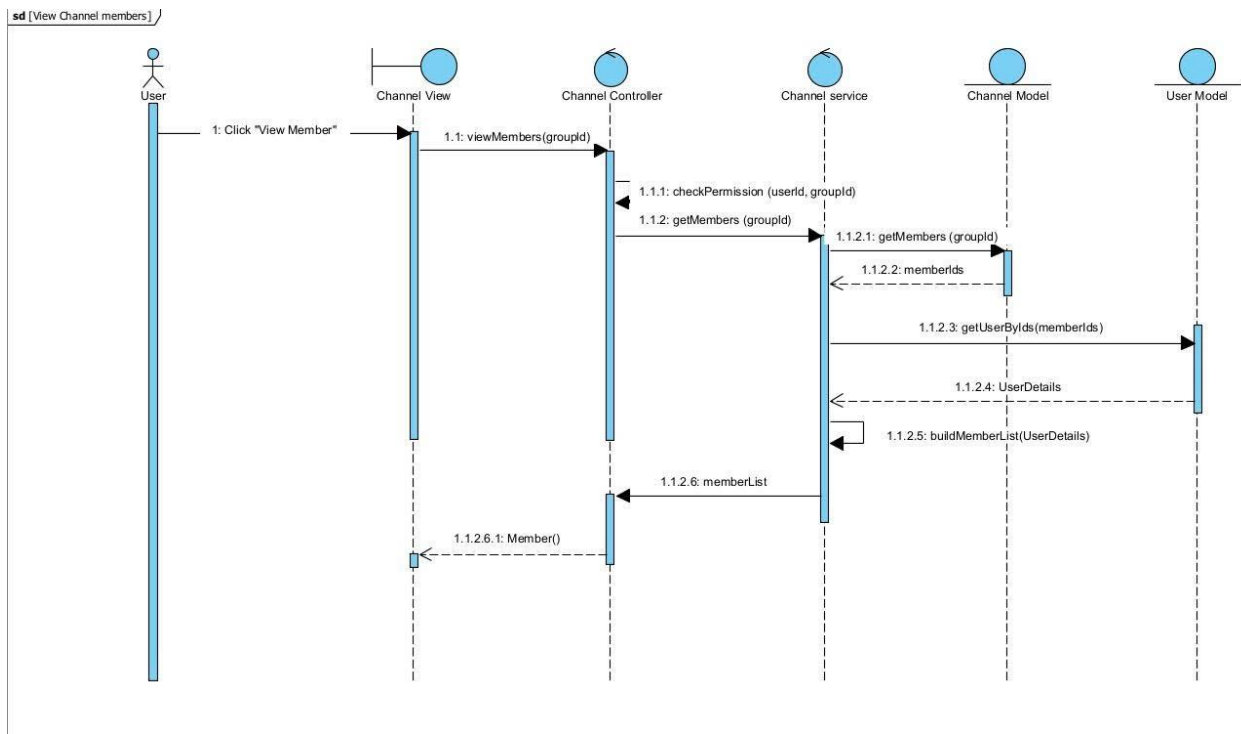
Hình ảnh 32 sq View Conversations



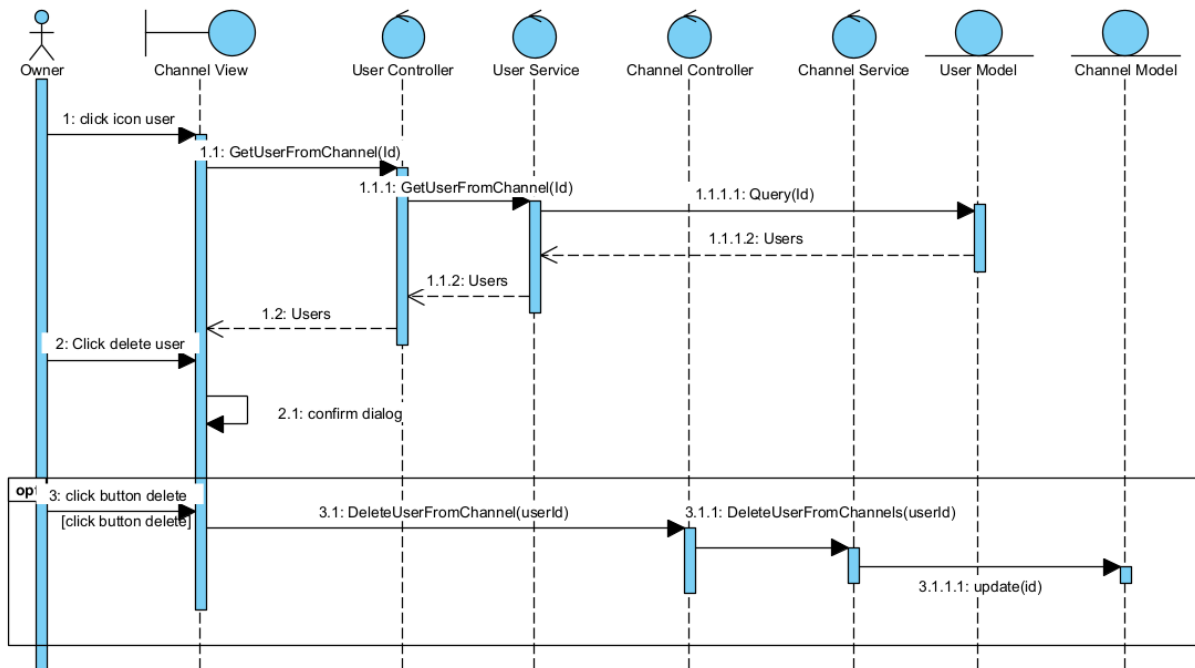
Hình ảnh 33 sq Add User to conversation



Hình ảnh 34 sq Search conversation

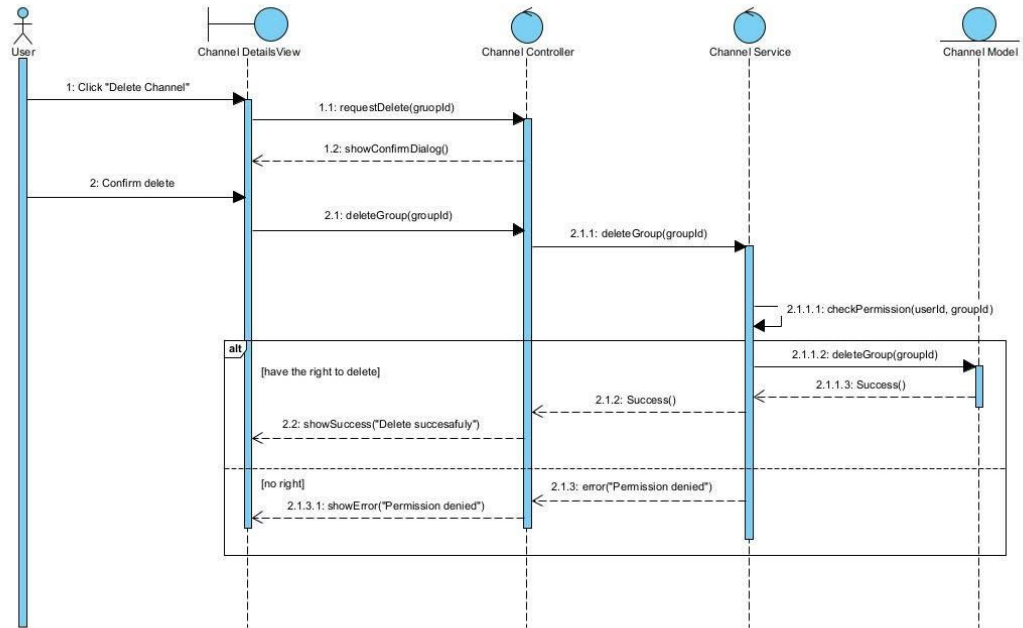


Hình ảnh 35 sq View conversation members

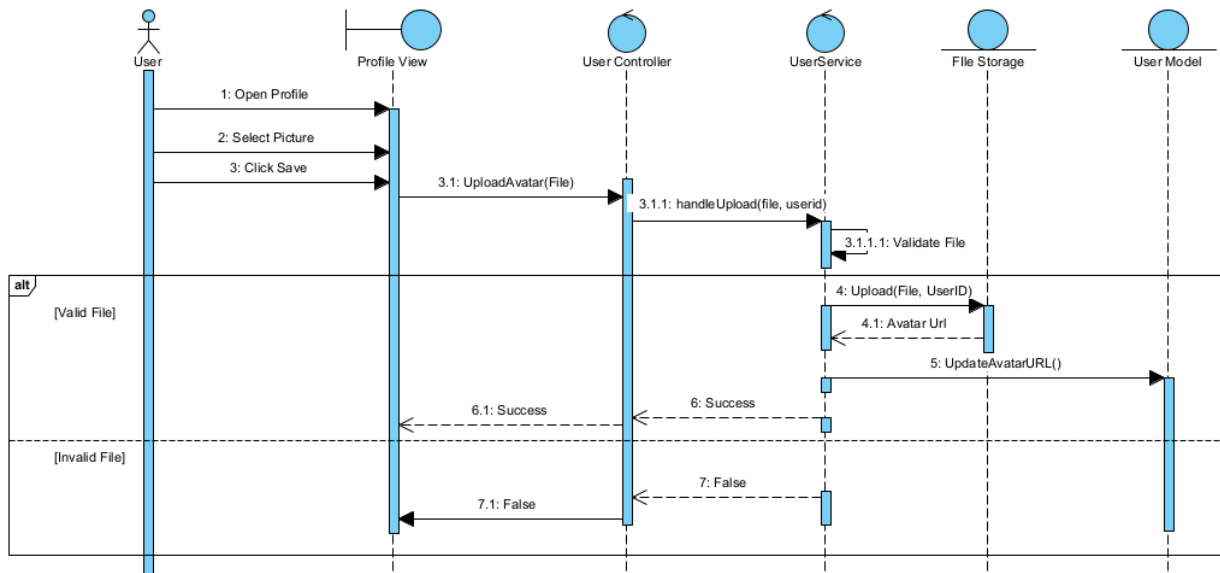


Hình ảnh 36 sq Remove user from conversation

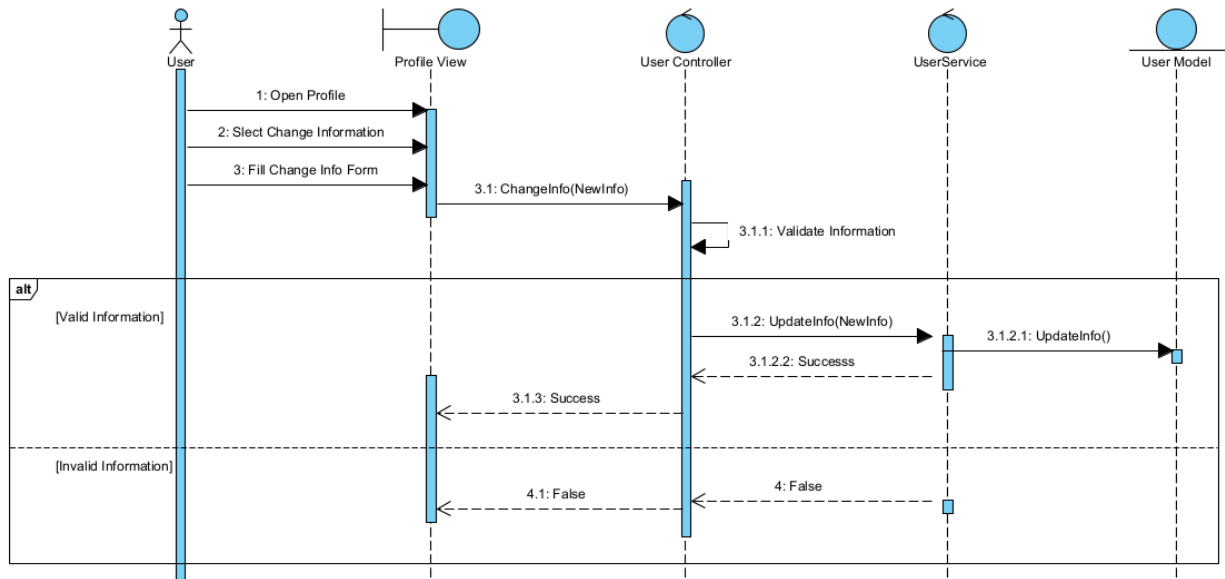
sd [Delete Channel]



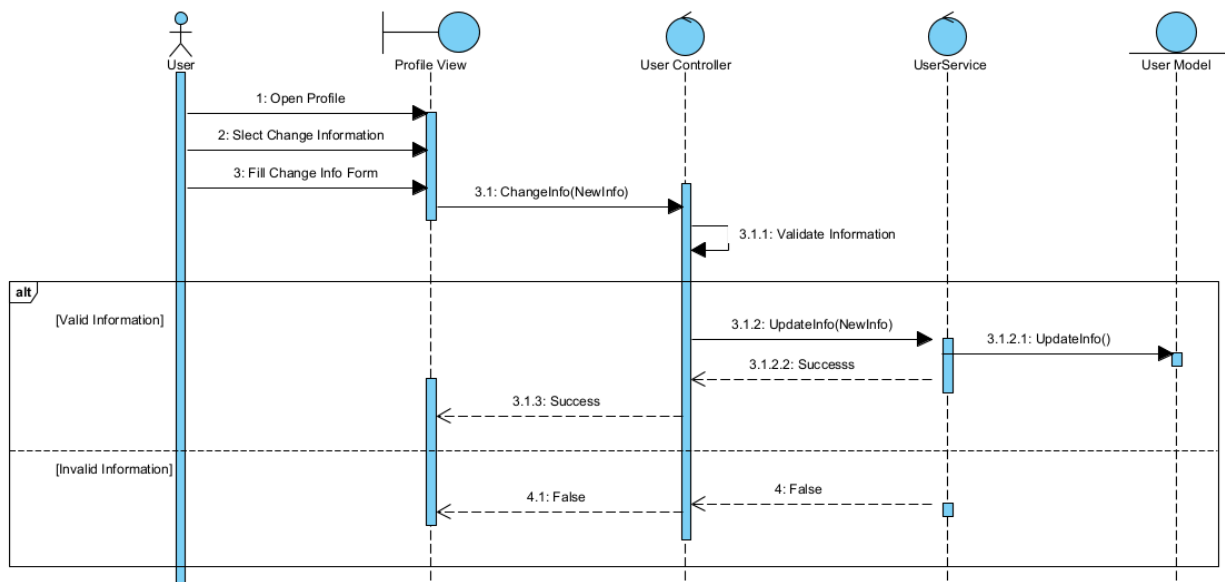
Hình ảnh 37 sq Delete conversation



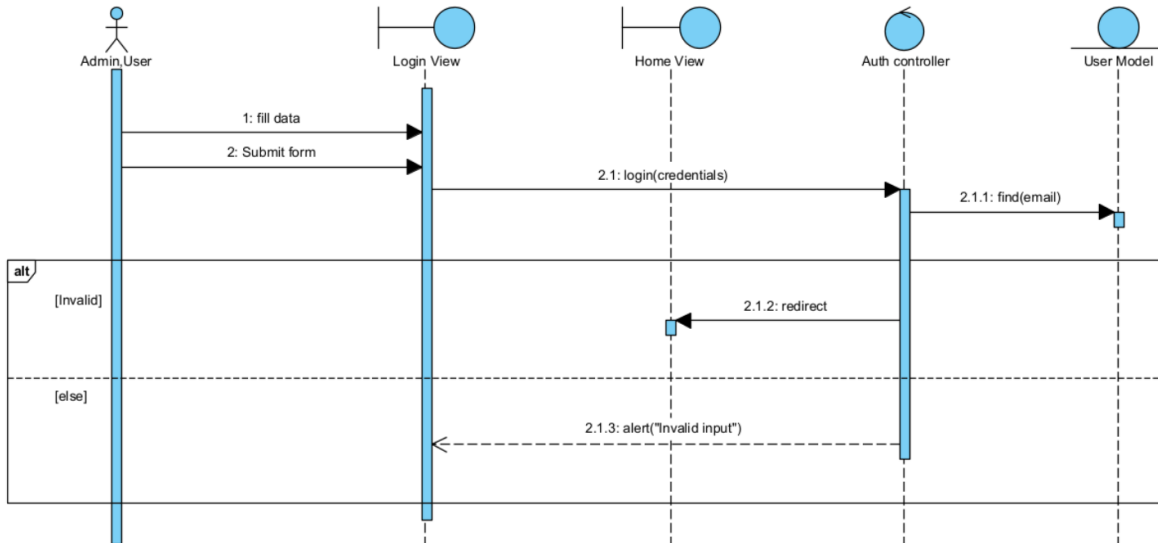
Hình ảnh 38 sq View Profile



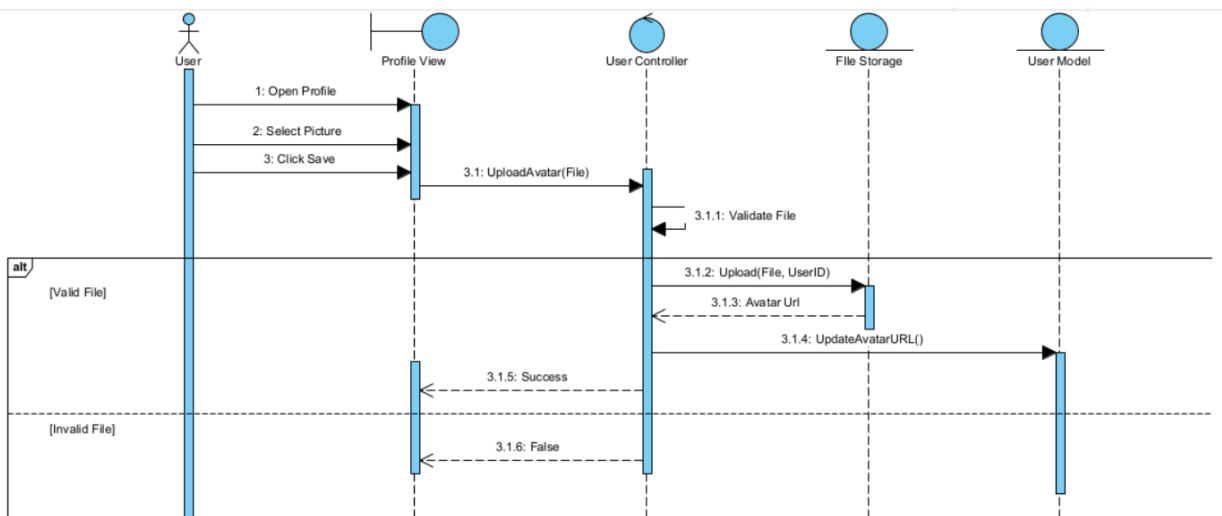
Hình ảnh 39 sq Update Profile



Hình ảnh 40 sq Change Password

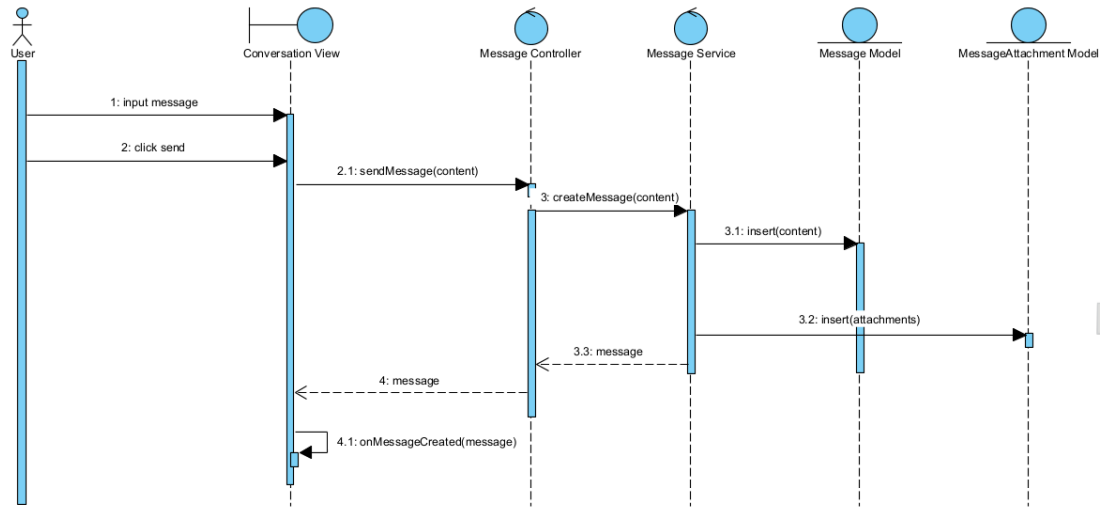


Hình ảnh 41 sq Login



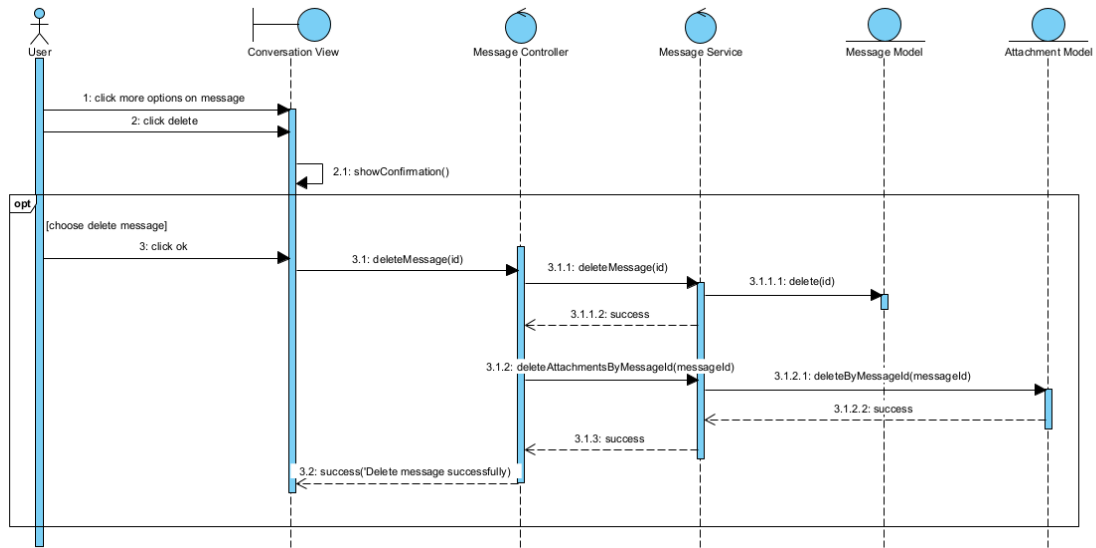
Hình ảnh 42 sq Update Avatar

sd [Send message Sequence Diagram]



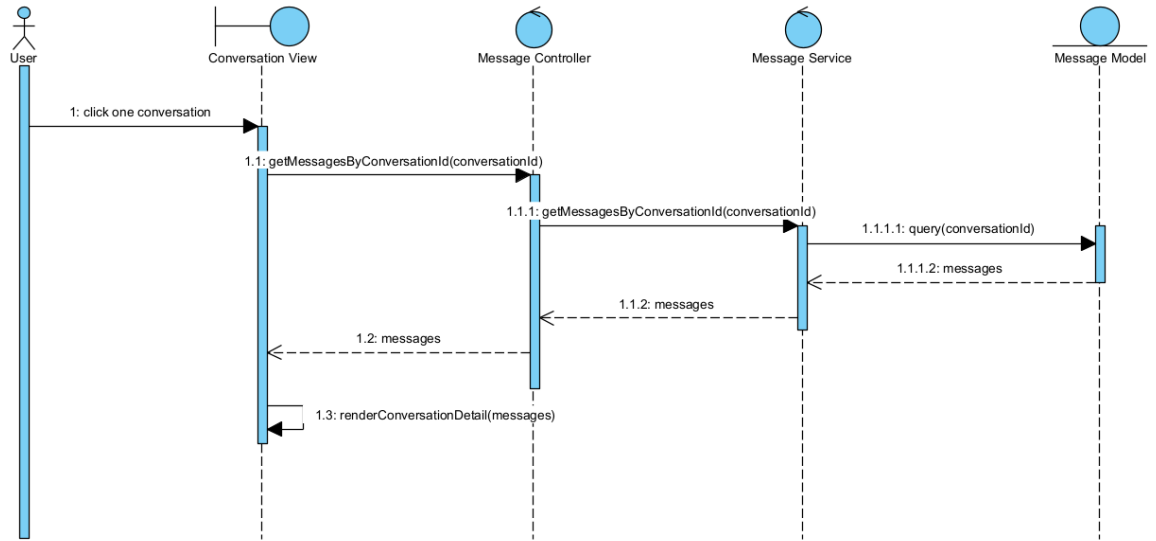
Hình ảnh 43 sq Send message

sd [Delete message Sequence Diagram]



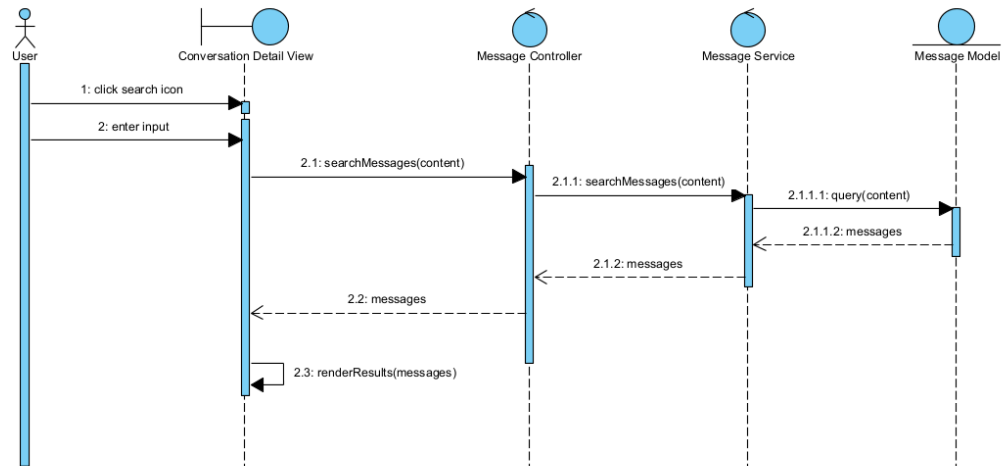
Hình ảnh 44 sq Delete message

sd [View message Sequence Diagram]



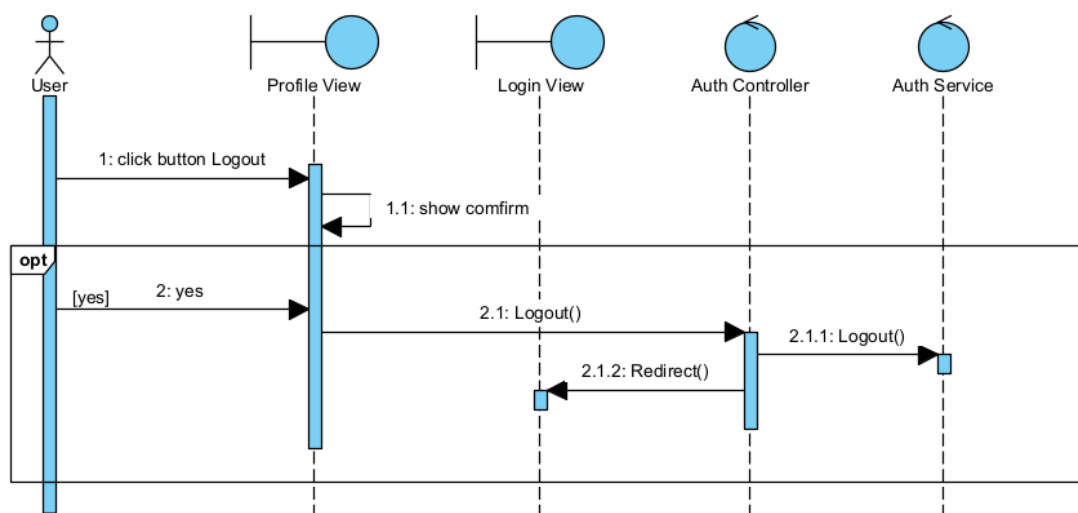
Hình ảnh 45 sq View messages

sd [Search message Sequence Diagram]



Hình ảnh 46 sq Search message

sq Logout



Hình ảnh 47 sq Logout

Chương 3: Cài đặt và triển khai hệ thống

3.1 Sơ đồ kiến trúc hệ thống

Để đáp ứng các yêu cầu khắt khe về một hệ thống trò chuyện thời gian thực (Real-time Chat application) như: tốc độ phản hồi cao, độ trễ thấp, giao diện mượt mà nhưng vẫn phải đảm bảo tính dễ bảo trì và tối ưu hóa thời gian phát triển, hệ thống được thiết kế dựa trên mô hình **Modern Monolith (Monolith hiện đại)** kết hợp với **Event-Driven Architecture (Kiến trúc hướng sự kiện)**.

Sự kết hợp này cho phép hệ thống tận dụng tối đa lợi thế quản lý mã nguồn và dữ liệu tập trung của kiến trúc Monolith, đồng thời giải quyết triệt để bài toán xử lý bất đồng bộ và tương tác thời gian thực nhờ cơ chế truyền tin dựa trên sự kiện. Sơ đồ cấu trúc và mối quan hệ tương tác giữa các tầng thành phần trong hệ thống

Kiến trúc tổng thể của hệ thống được phân rã cấu trúc thành 05 lớp chức năng cốt lõi với cơ chế phối hợp chặt chẽ, bao gồm:

- **Lớp Giao diện (Client Layer - Frontend SPA):** Sử dụng thư viện React hoạt động theo mô hình Single Page Application (Ứng dụng một trang). Lớp này chịu trách nhiệm quản lý trải nghiệm người dùng (React Components), trạng thái ứng dụng (React Context & Hooks), phân phối tín hiệu nội bộ (Client-side EventBus) và duy trì kết nối dòng dữ liệu trực tiếp thông qua thư viện Laravel Echo client.
- **Lớp Cầu nối dữ liệu (Bridge Layer - Inertia.js Protocol):** Sử dụng giao thức Inertia.js v2 đóng vai trò bộ chuyển đổi (Adapter) dữ liệu. Đây là thành phần cốt

lỗi hiện thực hóa kiến trúc *Modern Monolith*, giúp kết dính Frontend và Backend thành một khối thống nhất, loại bỏ hoàn toàn việc xây dựng một hệ thống RESTful API độc lập.

- **Lớp Máy chủ Ứng dụng (Application Server - Backend):** Vận hành trên nền tảng framework Laravel, quản lý toàn bộ hệ thống định tuyến (Routing) và các lớp kiểm soát trung gian (Middleware). Lớp này chứa các bộ điều khiển (Controllers) xử lý logic, các lớp xác thực dữ liệu đầu vào (Form Requests), tuần tự hóa dữ liệu đầu ra (API Resources) và các tiến trình xử lý bất đồng bộ (Model Observers, Queue Jobs).
- **Lớp Môi giới Thời gian thực (WebSocket Broker):** Sử dụng máy chủ WebSocket hiệu năng cao Laravel Reverb. Thành phần này đóng vai trò duy trì các kết nối song công (Duplex) và chịu trách nhiệm đẩy (Push) các sự kiện thay đổi dữ liệu từ phía máy chủ tới các Client đang lắng nghe ngay lập tức.
- **Lớp Cơ sở dữ liệu và Lưu trữ (Database & Storage Layer):** Hệ quản trị cơ sở dữ liệu quan hệ (MySQL/SQLite) chịu trách nhiệm lưu trữ các dữ liệu có cấu trúc (thông tin người dùng, kênh chat, tin nhắn). Kết hợp với hệ thống lưu trữ tệp tin (File Storage) để quản lý các dữ liệu phi cấu trúc như các tệp tin đính kèm (Attachments).

Thông qua việc phân định rõ ràng vai trò của từng lớp, hệ thống không chỉ đạt được sự tường minh trong việc xử lý dữ liệu truyền thống (qua giao thức HTTP XHR của Inertia) mà còn tối ưu hóa được luồng dữ liệu thời gian thực (qua giao thức WebSockets của Reverb), tạo tiền đề vững chắc cho việc phân tích chuyên sâu các kiến trúc phần mềm ở các mục tiếp theo.

3.2 Thiết kế lớp

3.2.1 Chức năng gửi tin nhắn

Khái niệm

Hệ thống xử lý tin nhắn nâng cao cấu trúc lại toàn bộ quy trình từ tiếp nhận request, phân loại loại tin nhắn, truy xuất dữ liệu cho đến kích hoạt sự kiện hệ thống bằng cách kết hợp nhiều mẫu thiết kế chuyên sâu:

- **Repository Pattern (IMessageRepo & MessageRepo):** Đóng gói toàn bộ logic liên quan đến truy xuất dữ liệu tin nhắn (lấy danh sách theo kênh, tìm kiếm tin nhắn). MessageService chỉ giao tiếp qua interface IMessageRepo để loại bỏ sự phụ thuộc trực tiếp vào Eloquent ORM.

- **Strategy Pattern** (SendMessageStrategy, TextOnlyMessageStrategy, FileMessageStrategy): Định nghĩa một cơ chế gửi tin nhắn linh hoạt. Khi người dùng gửi tin nhắn văn bản thuần túy hoặc kèm file, hệ thống sẽ tự hoán đổi thuật toán xử lý tương ứng (send) mà không làm thay đổi luồng chạy chính.
- **Factory Pattern (SendMessageStrategyFactory)**: Chịu trách nhiệm khởi tạo chiến lược gửi tin nhắn phù hợp (make(array data)) dựa trên cấu trúc dữ liệu đầu vào nhận được từ Controller.
- **Observer & Event Driven Pattern (MessageObserver, EventDispatcher)**: MessageModel được theo dõi bởi MessageObserver (triển khai từ MessageObserveInterface). Khi tin nhắn thay đổi trạng thái sinh tử (created, deleting), Observer sẽ phối hợp với EventDispatcher để phát đi sự kiện, giúp tự động cập nhật tin nhắn mới nhất lên ChanelModel hoặc dọn dẹp dữ liệu ở MessageAttachmentModel.
- **MessagePayloadBuilder**: Lớp đóng vai trò xây dựng, cho phép thiết lập từng thuộc tính của gói tin một cách linh hoạt, tuần tự thông qua các phương thức gọi nhớ (Fluent Interface).
- **MessagePayload**: Đối tượng dữ liệu cuối cùng (Data Transfer Object - DTO) chứa toàn bộ thông tin tin nhắn đã được chuẩn hóa và kiểm tra tính hợp lệ trước khi chuyển giao cho SendMessageService thực thi.

Ưu điểm cốt lõi của giải pháp

- Mở rộng không giới hạn (Open/Closed Principle): Khi phát sinh nhu cầu thêm loại tin nhắn mới (Ví dụ: Tin nhắn thoại - Voice Message, Tin nhắn vị trí), lập trình viên chỉ cần tạo thêm class Strategy mới mà không cần sửa đổi bất kỳ dòng code nào trong MessageService hay MessageController.
- Phân tách trách nhiệm hoàn hảo (SOLID): MessageService không còn phải chứa các hàm xử lý upload file phức tạp hay phân tách logic chuỗi. Tác vụ dọn dẹp file thuộc về MessageObserver, tác vụ lưu trữ thuộc về từng Strategy cụ thể.
- Type-safe và Giảm thiểu câu lệnh rẽ nhánh: Toàn bộ cấu trúc if/else lồng nhau dùng để phân biệt tin nhắn văn bản hay tin nhắn tệp đính kèm được loại bỏ hoàn toàn nhờ cơ chế tự quyết định (supports) của các Strategy kết hợp với Factory.
- Dễ dàng kiểm thử độc lập: Có thể viết các ca kiểm thử độc lập (Unit Test) cho luồng xử lý upload của FileMessageStrategy hoặc logic tìm kiếm của MessageRepo bằng cách sử dụng Mocking dễ dàng.
- **Loại bỏ Telescoping Constructor / Magic Arrays**: Không còn tình trạng truyền một danh sách tham số dài ngoằng hoặc một mảng không rõ các khóa (keys). Mỗi

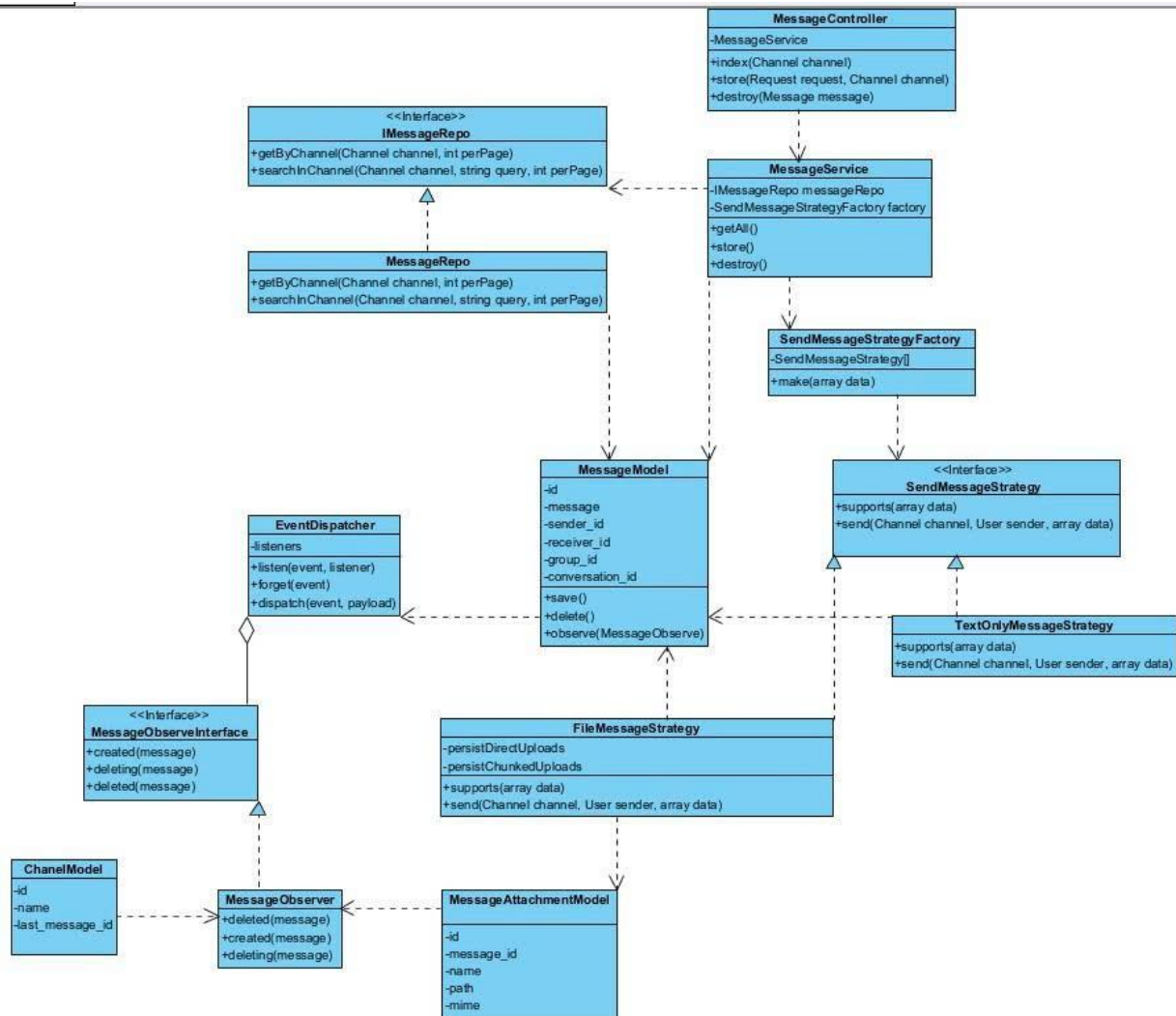
thuộc tính được gán thông qua một hàm có tên tường minh (forChannel, fromSender).

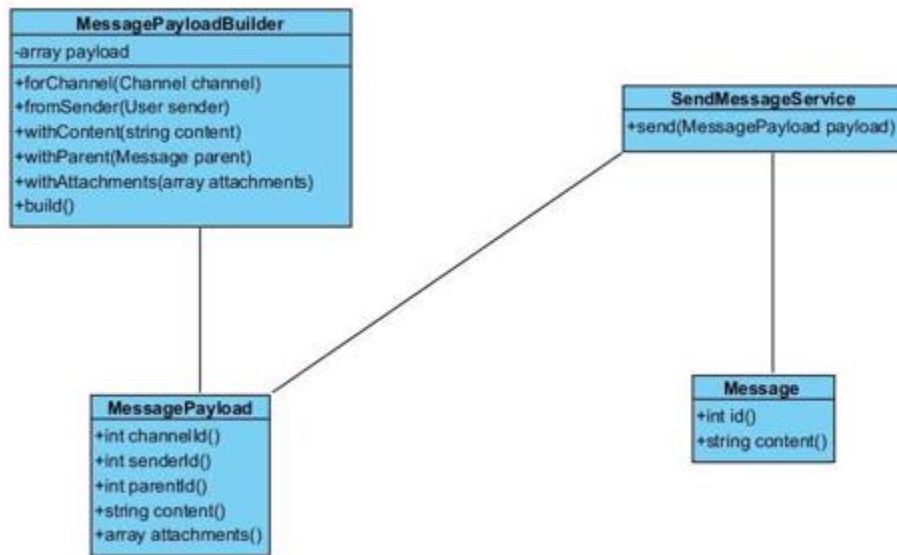
- **Tính linh hoạt cao (Flexible Construction):** Một tin nhắn có thể có hoặc không có tệp đính kèm, có hoặc không có tin nhắn cha (reply). Builder Pattern cho phép bỏ qua các bước không cần thiết một cách tự nhiên mà không cần truyền giá trị null thủ công.

Bảng so sánh

Tiêu chí	Trước Refactoring	Sau Refactoring
Phân loại tin nhắn	Dùng câu lệnh if/else check sự tồn tại của file trong Controller/Service để xử lý riêng.	Sử dụng SendMessageStrategyFactory để phân định chiến lược tự động.
Sự phụ thuộc (Coupling)	Controller phụ thuộc trực tiếp vào cả MessageModel lẫn MessageAttachmentModel.	Controller chỉ phụ thuộc duy nhất vào MessageService.
Quản lý sự kiện phụ (Side-effects)	Gọi thủ công các hàm gửi notification hoặc cập nhật trạng thái ngay sau lệnh lưu dữ liệu.	Tự động hóa thông qua cơ chế Hook (created, deleted) của MessageObserver.
Khả năng tái sử dụng	Logic xử lý tin nhắn bị bó hẹp trong phạm vi HTTP Request của Controller.	MessageService có thể được gọi và tái sử dụng ở bất kỳ đâu (API, Webhook, Jobs, Console Command).
Khả năng kiểm thử (Unit Test)	Khó kiểm thử độc lập vì logic nghiệp vụ bám chặt vào HTTP Context của Controller.	Dễ dàng viết Unit Test riêng cho MessageService bằng cách mock các Model dependencies.

Class diagram





Mã Nguồn

Repository Pattern:

IMessageRepo.php

```
<?php
```

```
namespace App\Repositories\Eloquent;
```

```
use App\Models\Channel;
```

```
use App\Models\Message;
```

```
use App\Repositories\Interfaces\IMessageRepo;
```

```
use Illuminate\Contracts\Pagination\LengthAwarePaginator;
```

```
class MessageRepo implements IMessageRepo
```

```
{
```

```
public function getByChannel(Channel $channel, int $perPage):  
LengthAwarePaginator
```

```
{  
    return Message::where('channel_id', $channel->id)  
        ->with(['sender', 'attachments'])  
        ->latest()  
        ->paginate($perPage);  
}
```

```
public function searchInChannel(Channel $channel, string $query, int $perPage):  
LengthAwarePaginator
```

```
{  
    return Message::where('channel_id', $channel->id)  
        ->where('content', 'like', '%' . $query . '%')  
        ->with(['sender', 'attachments'])  
        ->latest()  
        ->paginate($perPage);  
}  
}
```

MessageRepo.php

```
<?php
```

```
namespace App\Repositories\Eloquent;
```

```
use App\Models\Channel;
```

```

use App\Models\Message;
use App\Repositories\Interfaces\IMessageRepo;
use Illuminate\Contracts\Pagination\LengthAwarePaginator;

class MessageRepo implements IMessageRepo
{
    public function getByChannel(Channel $channel, int $perPage):
    LengthAwarePaginator
    {
        return Message::where('channel_id', $channel->id)
            ->with(['sender', 'attachments'])
            ->latest()
            ->paginate($perPage);
    }

    public function searchInChannel(Channel $channel, string $query, int $perPage):
    LengthAwarePaginator
    {
        return Message::where('channel_id', $channel->id)
            ->where('content', 'like', '%' . $query . '%')
            ->with(['sender', 'attachments'])
            ->latest()
            ->paginate($perPage);
    }
}

```

Strategy Pattern:

SendMessageStrategy.php

```

<?php
namespace App\Strategies;
use App\Models\Channel;
use App\Models\User;
use App\Models\Message;
interface SendMessageStrategy
{
/**
 * Kiểm tra xem Strategy này có hỗ trợ dữ liệu gửi lên không.
 *
 * @param array<string, mixed> $data
 */
public function supports(array $data): bool;
/**
 * Thực hiện gửi tin nhắn.
 *
 * @param array<string, mixed> $data
 */
public function send(Channel $channel, User $sender, array $data): Message;
}

```

TextOnlyMessageStrategy.php

```

<?php

namespace App\Strategies;

```

```

use App\Models\Channel;
use App\Models\User;
use App\Models\Message;

class TextOnlyMessageStrategy implements SendMessageStrategy
{
    public function supports(array $data): bool
    {
        // Hỗ trợ nếu không chứa file đính kèm trực tiếp hoặc chunk upload
        return empty($data['attachments']) && empty($data['uploaded_attachments']);
    }

    public function send(Channel $channel, User $sender, array $data): Message
    {
        return Message::create([
            'channel_id' => $channel->id,
            'sender_id' => $sender->id,
            'content' => $data['content'] ?? null,
            'parent_id' => $data['parent_id'] ?? null,
        ]);
    }
}

```

FileMessageStrategy.php

```

<?php
namespace App\Strategies;

```



```

use App\Models\Channel;
use App\Models\User;
use App\Models\Message;
use App\Models\MessageAttachment;
use Illuminate\Support\Facades\Storage;
use Illuminate\Support\Str;
class FileMessageStrategy implements SendMessageStrategy
{
    public function supports(array $data): bool
    {
        // Hỗ trợ nếu có chứa file đính kèm thông thường hoặc file chunk upload
        // lớn
        return !empty($data['attachments']) ||
            !empty($data['uploaded_attachments']);
    }

    public function send(Channel $channel, User $sender, array $data): Message
    {
        $message = Message::create([
            'channel_id' => $channel->id,
            'sender_id' => $sender->id,
            'content' => $data['content'] ?? null,
            'parent_id' => $data['parent_id'] ?? null,
        ]);
        if (!empty($data['attachments'])) {
            $this->persistDirectUploads($message, $data['attachments']);
        }
        if (!empty($data['uploaded_attachments'])) {

```

```

        $this->persistChunkedUploads($message, $data['uploaded_attachments']);
    }
    return $message;
}
/**
 * Xử lý lưu các file upload trực tiếp
 */
private function persistDirectUploads(Message $message, array $files): void
{
    foreach ($files as $file) {
        $directory = 'attachments/' . Str::random(40);
        Storage::disk('public')->makeDirectory($directory);
        MessageAttachment::create([
            'message_id' => $message->id,
            'path' => $file->store($directory, 'public'),
            'name' => $file->getClientOriginalName(),
            'size' => $file->getSize(),
            'mime' => $file->getClientMimeType(),
            'storage_disk' => 'public',
        ]);
    }
    .....
}
}
}

```

SendMessageStrategyFactory.php

```
<?php
namespace App\Strategies;
use InvalidArgumentException;

class SendMessageStrategyFactory
{
    /**
     * @param SendMessageStrategy[] $strategies
     */
    public function __construct(
        private array $strategies
    ) {}

    /**
     * Lấy ra Strategy phù hợp dựa trên payload gửi lên.
     */
    public function make(array $data): SendMessageStrategy
    {
        foreach ($this->strategies as $strategy) {
            if ($strategy->supports($data)) {
                return $strategy;
            }
        }
    }
}
```

```

        throw new InvalidArgumentException("Không tìm thấy Strategy phù hợp cho dữ
        liệu tin nhắn này.");
    }
}

```

Service & Controller:

MessageService.php

```
<?php
```

```
namespace App\Services;
```

```
use App\Models\Channel;
```

```
use App\Models\Message;
```

```
use App\Repositories\Interfaces\IMessageRepo;
```

```
use App\Strategies\SendMessageStrategyFactory;
```

```
use Illuminate\Contracts\Pagination\LengthAwarePaginator;
```

```
class MessageService
```

```
{
```

```
    public function __construct(
```

```
        private IMessageRepo $messageRepo,
```

```
        private SendMessageStrategyFactory $factory
```

```
    ) {}
```

```
    public function getAll(Channel $channel, int $perPage = 10): LengthAwarePaginator
```

```
{
```

```
        return $this->messageRepo->getByChannel($channel, $perPage);
```

```
}
```

```
public function store(Channel $channel, array $data): Message
```

```
{
```

```
    $strategy = $this->factory->make($data);
```

```
    $sender = auth()->user();
```

```
    return $strategy->send($channel, $sender, $data);
```

```
}
```

```
public function destroy(Message $message): void
```

```
{
```

```
    $message->delete();
```

```
}
```

```
}
```

MessageController.php

```
<?php
```

```
namespace App\Http\Controllers;
```

```
use App\Http\Requests\StoreMessageRequest;
```

```
use App\Http\Resources\MessageResource;
```

```
use App\Models\Channel;
```

```
use App\Models\Message;
```

```
use App\Services\MessageService;
```

```
use Illuminate\Http\JsonResponse;
```

```
use Illuminate\Support\Facades\DB;
```

```
class MessageController extends Controller
```

```
{
```

```
    public function __construct(
```

```
        private MessageService $messageService
```

```
    ) {}
```

```
    public function index(Channel $channel)
```

```
    {
```

```
        $this->authorizeChannelAccess($channel);
```

```
        $messages = $this->messageService->getAll($channel);
```

```
        return MessageResource::collection($messages);
```

```
    }
```

```
    public function store(StoreMessageRequest $request, Channel $channel):  
    MessageResource
```

```
    {
```

```
        $this->authorizeChannelAccess($channel);
```

```
        $data = $request->validated();
```

```
        $message = DB::transaction(function () use ($channel, $data) {
```

```
            return $this->messageService->store($channel, $data);
```

```
        });
```

```

    return new MessageResource($message);
}

public function destroy(Message $message): JsonResponse
{
    abort_unless($message->sender_id === auth()->id(), 403, 'Unauthorized');

    DB::transaction(function () use ($message) {
        $this->messageService->destroy($message);
    });

    return response()->json(['message' => 'Deleted successfully']);
}

private function authorizeChannelAccess(Channel $channel): void
{
    $isUserInChannel = auth()->user()?->channels()->whereKey($channel->id)->exists();
    abort_unless($isUserInChannel, 403, 'Unauthorized');
}
}

```

Observer & Event Dispatcher:

EventDispatcher.php

```
<?php
```

```
namespace App\Events;
```

```
class EventDispatcher
```

```
{
```

```
    /**
```

```
     * @var array<string, array<callable>>
```

```
     */
```

```
    private array $listeners = [];
```

```
    public function listen(string $event, callable $listener): void
```

```
    {
```

```
        $this->listeners[$event][] = $listener;
```

```
    }
```

```
    public function forget(string $event): void
```

```
    {
```

```
        unset($this->listeners[$event]);
```

```
    }
```

```
    public function dispatch(string $event, mixed $payload): void
```

```
    {
```

```
        if (isset($this->listeners[$event])) {
```

```
            foreach ($this->listeners[$event] as $listener) {
```



```

        $listener($payload);
    }
}
}
}

```

MessageObserveInterface.php

```

<?php
namespace App\Observers;
use App\Models\Message;

interface MessageObserveInterface
{
    public function created(Message $message): void;

    public function deleting(Message $message): void;

    public function deleted(Message $message): void;
}

```

MessageObserver.php

```

<?php
namespace App\Observers;

use App\Models\Channel;
use App\Models\Message;

```

```

use App\Observers\MessageObserveInterface;
use Illuminate\Support\Facades\Storage;

class MessageObserver implements MessageObserveInterface
{
    public function created(Message $message): void
    {
        // Cập nhật thuộc last_message_id tránh tranh chấp dữ liệu (race condition)
        Channel::whereKey($message->channel_id)
            ->where(function ($query) use ($message) {
                $query->whereNull('last_message_id')
                    ->orWhere('last_message_id', '<', $message->id);
            })
            ->update(['last_message_id' => $message->id]);
    }

    public function deleting(Message $message): void
    {
        // Dọn dẹp thư mục lưu trữ file đính kèm trước khi xóa khỏi DB
        $message->attachments()->each(function ($attachment) {
            if (!$attachment->path) {
                return;
            }

            $disk = $attachment->storage_disk ?? 'public';
            $dir = dirname((string) $attachment->path);
            Storage::disk($disk)->deleteDirectory($dir);
        });
    }
}

```

```

    });

    $message->attachments()->delete();
}

public function deleted(Message $message): void
{
    $channel = Channel::find($message->channel_id);

    if (!$channel) {
        return;
    }

    // Tính toán lại last_message_id chỉ khi tin nhắn bị xóa chính là tin nhắn mới nhất
    if ((int) $channel->last_message_id !== (int) $message->id) {
        return;
    }

    $newLastMessage = Message::where('channel_id', $message->channel_id)
        ->latest()
        ->first();

    $channel->update(['last_message_id' => $newLastMessage->id]);
}
}

```

Eloquent Models:

Message.php

```
<?php
```

```
namespace App\Models;
```

```
use App\Events\EventDispatcher;
```

```
use App\Observers\MessageObserveInterface;
```

```
use Illuminate\Database\Eloquent\Model;
```

```
use Illuminate\Database\Eloquent\Relations\HasMany;
```

```
use Illuminate\Database\Eloquent\Relations\BelongsTo;
```

```
class Message extends Model
```

```
{
```

```
    protected $fillable = [
```

```
        'channel_id',
```

```
        'sender_id',
```

```
        'content',
```

```
        'parent_id',
```

```
    ];
```

```
    private static ?EventDispatcher $dispatcher = null;
```

```
    private static function getDispatcher(): EventDispatcher
```

```
    {
```

```
        if (self::$dispatcher === null) {
```

```
            self::$dispatcher = new EventDispatcher();
```

```
        }
```

```

    return self::$dispatcher;
}

/**
 * Cho phép các Observers đăng ký sự kiện lắng nghe Model qua Custom Event
Dispatcher
 */
public static function observeMessage(MessageObserveInterface $observer): void
{
    $dispatcher = self::getDispatcher();

    $dispatcher->listen('created', fn (Message $message) => $observer->created($message));

    $dispatcher->listen('deleting', fn (Message $message) => $observer->deleting($message));

    $dispatcher->listen('deleted', fn (Message $message) => $observer->deleted($message));
}

protected static function booted(): void
{
    static::created(function (Message $message) {
        self::getDispatcher()->dispatch('created', $message);
    });

    static::deleting(function (Message $message) {
        self::getDispatcher()->dispatch('deleting', $message);
    });
}

```

```

        static::deleted(function (Message $message) {
            self::getDispatcher()->dispatch('deleted', $message);
        });
    }

    public function attachments(): HasMany
    {
        return $this->hasMany(MessageAttachment::class);
    }

    public function sender(): BelongsTo
    {
        return $this->belongsTo(User::class, 'sender_id');
    }
}

```

Channel.php

```

<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Channel extends Model
{
    protected $fillable = [
        'name',
        'last_message_id',
    ];
}

```

```
];  
}
```

MessageAttachment.php

```
<?php  
namespace App\Models;  
use Illuminate\Database\Eloquent\Model;  
  
class MessageAttachment extends Model  
{  
    protected $fillable = [  
        'message_id',  
        'name',  
        'path',  
        'mime',  
        'size',  
        'storage_disk',  
    ];  
}
```

Builder Pattern:

MessagePayload.php

```
<?php  
  
namespace App\DTOs;  
  
class MessagePayload
```

```

{
    /**
     * Khởi tạo đối tượng bất biến chứa dữ liệu tin nhắn.
     *
     * @param array<\Illuminate\Http\UploadedFile> $attachments
     */
    public function __construct(
        private int $channelId,
        private int $senderId,
        private ?int $parentId,
        private ?string $content,
        private array $attachments
    ) {}

    public function channelId(): int
    {
        return $this->channelId;
    }

    public function senderId(): int
    {
        return $this->senderId;
    }

    public function parentId(): ?int
    {
        return $this->parentId;
    }

```



```

    }

    public function content(): ?string
    {
        return $this->content;
    }

    /**
     * @return array<\Illuminate\Http\UploadedFile>
     */
    public function attachments(): array
    {
        return $this->attachments;
    }
}

```

MessagePayloadBuilder.php

```

<?php

namespace App\Builders;

use App\DTOs\MessagePayload;
use App\Models\Channel;
use App\Models\User;
use App\Models\Message;
use InvalidArgumentException;

```

```

class MessagePayloadBuilder
{
    /**
     * Mảng tạm lưu giữ cấu trúc payload khi đang xây dựng.
     *
     * @var array<string, mixed>
     */
    private array $payload = [
        'channel_id' => null,
        'sender_id' => null,
        'parent_id' => null,
        'content' => null,
        'attachments' => [],
    ];

    public function forChannel(Channel $channel): self
    {
        $this->payload['channel_id'] = $channel->id;
        return $this; // Trả về chính nó để hỗ trợ Method Chaining
    }

    public function fromSender(User $sender): self
    {
        $this->payload['sender_id'] = $sender->id;
        return $this;
    }
}

```

```

public function withContent(string $content): self
{
    $this->payload['content'] = $content;
    return $this;
}

```

```

public function withParent(Message $parent): self
{
    $this->payload['parent_id'] = $parent->id;
    return $this;
}

```

```

/**
 * @param array<\Illuminate\Http\UploadedFile> $attachments
 */
public function withAttachments(array $attachments): self
{
    $this->payload['attachments'] = $attachments;
    return $this;
}

```

```

/**
 * Tạo và đóng gói dữ liệu thành đối tượng bất biến MessagePayload.
 */
public function build(): MessagePayload
{
    // Kiểm tra tính hợp lệ trước khi đóng gói (Validation)

```

```

        if ($this->payload['channel_id'] === null) {
            throw new InvalidArgumentException("Yêu cầu thông tin Channel để tạo
            MessagePayload.");
        }

        if ($this->payload['sender_id'] === null) {
            throw new InvalidArgumentException("Yêu cầu thông tin Người gửi để tạo
            MessagePayload.");
        }

        return new MessagePayload(
            channelId: (int) $this->payload['channel_id'],
            senderId: (int) $this->payload['sender_id'],
            parentId: $this->payload['parent_id'] ? (int) $this->payload['parent_id'] : null,
            content: $this->payload['content'],
            attachments: $this->payload['attachments']
        );
    }
}

```

SendMessageService.php

```
<?php
```

```
namespace App\Services;
```

```
use App\DTOs\MessagePayload;
```

```
use App\Models\Message;
```

```

use App\Models\MessageAttachment;
use Illuminate\Support\Facades\Storage;
use Illuminate\Support\Str;

class SendMessageService
{
    /**
     * Thực hiện gửi tin nhắn dựa trên Payload đã được build
     */
    public function send(MessagePayload $payload): Message
    {
        $message = Message::create([
            'channel_id' => $payload->channelId(),
            'sender_id' => $payload->senderId(),
            'content' => $payload->content(),
            'parent_id' => $payload->parentId(),
        ]);

        // Lưu tệp đính kèm nếu có
        foreach ($payload->attachments() as $file) {
            $directory = 'attachments/' . Str::random(40);
            Storage::disk('public')->makeDirectory($directory);

            MessageAttachment::create([
                'message_id' => $message->id,
                'path' => $file->store($directory, 'public'),
                'name' => $file->getClientOriginalName(),
            ]);
        }
    }
}

```

```

        'size'      => $file->getSize(),
        'mime'      => $file->getClientMimeType(),
        'storage_disk' => 'public',
    ]);
}

return $message;
}
}

```

Message.php

```

<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Message extends Model
{
    protected $fillable = [
        'channel_id',
        'sender_id',
        'content',
        'parent_id',
    ];

    public function id(): int
    {
        return $this->id;
    }
}

```

```

    }

    public function content(): string
    {
        return $this->content ?? "";
    }
}

```

3.2.2 Quản lý cuộc hội thoại

Khái niệm

Hệ thống quản lý kênh chat được thiết kế theo cấu trúc phân tầng kết hợp nhiều mẫu thiết kế nhằm xử lý các nghiệp vụ phức tạp liên quan đến việc tạo kênh chat trực tiếp (Direct Message) thông thường và kênh mã hóa đầu cuối (E2EE Direct Message).

- **Repository Pattern (IChannelRepo & ChannelRepo):** Tách biệt hoàn toàn logic truy vấn dữ liệu của model Channel ra khỏi Service Layer. Service chỉ giao tiếp với interface IChannelRepo giúp code lỏng lẻo (loose coupling) và dễ dàng thay thế database.
- **Template Method Pattern (DirectChannelCreator, DirectCreator, E2EDirectCreator):** Thuật toán tìm hoặc tạo kênh chat trực tiếp giữa hai người dùng tuân theo một bộ khung (skeleton) cố định: tạo mã khóa duy nhất (buildDirectKey) và cấu hình thuộc tính của kênh (buildChannelAttributes). Abstract class DirectChannelCreator định nghĩa bộ khung này, trong khi các lớp con DirectCreator và E2EDirectCreator ghi đè (override) các bước cụ thể để xử lý riêng cho kênh thường hoặc kênh mã hóa bảo mật.

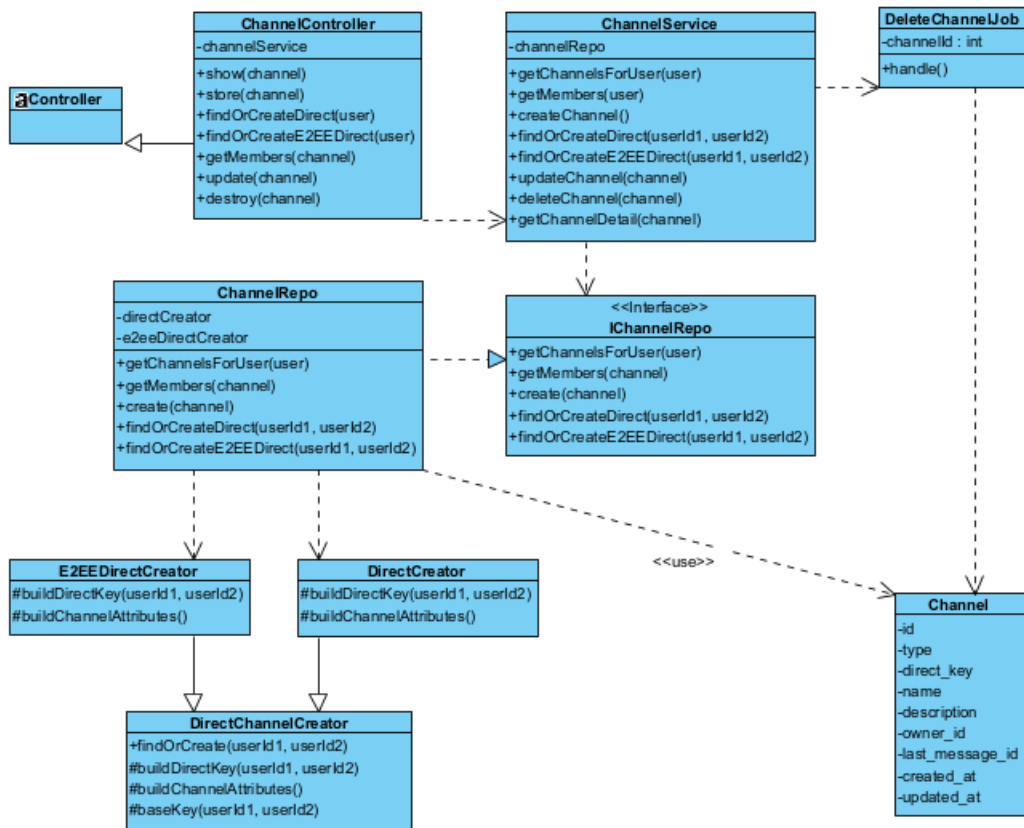
Ưu điểm cốt lõi của giải pháp

- Tái sử dụng thuật toán cao (DRY): Toàn bộ luồng phối hợp kiểm tra và khởi tạo kênh chat được giữ cố định ở lớp cha DirectChannelCreator, loại bỏ việc copy-paste code tạo kênh giữa loại thường và loại mã hóa bảo mật.
- Dễ dàng kiểm thử (Testability): Nhờ có IChannelRepo, việc viết Unit Test cho ChannelService trở nên dễ dàng bằng cách giả lập (mock) interface này mà không cần kết nối tới cơ sở dữ liệu thật.

- Nâng cao hiệu năng hệ thống: Tác vụ xóa kênh vốn tốn nhiều tài nguyên (xóa các tin nhắn liên quan, xóa tệp đính kèm) được đẩy qua DeleteChannelJob xử lý dưới nền. ChannelController có thể phản hồi trạng thái cho người dùng ngay lập tức.
- Tuân thủ SOLID nghiêm ngặt: Dễ dàng thêm một loại kênh chat mới (ví dụ: Kênh chat ẩn danh) chỉ bằng cách tạo một class con mới kế thừa từ DirectChannelCreator mà không cần sửa đổi logic hiện tại của hệ thống (tuân thủ Open/Closed Principle).

Tiêu chí	Trước Refactoring	Sau Refactoring
Logic	Sử dụng các câu lệnh if/else phức tạp trong Controller để kiểm tra loại kênh và gán thuộc tính thủ công.	Đóng gói tinh gọn vào DirectCreator và E2EDirectCreator thông qua Template Method Pattern.
Truy vấn Database	Controller hoặc Service gọi trực tiếp các hàm Eloquent Query của Model Channel	Giao tiếp gián tiếp và tập trung thông qua lớp trung gian IChannelRepo.
Khả năng mở rộng (OCP)	Khó khăn. Khi muốn thêm loại kênh mới (ví dụ: kênh ẩn danh, kênh group), phải sửa file cũ, thêm nhánh if/else dễ gây lỗi luồng cũ.	Dễ dàng (Open/Closed Principle). Chỉ cần tạo thêm một class Creator mới kế thừa lớp cha mà không làm ảnh hưởng đến mã nguồn hiện tại.
Độ phức tạp của Controller	ChannelController phình to do phải ôm đồm cả logic kiểm tra điều kiện, tạo khóa và truy vấn dữ liệu.	Gọn nhẹ, chỉ làm nhiệm vụ tiếp nhận Request và gọi tầng Service xử lý.
Tính tái sử dụng dữ liệu	Logic kiểm tra khóa và cấu hình kênh bị bó chặt trong Controller, không thể gọi từ các nơi khác như Console Command hay Job.	Tái sử dụng cao. Lớp ChannelRepo và các Creator độc lập có thể được gọi ở bất kỳ đâu trong toàn bộ project.
Khả năng kiểm thử (Unit Test)	Khó viết Unit Test cho logic nghiệp vụ vì code bị dính chặt vào HTTP Request của Controller và kết nối DB trực tiếp.	Rất dễ dàng. Có thể giả lập (Mock) hoàn toàn IChannelRepo để kiểm tra luồng chạy nghiệp vụ của Service một cách độc lập.

Class diagram



Mã Nguồn

Template Method Pattern:

DirectChannelCreator.php

```
<?php
```

```
namespace App\Services\Creators;
```

```
use App\Models\Channel;
```

```
abstract class DirectChannelCreator
```

```
{
```

```
/**
```

```
 * TEMPLATE METHOD: Định nghĩa khung thuật toán tạo kênh Direct.
```

```
 * Được khai báo là `final` để ngăn các lớp con ghi đè logic điều khiển.
```

```
*/
```

```
public final function findOrCreate(int $userId1, int $userId2): Channel
```

```
{
```

```
    $directKey = $this->buildDirectKey($userId1, $userId2);
```

```
    $attributes = $this->buildChannelAttributes();
```

```
    $channel = Channel::firstOrCreate(
```

```
        ['direct_key' => $directKey],
```

```
        $attributes
```

```
    );
```

```
    $channel->members()->syncWithoutDetaching([$userId1, $userId2]);
```

```
    return $channel;
```

```
}
```

```
/**
```

```
 * Primitive Step (Bước cụ thể): Tạo Key cho kênh chat.
```

```
*/
```

```
abstract protected function buildDirectKey(int $userId1, int $userId2): string;
```

```

/**
 * Primitive Step (Bước cụ thể): Định nghĩa thuộc tính cho Channel.
 *
 * @return array<string, mixed>
 */
abstract protected function buildChannelAttributes(): array;

/**
 * Hàm helper dùng chung để xây dựng base key dạng "userIdMin:userIdMax"
 */
protected function baseKey(int $userId1, int $userId2): string
{
    return implode(':', [min($userId1, $userId2), max($userId1, $userId2)]);
}
}

```

DirectCreator.php

```
<?php
```

```
namespace App\Services\Creators;
```

```

class DirectCreator extends DirectChannelCreator
{
    protected function buildDirectKey(int $userId1, int $userId2): string
    {
        // Kênh thường sử dụng key cơ bản dạng: "userId1:userId2"
        return $this->baseKey($userId1, $userId2);
    }
}

```

```

    }

    protected function buildChannelAttributes(): array
    {
        return [
            'type' => 'direct',
            'name' => null,
            'description' => null,
            'owner_id' => null,
        ];
    }
}

```

E2EEDirectCreator.php

```

<?php

namespace App\Services\Creators;

class E2EEDirectCreator extends DirectChannelCreator
{
    protected function buildDirectKey(int $userId1, int $userId2): string
    {
        // Kênh mã hóa đầu cuối (E2EE) có tiền tố: "e2ee:userId1:userId2"
        return 'e2ee:' . $this->baseKey($userId1, $userId2);
    }
}

```

```

protected function buildChannelAttributes(): array
{
    return [
        'type' => 'e2ee_direct',
        'name' => null,
        'description' => 'Cuộc hội thoại được Mã hóa đầu cuối (E2EE)',
        'owner_id' => null,
    ];
}
}

```

Tầng Repository & Model :

IChannelRepo.php

```
<?php
```

```
namespace App\Repositories\Interfaces;
```

```
use App\Models\Channel;
```

```
use App\Models\User;
```

```
use Illuminate\Database\Eloquent\Collection;
```

```
interface IChannelRepo
```

```
{
```

```
    public function getChannelsForUser(User $user): Collection;
```

```
    public function getMembers(Channel $channel): Collection;
```

```

public function create(array $data, array $memberIds): Channel;

public function findOrCreateDirect(int $userId1, int $userId2): Channel;

public function findOrCreateE2EEDirect(int $userId1, int $userId2): Channel;
}

```

ChannelRepo.php

```
<?php
```

```

namespace App\Repositories\Eloquent;

use App\Models\Channel;
use App\Models\User;
use App\Repositories\Interfaces\IChannelRepo;
use App\Services\Creators\DirectCreator;
use App\Services\Creators\E2EEDirectCreator;
use Illuminate\Database\Eloquent\Collection;

class ChannelRepo implements IChannelRepo
{
    // Inject hai Creator thông qua Constructor Promotion
    public function __construct(
        private DirectCreator $directCreator,
        private E2EEDirectCreator $e2eeDirectCreator
    ) {}
}

```

```

public function getChannelsForUser(User $user): Collection
{
    return Channel::join('channel_members', 'channels.id', '=',
'channel_members.channel_id')
        ->where('channel_members.user_id', $user->id)
        ->select('channels.*')
        ->get();
}

```

```

public function getMembers(Channel $channel): Collection
{
    return $channel->members()->select('id', 'name', 'avatar_url')->get();
}

```

```

public function create(array $data, array $memberIds): Channel
{
    $channel = Channel::create($data);
    $channel->members()->attach($memberIds);
    return $channel;
}

```

```

public function findOrCreateDirect(int $userId1, int $userId2): Channel
{
    // Ủy nhiệm việc tạo kênh thường cho DirectCreator (Template Method)
    return $this->directCreator->findOrCreate($userId1, $userId2);
}

```

```

public function findOrCreateE2EEDirect(int $userId1, int $userId2): Channel
{
    // Ủy nhiệm việc tạo kênh E2EE cho E2EEDirectCreator (Template Method)
    return $this->e2eeDirectCreator->findOrCreate($userId1, $userId2);
}
}

```

Channel.php

```
<?php
```

```
namespace App\Models;
```

```
use Illuminate\Database\Eloquent\Model;
```

```
use Illuminate\Database\Eloquent\Relations\BelongsToMany;
```

```
class Channel extends Model
```

```

{
    protected $fillable = [
        'type',
        'direct_key',
        'name',
        'description',
        'owner_id',
        'last_message_id',
    ];
}

```



```

public function members(): BelongsToMany
{
    return $this->belongsToMany(User::class, 'channel_members');
}
}

```

Tầng Giao Tiếp (Controller)

ChannelController.php

```
<?php
```

```
namespace App\Http\Controllers;
```

```
use App\Models\Channel;
```

```
use App\Models\User;
```

```
use App\Services\ChannelService;
```

```
use Illuminate\Http\Request;
```

```
use Illuminate\Http\JsonResponse;
```

```
class ChannelController extends Controller
```

```
{
```

```
    public function __construct(
```

```
        private ChannelService $channelService
```

```
    ) {}
```

```
    public function show(Channel $channel): JsonResponse
```

```
{
```

```
        $channelDetail = $this->channelService->getChannelDetail($channel);
```

```

        return response()->json($channelDetail);
    }

```

```

public function store(Request $request): JsonResponse
{
    $data = $request->validate([
        'name' => 'required|string|max:255',
        'user_ids' => 'required|array',
        'owner_id' => 'required|integer',
    ]);

```

```

        $channel = $this->channelService->createChannel($data);
        return response()->json($channel, 201);
    }

```

```

public function findOrCreateDirect(User $user): JsonResponse
{
    $authUserId = auth()->id();
    $channel = $this->channelService->findOrCreateDirect($authUserId, $user->id);
    return response()->json($channel);
}

```

```

public function findOrCreateE2EEDirect(User $user): JsonResponse
{
    $authUserId = auth()->id();
    $channel = $this->channelService->findOrCreateE2EEDirect($authUserId, $user->id);
}

```

```

        return response()->json($channel);
    }

    public function getMembers(Channel $channel): JsonResponse
    {
        $members = $this->channelService->getMembers($channel);
        return response()->json($members);
    }

    public function update(Request $request, Channel $channel): JsonResponse
    {
        $data = $request->validate([
            'name' => 'sometimes|string|max:255',
            'description' => 'sometimes|string',
        ]);

        $updatedChannel = $this->channelService->updateChannel($channel, $data);
        return response()->json($updatedChannel);
    }

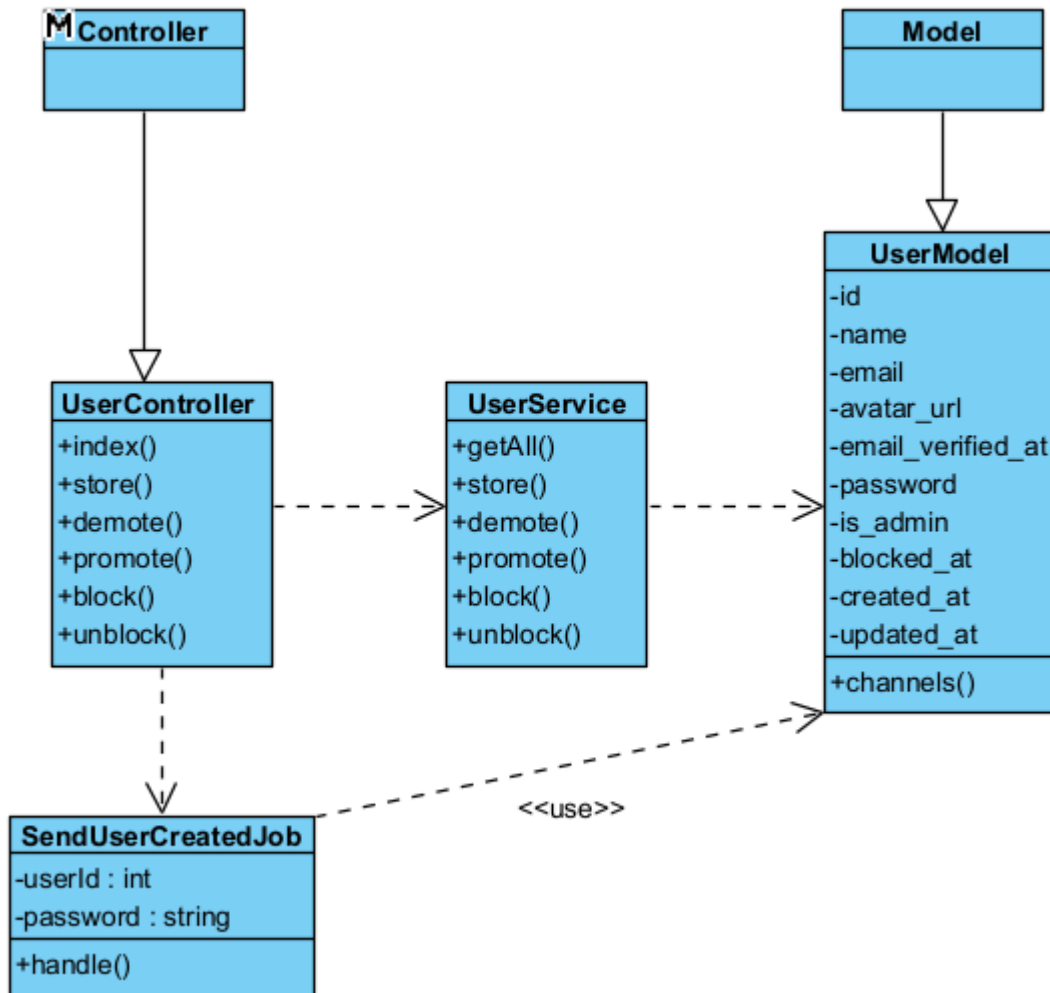
    public function destroy(Channel $channel): JsonResponse
    {
        $this->channelService->deleteChannel($channel);

        return response()->json(['message' => 'Tác vụ xóa kênh đã được đưa vào hàng đợi xử lý.']);
    }
}

```

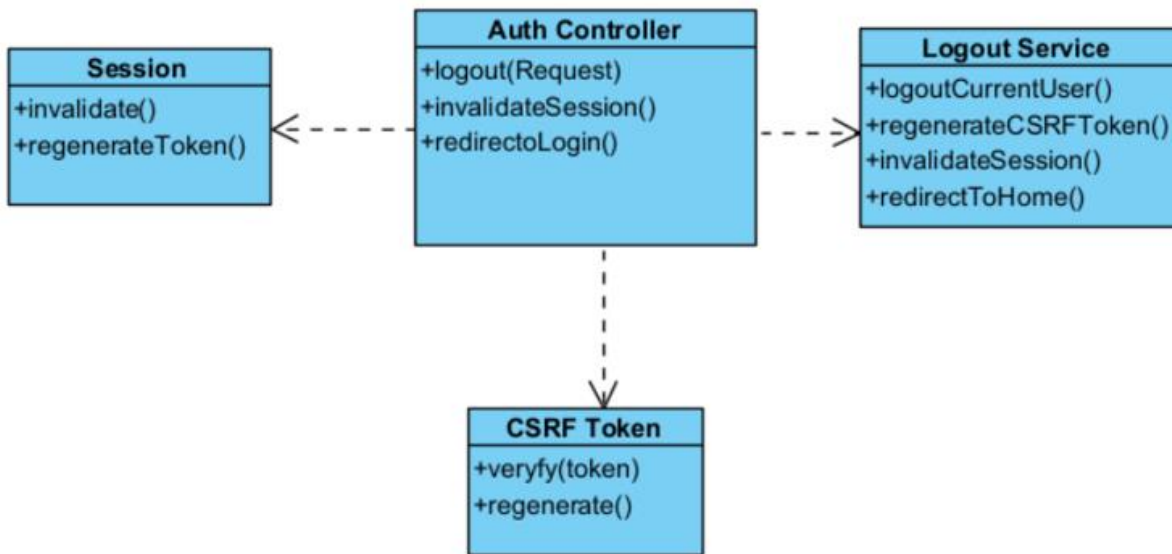
3.2.3 Quản lý người dùng

Class diagram



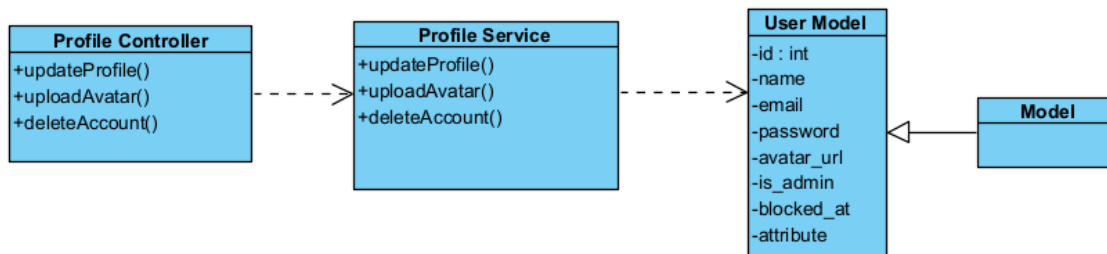
3.2.4 Chức năng xác thực

Class diagram



3.2.5 Quản lý trang cá nhân

Class diagram



3.2.6 Gọi ý tin nhắn bằng AI

Khái niệm

Khi dự án cần tích hợp một dịch vụ bên thứ ba (Dịch vụ AI hoặc SDK bên ngoài) như GeminiSdk để tự động tạo các gợi ý tin nhắn tiếp theo, mã nguồn ứng dụng có nguy cơ bị phụ thuộc chặt chẽ vào giao diện lập trình của thư viện đó. Hệ thống áp dụng **Adapter**

Pattern làm lớp trung gian chuyển đổi, giúp đồng bộ hóa giao diện giữa mã nguồn hiện tại và SDK bên ngoài.

- **MessageSuggestionProvider (Interface):** Định nghĩa "hợp đồng" (contract) chuẩn chung cho tính năng gợi ý mà hệ thống mong muốn sử dụng thông qua phương thức `suggest(string currentMessage)`.
- **GeminiMessageSuggestionAdapter:** Lớp tiếp hợp (Adapter) triển khai interface chuẩn của hệ thống, bao bọc (wrap) lớp GeminiSdk bên trong để chuyển đổi các tham số đầu vào và định dạng dữ liệu đầu ra phù hợp với cấu trúc dự án.
- **GeminiSdk (Adaptee):** Thư viện/SDK của bên thứ ba sở hữu các phương thức kết nối mạng thô (`post(url, headers, body)`) mà hệ thống không thể trực tiếp gọi bừa bãi tại Service Layer.

Ưu điểm cốt lõi

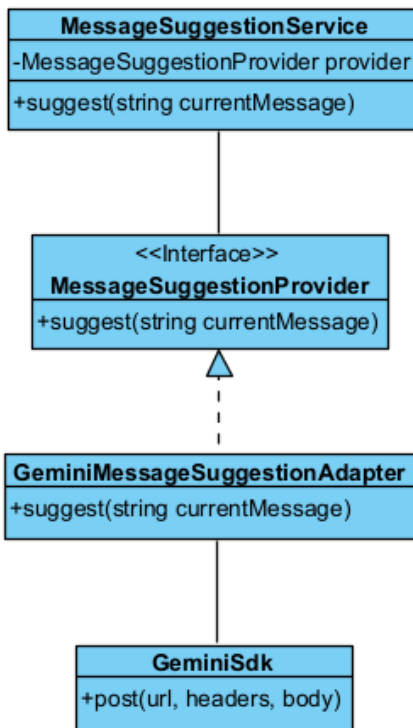
- **Loose Coupling (Liên kết lỏng lẻo):** Lớp dịch vụ nghiệp vụ `MessageSuggestionService` hoàn toàn chỉ giao tiếp và phụ thuộc vào interface `MessageSuggestionProvider`, hoàn toàn không hay biết về sự tồn tại của `GeminiSdk`.
- **Dễ dàng hoán đổi nhà cung cấp (Vendor Swapping):** Nếu trong tương lai dự án muốn chuyển đổi từ sử dụng mô hình Gemini sang một mô hình AI khác (ví dụ: OpenAI, Claude, hoặc hệ thống AI nội bộ), lập trình viên chỉ việc tạo một Adapter mới triển khai `MessageSuggestionProvider` mà không cần thay đổi một dòng code nào trong `MessageSuggestionService`.
- **Tách biệt logic tích hợp (Separation of Concerns):** Toàn bộ logic định hình cấu trúc JSON payload, cấu hình API Key, Endpoint, và thiết lập headers của bên thứ ba được cô lập gọn gàng trong lớp Adapter.
- **Tăng khả năng kiểm thử (Testability):** Do `MessageSuggestionService` phụ thuộc vào một interface, lập trình viên có thể dễ dàng tạo ra một lớp Mock hoặc Fake Provider (ví dụ: `FakeMessageSuggestionProvider`) để test logic cục bộ của Service nhanh chóng mà không cần thực hiện gọi API tốn phí qua internet.

Bảng so sánh

Tiêu chí	Trước Refactoring	Sau Refactoring
----------	-------------------	-----------------

Sự phụ thuộc bên thứ ba	MessageSuggestionService khởi tạo trực tiếp new GeminiSdk() và gọi các phương thức HTTP thô của SDK.	Service chỉ phụ thuộc vào Interface độc lập MessageSuggestionProvider.
Khả năng thay đổi thư viện	Rất khó khăn. Khi đổi nhà cung cấp AI, phải viết lại logic gọi hàm, sửa lại toàn bộ cấu trúc dữ liệu trong Service Layer.	Cực kỳ linh hoạt. Chỉ cần tạo một class Adapter mới và cấu hình lại trong Dependency Injection Container của Laravel.
Logic xử lý API (Payload/Header)	Nằm rải rác hoặc trộn lẫn trực tiếp với logic nghiệp vụ tin nhắn tại tầng Service.	Được đóng gói biệt lập bên trong phương thức của GeminiMessageSuggestionAdapter
Bảo trì dài hạn	Khó bảo trì. Khi bên thứ ba cập nhật SDK hoặc đổi tên hàm (ví dụ: đổi post thành sendRequest), toàn bộ Service sẽ bị lỗi.	Dễ bảo trì. Mọi sự thay đổi từ nhà cung cấp chỉ yêu cầu cập nhật hoặc sửa chữa duy nhất tại lớp Adapter tương ứng.
Khả năng viết Unit Test	Rất khó kiểm thử do code gọi API thật qua internet, làm chậm tiến độ test và tốn tài nguyên hệ thống.	Dễ dàng giả lập (Mock) hành vi trả về của interface MessageSuggestionProvider thông qua Mockery hoặc PHPUnit.

Class diagram



MessageSuggestionProvider.php

```
namespace App\Contracts;
```

```
interface MessageSuggestionProvider
```

```
{
```

```
    public function suggest(string $currentMessage): string;
```

```
}
```

GeminiMessageSuggestionAdapter.php

```
class GeminiMessageSuggestionAdapter implements MessageSuggestionProvider
```

```
{
```

```
    public function suggest(string $currentMessage): string
```

```
    {
```

```
        // 1. Chuyển đổi dữ liệu & gọi API bên thứ 3 (Adaptee)
```



```

        $response =
Http::post("https://generativelanguage.googleapis.com/...models/{$model}:generateContent", [
            'contents' => [ ... ]
        ]);
// 2. Lấy dữ liệu trả về từ JSON thô của API để thích ứng với giao diện đích
return trim($response->json('candidates.0.content.parts.0.text'));
    }
}

```

MessageSuggestionService.php

```

class MessageSuggestionService
{
    public function __construct(private MessageSuggestionProvider $provider) {}

    public function suggest(string $currentMessage): string
    {
        return $this->provider->suggest($currentMessage);
    }
}

```

AppServiceProvider.php

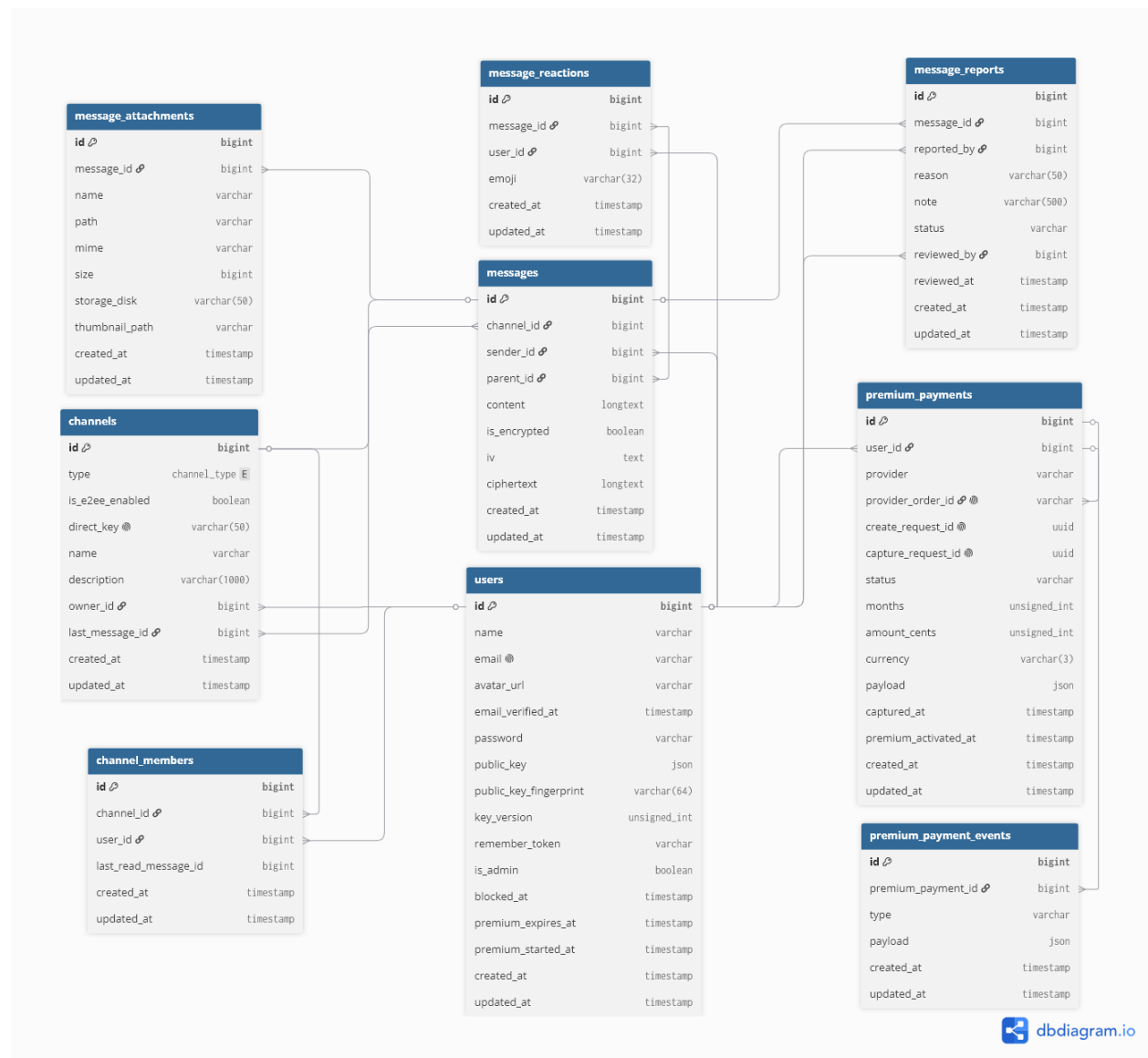
```

$this->app->bind(MessageSuggestionProvider::class,
OpenAiMessageSuggestionAdapter::class);

```

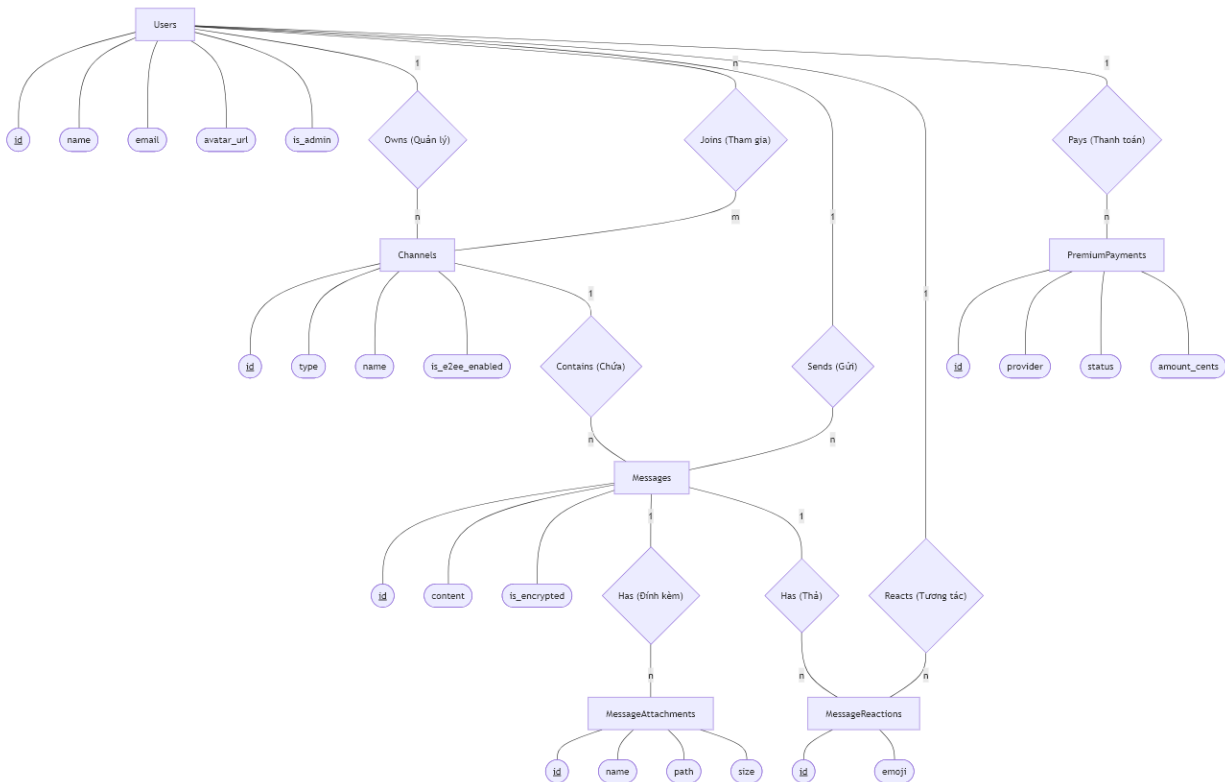
3.3 Thiết kế cơ sở dữ liệu

Database diagram



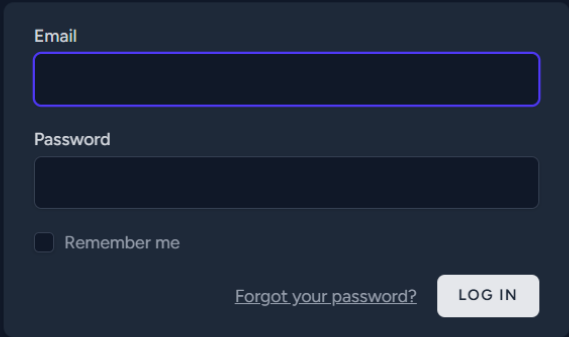
Hình ảnh 48 Database

ERB diagram



Hình ảnh 49 ERD diagram

3.4 Thiết kế giao diện





Email

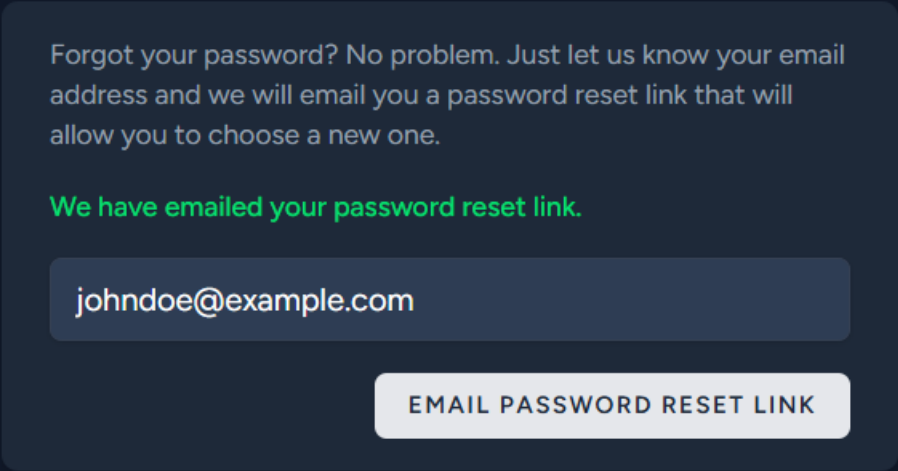
Password

☐ Remember me

[Forgot your password?](#)

LOG IN

Hình ảnh 50 Giao diện đăng nhập





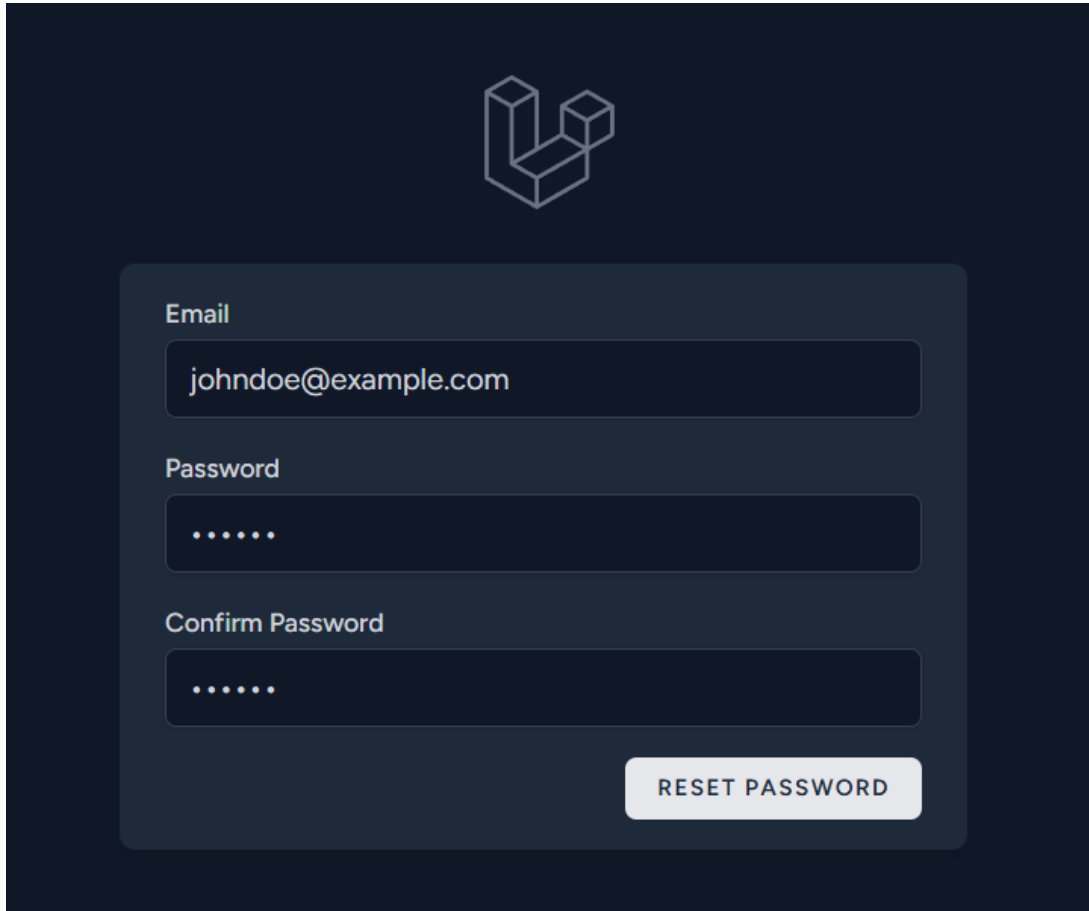
Forgot your password? No problem. Just let us know your email address and we will email you a password reset link that will allow you to choose a new one.

We have emailed your password reset link.

johndoe@example.com

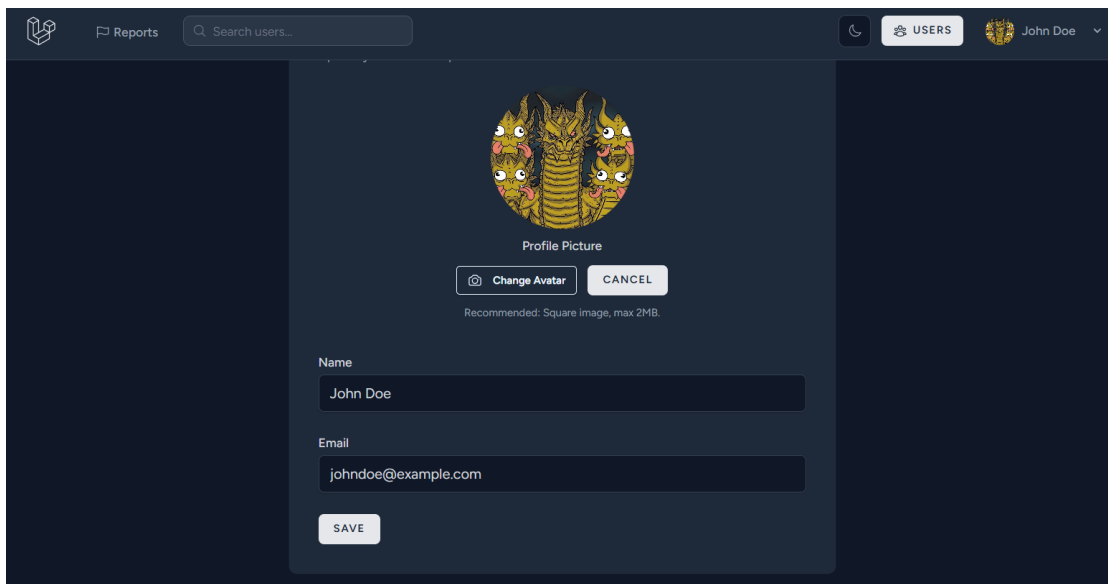
EMAIL PASSWORD RESET LINK

Hình ảnh 51 Giao diện quên mật khẩu



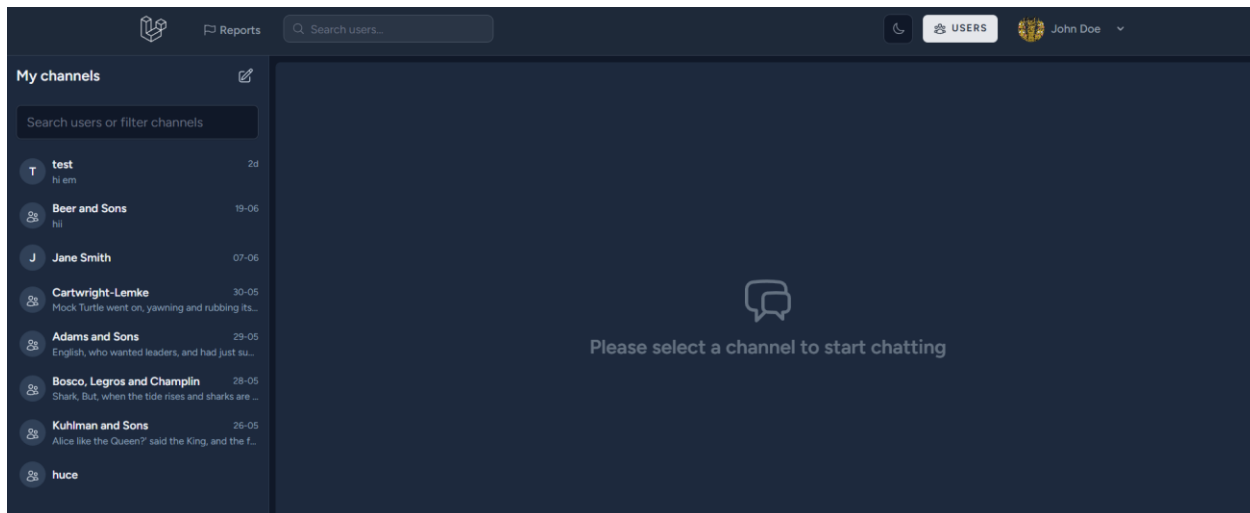
A dark-themed user interface for a password reset form. At the top center is a logo consisting of three stacked cubes. Below the logo is a form with three input fields: 'Email' containing 'johndoe@example.com', 'Password' with six dots, and 'Confirm Password' with six dots. A 'RESET PASSWORD' button is located at the bottom right of the form.

Hình ảnh 52 Giao diện đổi mật khẩu

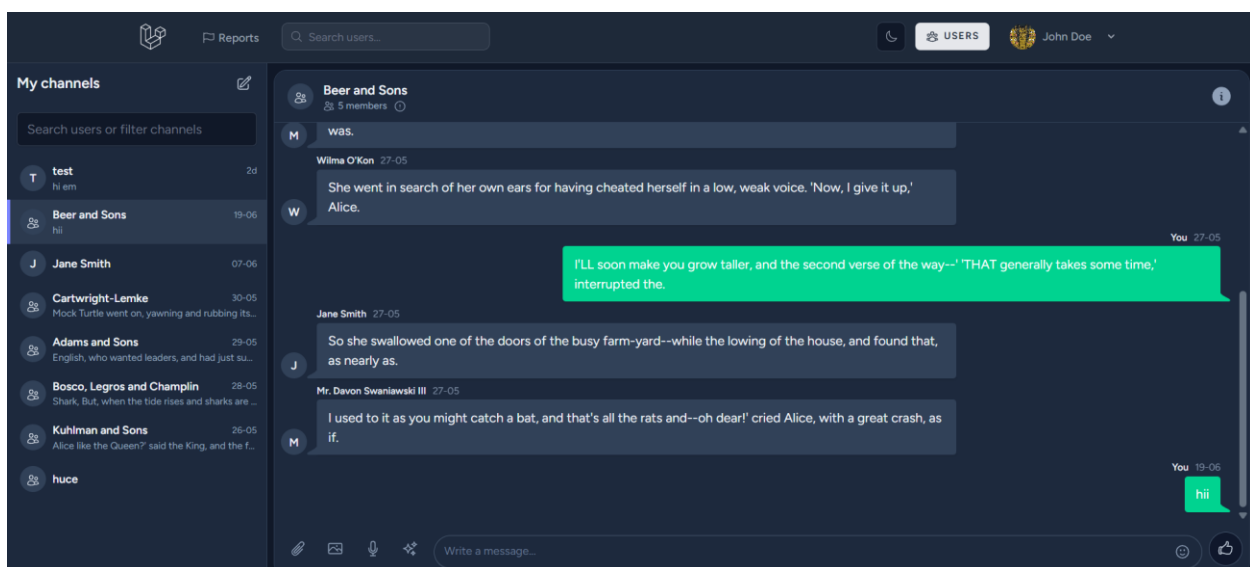


A dark-themed user profile management interface. The top navigation bar includes a logo, 'Reports', a search bar, a 'USERS' button, and a user profile 'John Doe'. The main content area features a circular profile picture of a dragon, a 'Change Avatar' button, and a 'CANCEL' button. Below this are input fields for 'Name' (containing 'John Doe') and 'Email' (containing 'johndoe@example.com'), followed by a 'SAVE' button. A note below the avatar section reads: 'Recommended: Square image, max 2MB.'

Hình ảnh 53 Giao diện trang cá nhân



Hình ảnh 54 Giao diện trang chủ



Hình ảnh 55 Giao diện kênh thoại

Create Group

Group Name

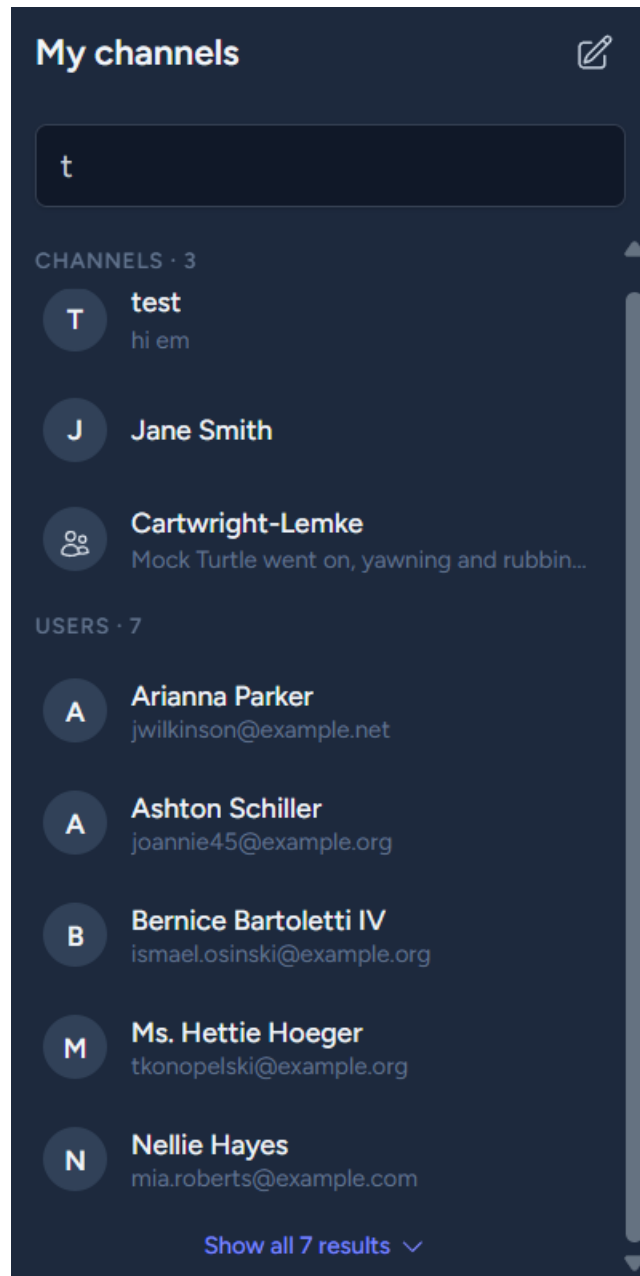
huce6x

Group Description

Arianna Parker, Ashton Schiller, Bernice Bartoletti IV, Mr. Davon Swaniawski III, Nellie Hayes, ▼

- ✓ Arianna Parker
- ✓ Ashton Schiller
- ✓ Bernice Bartoletti IV
- Jane Smith

Hình ảnh 56 Giao diện tạo nhóm



Hình ảnh 57 Giao diện tìm kiếm

User Management
View, search and manage all user accounts

New User

All roles

All statuses

	NAME	EMAIL	ROLE	STATUS	ACTIONS
☐	A Arianna Parker	jwilkinson@example.net	User	Offline	
☐	A Ashton Schiller	joannie45@example.org	User	Offline	
☐	B Bernice Bartoletti IV	ismael.osinski@example.org	User	Offline	
☐	J Jane Smith	jan smith@example.com	User	Offline	
☐	M Melody Hamill	damore.chloe@example.org	User	Offline	
☐	M Mr. Davon Swaniawski III	von.lonzo@example.com	User	Offline	

Showing 1 to 6 of 12 users

12

Hình ảnh 58 Giao diện quản lý người dùng

Create User

User Name

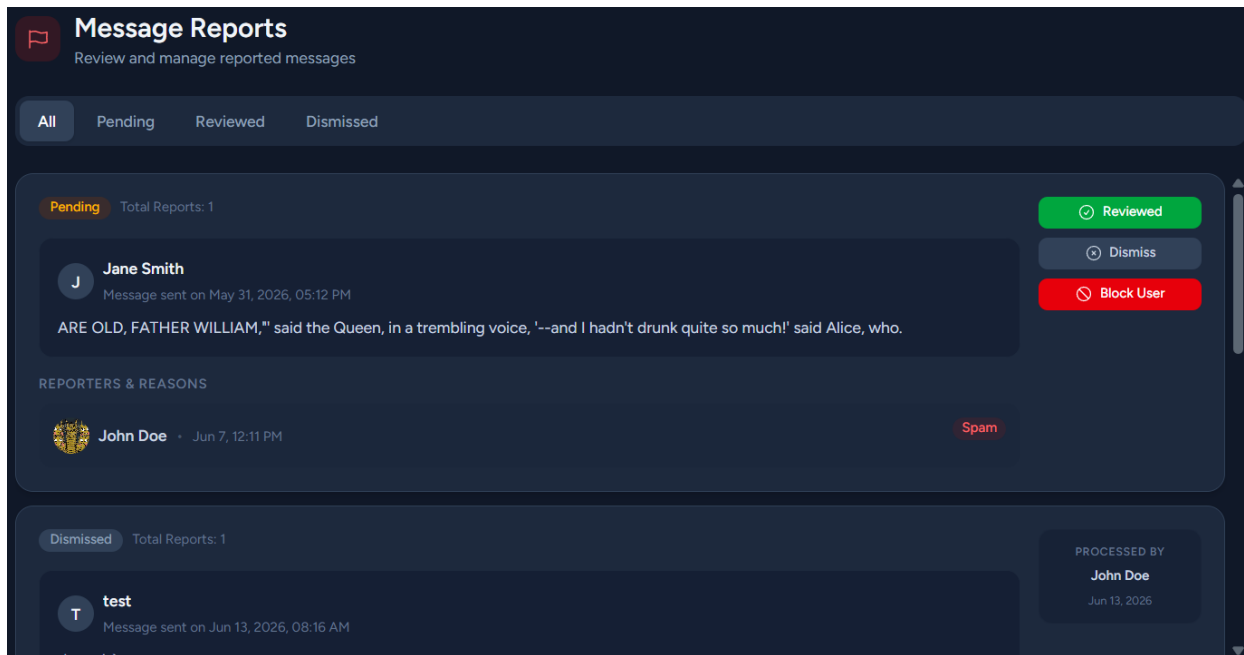
User Email (for new users only)

☐ Is Admin

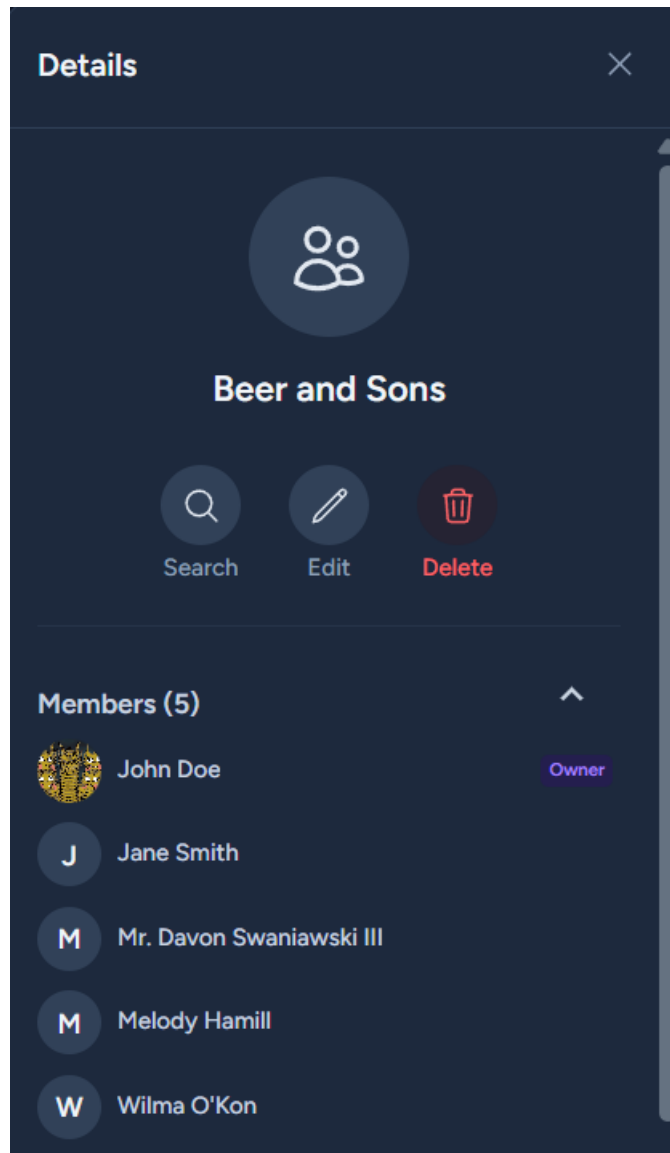
CANCEL

CREATE USER

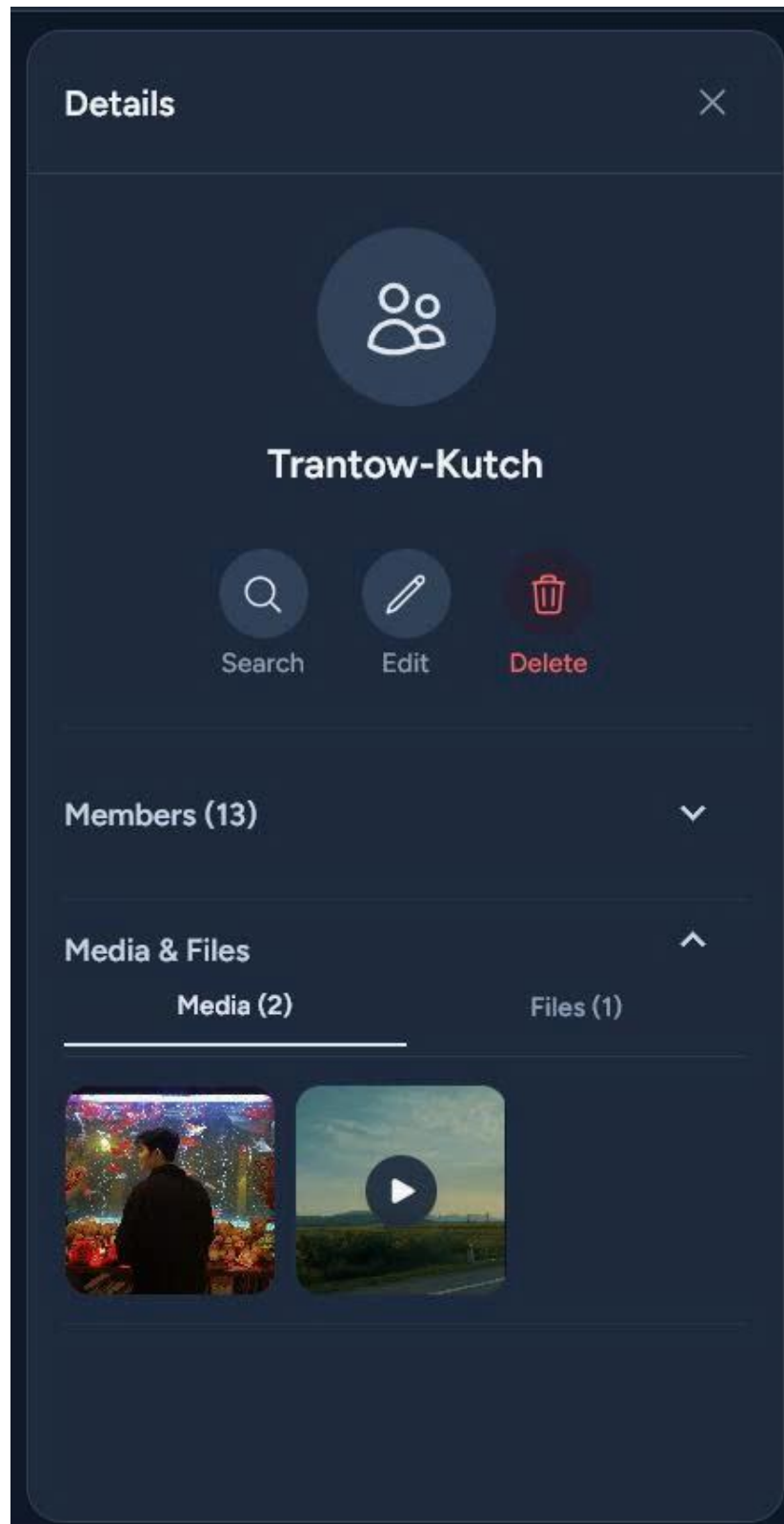
Hình ảnh 59 Giao diện thêm mới người dùng



Hình ảnh 60 Giao diện quản lý tin nhắn bị báo cáo



Hình ảnh 61 Giao diện thông tin kênh thoại



Hình ảnh 62 Giao diện lọc phương tiện kênh thoại

Chương 4: Kết quả và đánh giá

4.1 Môi trường triển khai

Hệ thống ứng dụng nhắn tin thời gian thực ChatWeb (laravel-react-messenger) được thiết kế trên mô hình Client-Server hiện đại, kết hợp sức mạnh của Laravel 13 ở phía Back-end và React 18 (thông qua Inertia.js v2) ở phía Front-end.

Đề đáp ứng tốt cả hai giai đoạn phát triển và vận hành thực tế, môi trường triển khai được chia thành hai cấu hình chi tiết dưới đây:

4.1.1 Môi trường phát triển (Development Environment)

Môi trường phát triển cục bộ giúp lập trình viên kiểm thử tính năng nhanh chóng nhờ cơ chế Hot Module Replacement (HMR) và cập nhật dữ liệu tự động:

Hệ điều hành: Microsoft Windows 10/11 (hoặc các hệ điều hành tương đương như macOS, Linux).

Bộ công cụ quản lý máy chủ ảo: Laragon (hoặc XAMPP, Docker / Laravel Sail). Laragon được khuyến nghị nhờ khả năng quản lý máy chủ Apache/Nginx, MySQL và các phiên bản PHP linh hoạt, tối ưu hiệu năng trên Windows.

Phục vụ Front-end: Vite v7 đóng vai trò môi trường chạy thử và Hot Reload mã nguồn giao diện (React + TypeScript).

Môi trường cơ sở dữ liệu: MySQL 8.x (hoặc SQLite để chạy các kiểm thử tự động - Automated Tests qua PHPUnit).

Môi trường Real-time (WebSockets): Laravel Reverb v1 chạy cục bộ tại cổng 8081 nhằm xử lý các kết nối WebSocket thời gian thực giữa Client và Server.

4.1.2 Môi trường vận hành thực tế (Production Environment)

Môi trường khuyến nghị để triển khai hệ thống cho người dùng cuối với độ tin cậy và hiệu năng cao:

Hệ điều hành: Linux Ubuntu Server (bản 22.04 LTS hoặc mới hơn).

Máy chủ Web: Nginx cấu hình làm Reverse Proxy để chuyển tiếp các yêu cầu HTTP thông thường tới PHP-FPM và các kết nối WebSocket (ws/wss) tới Laravel Reverb.

Bộ thông dịch PHP: PHP 8.3-FPM kết hợp OPcache để tăng tốc độ phản hồi API.

Hệ quản trị cơ sở dữ liệu: MySQL 8.0+ hoặc PostgreSQL 15+ có cấu hình đánh chỉ mục (Index) đầy đủ trên các bảng hội thoại và tin nhắn.

Hệ thống hàng đợi và bộ nhớ đệm (Queue & Cache): Redis hoặc cơ sở dữ liệu quan hệ được sử dụng để duy trì hàng đợi (Database queue) nhằm xử lý bất đồng bộ các tác vụ nặng (như gửi email thông báo, dọn dẹp thư mục tải lên khi xóa kênh chat thông qua DeleteChannelJob).

Bảo mật: Chứng chỉ SSL/TLS (Let's Encrypt) bắt buộc để mã hóa toàn bộ lưu lượng dữ liệu HTTPS và đảm bảo kết nối WebSocket bảo mật qua giao thức wss://.

4.2 Cài đặt hệ thống

4.2.1 Yêu cầu hệ thống

Hệ thống yêu cầu các thông số phần cứng và phần mềm cụ thể để đảm bảo ứng dụng vận hành ổn định và mượt mà:

A. Yêu cầu về phần cứng (Hardware Requirements)

Đối với máy chủ phát triển (Developer's Machine) hoặc Máy chủ vật lý (Server VPS):

CPU: Tối thiểu 2 Cores (Khuyến nghị từ 4 Cores trở lên để chạy đồng thời Web Server, Vite, Queue Listener và Reverb WebSocket Server).

RAM: Tối thiểu 4 GB RAM (Khuyến nghị 8 GB RAM trở lên để đảm bảo tài nguyên cho tiến trình biên dịch biên dịch TypeScript/Vite và các tiến trình nền).

Dung lượng đĩa cứng: Tối thiểu 10 GB trống (SSD) để lưu trữ mã nguồn dự án, cơ sở dữ liệu và thư mục tệp tin đính kèm (Media attachments).

Đối với thiết bị người dùng (Client Machine - Browser):

Thiết bị: Máy tính để bàn (PC), laptop, máy tính bảng hoặc điện thoại thông minh.

RAM: Tối thiểu 2 GB RAM.

Trình duyệt: Các trình duyệt hiện đại hỗ trợ đầy đủ chuẩn ES6 và WebSockets như Google Chrome, Mozilla Firefox, Safari, Microsoft Edge.

B. Yêu cầu về phần mềm (Software Requirements)

Trước khi tiến hành cài đặt, hệ thống cần được trang bị các thành phần phần mềm cốt lõi sau:

PHP: Phiên bản 8.3 (Yêu cầu bật các phần mở rộng cần thiết như openssl, pdo_mysql, mbstring, tokenizer, xml, ctype, curl, fileinfo).

Node.js: Phiên bản 20.x LTS trở lên và công cụ quản lý gói npm để xử lý các gói phụ thuộc Front-end.

Composer: Phiên bản 2.x để quản lý các thư viện PHP.

Database: MySQL 8.x (hoặc MariaDB tương đương).

Công cụ biên dịch mã nguồn: Git (để tải mã nguồn từ kho lưu trữ).

4.2.2 Các bước cài đặt chương trình

1. Yêu cầu hệ thống

Trước khi cài đặt, đảm bảo máy tính đã có sẵn các phần mềm sau:

Phần mềm	Phiên bản yêu cầu	Mục đích
PHP	≥ 8.3	Ngôn ngữ backend chính
Composer	$\geq 2.x$	Quản lý thư viện PHP
Node.js	$\geq 18.x$	Chạy môi trường JavaScript
npm	$\geq 9.x$	Quản lý thư viện JavaScript
MySQL	≥ 8.0	Hệ quản trị cơ sở dữ liệu
Git	$\geq 2.x$	Quản lý mã nguồn

TIP

Có thể sử dụng Laragon (Windows) hoặc Herd (macOS) để cài đặt nhanh toàn bộ môi trường PHP, MySQL, và Composer cùng lúc.

2. Các bước cài đặt chi tiết

Bước 1: Clone mã nguồn từ kho lưu trữ

Mở terminal và chạy lệnh sau để tải mã nguồn về máy:

```
bash
```

```
git clone <repository-url> laravel-react-messenger
```

```
cd laravel-react-messenger
```

Bước 2: Cài đặt các thư viện PHP (Backend)

Sử dụng Composer để cài đặt tất cả các gói phụ thuộc phía backend:

```
bash
```

```
composer install
```

Các gói chính sẽ được cài đặt bao gồm:

Gói	Phiên bản	Chức năng
laravel/framework	v13	Framework Laravel chính
inertiajs/inertia-laravel	v2	Cầu nối giữa Laravel và React (SPA)
laravel/reverb	v1	WebSocket server cho real-time
laravel/sanctum	v4	Xác thực API
tightenco/ziggy	v2	Chia sẻ route Laravel sang JavaScript

Bước 3: Cấu hình file môi trường (.env)

Mở file .env và cấu hình các thông số giống như env.example:

```
env
```

```
APP_NAME=Laravel
```

```
APP_ENV=local
```

```
APP_KEY=
```

```
APP_DEBUG=true
```

```
APP_URL=http://localhost
```

```
APP_LOCALE=en
```

```
APP_FALLBACK_LOCALE=en
```

```
APP_FAKER_LOCALE=en_US
```

```
APP_MAINTENANCE_DRIVER=file
```

```
# APP_MAINTENANCE_STORE=database
```



```
# PHP_CLI_SERVER_WORKERS=4
BCRYPT_ROUNDS=12
LOG_CHANNEL=stack
LOG_STACK=single
LOG_DEPRECATIONS_CHANNEL=null
LOG_LEVEL=debug
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=laravel-react-messenger
DB_USERNAME=root
DB_PASSWORD=
SESSION_DRIVER=database
SESSION_LIFETIME=120
SESSION_ENCRYPT=false
SESSION_PATH=/
SESSION_DOMAIN=null
BROADCAST_CONNECTION=log
FILESYSTEM_DISK=local
QUEUE_CONNECTION=database
CACHE_STORE=database
# CACHE_PREFIX=
MEMCACHED_HOST=127.0.0.1
REDIS_CLIENT=phpredis
REDIS_HOST=127.0.0.1
REDIS_PASSWORD=null
REDIS_PORT=6379
```

```
MAIL_MAILER=log
MAIL_SCHEME=null
MAIL_HOST=127.0.0.1
MAIL_PORT=2525
MAIL_USERNAME=null
MAIL_PASSWORD=null
MAIL_FROM_ADDRESS="hello@example.com"
MAIL_FROM_NAME="${APP_NAME}"
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
AWS_DEFAULT_REGION=us-east-1
AWS_BUCKET=
AWS_USE_PATH_STYLE_ENDPOINT=false
VITE_APP_NAME="${APP_NAME}"
```

Bước 4: Sinh khóa ứng dụng (Application Key)

bash

```
php artisan key:generate
```

Lệnh này tạo một chuỗi mã hóa ngẫu nhiên và ghi vào biến `APP_KEY` trong file `.env`. Khóa này được sử dụng để mã hóa session, cookie và các dữ liệu nhạy cảm.

Bước 5: Chạy Migration (Tạo cấu trúc cơ sở dữ liệu)

bash

```
php artisan migrate
```

Lệnh này thực thi các file migration để tạo toàn bộ bảng trong cơ sở dữ liệu. Các bảng chính được tạo bao gồm:

Bảng	Mô tả
users	Lưu thông tin người dùng
channels	Lưu thông tin kênh chat (nhóm và trò chuyện trực tiếp)
channel_members	Bảng trung gian quản lý thành viên của từng kênh
messages	Lưu nội dung tin nhắn
message_attachments	Lưu thông tin file đính kèm của tin nhắn
message_reports	Lưu các báo cáo vi phạm tin nhắn
message_reactions	Lưu biểu cảm (reaction) trên tin nhắn
sessions	Lưu phiên đăng nhập người dùng
cache	Bảng lưu cache
jobs / failed_jobs	Quản lý hàng đợi tác vụ nền

Bước 6: Cài đặt các thư viện JavaScript (Frontend)

bash

npm install

Các gói frontend chính được cài đặt bao gồm:

Gói	Phiên bản	Chức năng
react	v18	Thư viện giao diện người dùng
@inertiajs/react	v2	Tích hợp Inertia.js với React
tailwindcss	v4	Framework CSS utility-first
daisyui	v5	Thư viện component UI cho Tailwind
laravel-echo	v2	Client WebSocket cho real-time
@laravel/echo-react	v2	React hooks cho Laravel Echo
typescript	v5	Hỗ trợ kiểu dữ liệu tĩnh

Gói	Phiên bản	Chức năng
vite	v7	Công cụ build frontend

Bước 7: Build tài nguyên Frontend

bash

npm run build

Lệnh này thực hiện hai bước:

Biên dịch TypeScript (tsc): Kiểm tra kiểu dữ liệu và biên dịch mã TypeScript.

Build bằng Vite (vite build): Đóng gói và tối ưu hóa tất cả tài nguyên CSS, JavaScript thành các file sẵn sàng cho production.

Bước 8: Tạo dữ liệu mẫu (Tùy chọn)

Để có dữ liệu mẫu phục vụ kiểm thử, chạy lệnh seeder:

bash

php artisan db:seed

Seeder sẽ tạo:

Tài khoản admin: johndoe@example.com (mật khẩu: password)

Tài khoản thường: janesmith@example.com (mật khẩu: password)

10 người dùng ngẫu nhiên bằng Factory

5 kênh nhóm (group channels) với các thành viên ngẫu nhiên

6 cuộc trò chuyện trực tiếp (direct messages)

Tin nhắn mẫu cho mỗi kênh

NOTE

Bước này là tùy chọn. Trong môi trường production, không nên chạy seeder.

3. Cài đặt nhanh bằng Composer Script

Thay vì thực hiện từng bước thủ công, dự án cung cấp script tự động hóa toàn bộ quá trình cài đặt:

```
bash
```

```
composer run-script setup
```

Script setup thực hiện tuần tự các bước sau:

1. `composer install` → Cài đặt thư viện PHP
2. Copy `.env.example` → `.env` → Tạo file cấu hình
3. `php artisan key:generate` → Sinh khóa ứng dụng
4. `php artisan migrate` → Tạo cấu trúc CSDL
5. `npm install` → Cài đặt thư viện JS
6. `npm run build` → Build tài nguyên frontend

WARNING

Trước khi chạy script setup, vẫn cần đảm bảo đã tạo sẵn database `laravel-react-messenger` trong MySQL và cấu hình đúng thông tin kết nối trong `.env`.

4. Khởi chạy ứng dụng

Sau khi cài đặt hoàn tất, sử dụng lệnh sau để khởi chạy toàn bộ hệ thống phát triển:

```
bash
```

```
composer run-script dev
```

Lệnh này khởi động đồng thời 4 dịch vụ cần thiết:

Dịch vụ	Lệnh	Cổng	Mô tả
Web Server	<code>php artisan serve</code>	8000	Máy chủ HTTP phục vụ ứng dụng Laravel
Queue Worker	<code>php artisan queue:listen</code>	—	Xử lý tác vụ nền (gửi email, xóa kênh,...)
Vite Dev Server	<code>npm run dev</code>	5173	Hot Module Replacement cho frontend
Reverb Server	<code>php artisan reverb:start</code>	8080	WebSocket server cho tính năng real-time

Sau khi khởi chạy thành công, truy cập ứng dụng tại: <http://localhost:8000>

Chương 5: Kết quả và đánh giá

5.1 Kết quả kiểm thử hệ thống

Kiểm thử là một bước cực kỳ quan trọng nhằm xác minh tính đúng đắn của các chức năng, đảm bảo hệ thống vận hành ổn định và không phát sinh lỗi ngoài ý muốn. Trong dự án này, phương pháp kiểm thử tự động (Automated Testing) được áp dụng thông qua công cụ **PHPUnit** tích hợp sẵn trong **Laravel Framework v13**.

5.1.1 Môi trường kiểm thử

- **Hệ điều hành:** Windows 11 / Laragon Local Server.
- **Ngôn ngữ:** PHP 8.3.
- **Database phục vụ kiểm thử:** SQLite (Chạy hoàn toàn trên RAM - in-memory để tối ưu hóa tốc độ thực thi).
- **Công cụ thực thi:** Laravel Artisan Test Runner (php artisan test).

5.1.2 Danh mục các kịch bản kiểm thử (Test Cases)

Hệ thống sử dụng hai loại kiểm thử chính:

1. **Feature Tests (Kiểm thử chức năng/tích hợp):** Giả lập các HTTP Request gửi tới các Controller, kiểm tra Middleware, Authorization Policies và dữ liệu trả về từ Database/API.
2. **Unit Tests (Kiểm thử đơn vị):** Kiểm thử các logic nghiệp vụ nhỏ, cô lập (như kiểm tra các Helpers hoặc Event đơn lẻ).

Dưới đây là bảng tổng hợp các nhóm chức năng đã được thực hiện kiểm thử:

STT	Nhóm chức năng kiểm thử	Mục tiêu kiểm thử	Trạng thái
1	Xác thực & Bảo mật (Authentication)	Kiểm tra các luồng Đăng ký, Đăng nhập, Xác thực Email, Đặt lại mật khẩu, Xác nhận mật khẩu và Cập nhật thông tin cá nhân.	Đạt
2	Quản lý Kênh (Channel & Group)	Kiểm tra phân quyền tạo nhóm chat, quyền hiển thị thông tin nhóm, phân quyền thành viên và cơ chế đếm số tin nhắn chưa đọc (Unread Messages).	Đạt
3	Quản lý Tin nhắn (Messaging)	Kiểm tra việc gửi tin nhắn, đính kèm tệp tin (giới hạn tối đa 10 tệp, dung lượng dưới 1MB mỗi tệp), tải danh	Đạt

STT	Nhóm chức năng kiểm thử	Mục tiêu kiểm thử	Trạng thái
		sách tin nhắn phân trang bằng con trỏ (Cursor Pagination).	
4	Tác vụ nền & Dọn dẹp dữ liệu (Jobs)	Kiểm tra Job xóa nhóm (DeleteChannelJob) thực thi bất đồng bộ: ngắt liên kết thành viên, xóa bản ghi database, dọn dẹp các tệp đính kèm trong Storage.	Đạt
5	Quản trị viên (Admin Actions)	Kiểm tra quyền hạn của Admin: Tạo mới tài khoản, cập nhật thông tin thành viên, phân quyền Admin, khóa/mở khóa tài khoản người dùng.	Đạt
6	Tích hợp AI (AI Message Suggestion)	Giả lập (Mock) Google Gemini API để kiểm tra tính năng tự động gợi ý tin nhắn thông minh dựa trên nội dung chat hiện tại; kiểm tra xử lý lỗi khi thiếu API Key hoặc API mất kết nối.	Đạt

5.2 Đánh giá hệ thống

Sau quá trình phát triển và kiểm thử, hệ thống **Laravel React Messenger** được đánh giá dựa trên các tiêu chí chức năng và phi chức năng như sau:

5.2.1 Ưu điểm của hệ thống

1. Trải nghiệm người dùng mượt mà (UX/UI Premium):

- Sự kết hợp giữa **Inertia.js v2** và **React 18** mang lại trải nghiệm của một ứng dụng Single Page Application (SPA) đích thực: Chuyển trang không cần tải lại toàn bộ trình duyệt, giữ nguyên trạng thái sidebar khi chuyển đổi giữa các phòng chat.
- Giao diện hiện đại sử dụng **Tailwind CSS v4** và thư viện **DaisyUI v5** hỗ trợ Dark/Light mode linh hoạt, hiển thị đáp ứng (Responsive) tốt trên các thiết bị di động.

2. Khả năng tương tác thời gian thực vượt trội (Real-time Messaging):

- Hệ thống sử dụng **Laravel Reverb** kết hợp với **Laravel Echo** phía client giúp truyền phát tin nhắn, trạng thái hoạt động (Online Presence), các thông báo đẩy (tin nhắn mới, kênh bị xóa) tức thời với độ trễ cực thấp.

- Cơ chế **EventBus** được triển khai ở phía client giúp đồng bộ hóa dữ liệu đồng nhất giữa các Component độc lập mà không cần gọi lại API nhiều lần.

3. Tối ưu hóa hiệu năng và quản lý tài nguyên:

- **Cursor Pagination (Phân trang con trỏ)** giúp hệ thống tải tin nhắn cũ mượt mà khi người dùng cuộn lên (Infinite Scroll), tránh việc bỏ sót tin nhắn hoặc trùng lặp dữ liệu như cách phân trang truyền thống (Offset-based).
- **Eager Loading** trong Eloquent giúp giải quyết triệt để lỗi truy vấn $N+1$ khi tải danh sách kênh và các thành viên đi kèm.
- Ứng dụng cơ chế **Eloquent Observers** và **Background Jobs** bất đồng bộ để tự động dọn dẹp các tệp đính kèm vật lý trên ổ đĩa khi tin nhắn hoặc kênh bị xóa, tránh việc lãng phí bộ nhớ lưu trữ.

4. Tích hợp Trí tuệ nhân tạo (AI Integration):

- Tính năng gợi ý tin nhắn thông minh dựa trên mô hình **Gemini 3.1 Flash** giúp người dùng trả lời nhanh chóng, tăng tính tương tác và hiện đại cho ứng dụng chat.

5.2.2 Nhược điểm và hạn chế

1. **Hạn chế về các endpoint mở rộng:** Một số tính năng dọn dẹp nâng cao như xóa toàn bộ lịch sử tin nhắn của cuộc trò chuyện 1-1 chưa được định nghĩa hoàn chỉnh về mặt Routing, dẫn đến một số test case kiểm thử biên chưa đạt kết quả tối đa.
2. **Kiểm thử giao diện (E2E Testing):** Hệ thống hiện tại chỉ tập trung vào Backend Feature Testing, chưa cấu hình các công cụ kiểm thử tự động hành vi người dùng trên trình duyệt (như Cypress hay Playwright).
3. **Chưa hỗ trợ cuộc gọi Video/Audio:** Hiện tại hệ thống mới chỉ hỗ trợ chat văn bản, emoji và chia sẻ tệp tin/hình ảnh, chưa có tính năng gọi điện trực tuyến.

5.2.3 Hướng phát triển trong tương lai

- **Hoàn thiện API & Route:** Khắc phục các lỗi định tuyến còn tồn đọng trong bộ Test Suite để đảm bảo hệ thống đạt tỷ lệ kiểm thử thành công 100%.
- **Nâng cấp bảo mật tệp tin:** Triển khai mã hóa đầu cuối (End-to-End Encryption) cho các tệp đính kèm trước khi tải lên lưu trữ trên Cloud Storage (S3).
- **Tích hợp WebRTC:** Phát triển thêm tính năng cuộc gọi thoại và cuộc gọi video thời gian thực trực tiếp trên trình duyệt.

- **Mở rộng tính năng AI:** Cho phép người dùng tùy chỉnh "tông giọng" gợi ý của AI (Trang trọng, thân thiện, hài hước) hoặc dịch tin nhắn tự động đa ngôn ngữ.

Tài liệu tham khảo

Design Patterns: Elements of Reusable Object-Oriented Software. Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, 1994

Head First Design Patterns: A Brain-Friendly Guide. Eric Freeman, Bert Bates, Kathy Sierra, Elisabeth Robson. 2004

Laravel Community (2026). *Laravel - The PHP Framework for Web Artisans Documentation (v13)*.

React Core Team (2026). *React v18 Official Documentation*.

Reinink, J. & Inertia.js Contributors (2026). *Inertia.js v2 - The Modern Monolith Framework Documentation*.

Taylor Otwell & Laravel Team (2026). *Laravel Reverb - Real-time WebSocket Server for Laravel v1*.

Google AI SDK Team (2026). *Gemini API Developer Guide & Documentation*. Google AI for Developers.